

Table of Contents

1	Dati e database	1
1.1	Il dato	1
1.2	Il database relazionale	1
1.3	SQL Server Management Studio	2
1.4	Creare un database	3
1.5	DDL e DML	4
2	Primi passi nel mondo delle query	6
2.1	Standard e non standard	6
2.2	SELECT e FROM	7
2.3	La clausola WHERE	9
2.4	La clausola ORDER BY	11
2.5	Limitare il numero di risultati	13
2.6	Lo schema di una tabella	15
2.7	Gli alias	16
2.8	Delimitare gli identificatori	19
2.9	La DISTINCT	21
2.10	DISTINCT e ORDER BY, un caso delicato	22
2.11	NULL: un altro caso delicato	24
3	Query: uno sguardo più approfondito	27
3.1	Combinare condizioni nella clausola WHERE	27
3.2	IN e NOT IN	29
3.3	<, >, loro varianti e BETWEEN	30
3.4	La funzione ISNULL	31
3.5	La funzione COALESCE	32
3.6	Il comando LIKE	33
3.7	Il costrutto CASE... WHEN	34
3.8	La funzione IIF	36

3.9	Le query raggruppate - GROUP BY	37
3.10	Le query raggruppate - HAVING	39
3.11	La COUNT, qualche dettaglio in più	41
3.12	Siamo uomini o macchine?	41
4	Tipi di dati e loro manipolazione in SQL Server	47
4.1	Tipi di dati in SQL Server	47
4.2	Principali tipi di dati in SQL Server	50
4.3	Le funzioni di conversione	52
5	Una carrellata sulle funzioni di SQL Server	57
5.1	Funzioni per la gestione delle date	57
5.1.1	Funzioni per il recupero della data corrente	57
5.1.2	Funzioni per estrarre elementi da una data o comporla	58
5.1.3	Funzioni per aggiungere periodi di tempo ad una data	60
5.2	Funzioni per la gestione delle stringhe	61
5.2.1	Concatenazione di stringhe	61
5.2.2	Estrazione e ricerca di una stringa	62
5.2.3	Lunghezza di una stringa	63
5.2.4	Modifica di una stringa	64
5.2.5	Formattazione di una stringa	65
5.3	Altre funzioni utili	66
5.3.1	La funzione CHOOSE	66
5.3.2	La funzione STRING_AGG	66
5.3.3	La funzione STRING_SPLIT	67
5.3.4	La funzione NULLIF	67
5.4	Operatori Bitwise	70
5.4.1	Operatore AND	70
5.4.2	Operatore OR	70
6	DDL: creare, distruggere e modificare oggetti	72
6.1	Creare una tabella - Il comando CREATE	72
6.1.1	Dare nomi alle colonne	75
6.1.2	Assegnare il tipo ad una colonna	77
6.1.3	Valori di default	78
6.1.4	Chiavi primarie e surrogate	79
6.1.5	PRIMARY KEY e IDENTITY	81

6.1.6	Colonne calcolate	83
6.2	Modificare una tabella - Il comando ALTER	84
6.3	Eliminare una tabella - Il comando DROP	86
6.4	Svuotare una tabella - Il comando TRUNCATE	87
7	DML: creare, distruggere e modificare record	88
7.1	Il comando INSERT	88
7.1.1	INSERT VALUES	88
7.1.2	INSERT SELECT	91
7.2	Il comando UPDATE	92
7.3	Il comando DELETE	93
7.4	Differenze tra DELETE e TRUNCATE	94
8	Interrogare più tabelle contemporaneamente	96
8.1	Query di più tabelle senza join	96
8.1.1	Il comando INTERSECT	96
8.1.2	Il comando EXCEPT	99
8.2	Le join	100
8.2.1	La INNER JOIN	101
8.2.2	La LEFT JOIN	104
8.2.3	La RIGHT JOIN	107
8.2.4	La FULL JOIN	108
8.2.5	Altri tipi di Join	109
8.2.6	La CROSS JOIN	112
8.3	Le subqueries - altri modi di combinare tabelle senza JOIN	114
8.3.1	Self-contained Subqueries	114
8.3.2	Subquery Correlate	116
8.3.3	Le Derived Tables	117
8.4	Le Common Table Expressions	119
8.4.1	Le CTE ricorsive	125
8.5	Le APPLY	128
8.5.1	La CROSS APPLY	128
8.5.2	La OUTER APPLY	130
8.5.3	Differenze tra CROSS APPLY e OUTER APPLY	131
9	Rafforzare l'integrità dei dati	133
9.1	Le chiavi primarie	134

9.2	I vincoli unique	138
9.3	Le chiavi esterne	139
9.4	Altri tipi di constraint	143
9.4.1	Check constraint	143
9.4.2	Default constraint	143
10	Lo stesso, ma su SSMS	145
10.1	Creare un nuovo database	145
10.2	Creazione di una nuova tabella	146
10.3	Modifica di una tabella	148
11	Le viste	150
11.1	L'opzione WITH SCHEMABINDING	152
11.2	UPDATE su viste	153
11.3	L'opzione WITH CHECK OPTION	153
11.4	Restrizioni per le viste	154
11.5	Modificare una vista	155
11.6	Eliminare una vista	155
12	Query avanzate	157
12.1	DML sfruttando le JOIN	157
12.1.1	UPDATE tramite JOIN	157
12.1.2	DELETE tramite JOIN	158
12.2	Uso della PIVOT	158
12.3	Uso della UNPIVOT	162
12.4	Utilizzo non banale delle funzioni MIN e MAX	163
12.4.1	Come ricavare MIN e MAX da una serie di valori	163
12.4.2	Come ricavare MIN e MAX da una serie di colonne	164
12.4.3	MIN e MAX raggruppati	165
12.5	La funzione MERGE	166
13	Funzioni utili	169
13.1	Le funzioni di rango	169
13.1.1	La funzione ROW_NUMBER	170
13.1.2	La funzione RANK	172
13.1.3	La funzione DENSE_RANK	173
13.1.4	Il comando NTILE	174

13.2	Come gestire la paginazione	174
13.2.1	FETCH NEXT	174
13.3	Riferimenti ad altre righe in una query	174
13.4	Il comando LAG	174
13.5	Conteggi sfasati	174
13.6	Per generare una stringa univoca	174
13.6.1	La funzione SCOPE_IDENTITY	175
13.7	Per la manipolazione delle date	175
13.7.1	DATEDIFF	175
13.7.2	DATEPART	175
13.7.3	DATENAME	175
13.7.4	DATEFROMPARTS	175
13.7.5	Per ricavare le singole parti di una data	175
13.7.6	EOMONTH	175
13.7.7	DATE_BUCKET	175
13.7.8	DATETRUNC	175
13.7.9	ISDATE	175
13.8	Per la manipolazione delle stringhe	175
13.8.1	La funzione PATINDEX	175
13.8.2	La funzione CHARINDEX	175
13.8.3	La funzione LEN	175
13.8.4	La funzione DATALENGTH	175
13.8.5	La funzione REPLICATE	175
13.8.6	La funzione CHOOSE	175
13.8.7	La funzione SPACE	175
13.8.8	La funzione STUFF	175
13.8.9	Da stringa a tabella, da tabella a stringa	175
13.8.10	Questa è una sottosezione	176
13.9	Questa è una sezione labellata	176
14	Tabelle Temporanee	178
15	Disegno del database	179
16	Gestione degli indici	180
17	La concorrenza	181

18 Conclusion and Future Work	182
18.1 Project summary	182
18.2 Problems faced and lessons learned	182
18.3 Future work	182

Capitolo 1

Dati e database

1.1 Il dato

In informatica, il termine "dato" si riferisce a una rappresentazione simbolica di informazione sotto forma di numeri, testo, immagini, suoni o qualsiasi altra forma che può essere elaborata da un computer. I dati costituiscono il fondamento di tutte le operazioni di elaborazione delle informazioni e sono il materiale grezzo utilizzato per generare conoscenza.

Se questi dati sono organizzati in una struttura tabellare o in altre strutture di dati definite, come database relazionali, sono detti **strutturati**.

1.2 Il database relazionale

Il concetto di "relazione" che sta alla base del database relazionale non è, come spesso si crede, legato alla relazione che può collegare una tabella con un'altra, ma alla relazione che intercorre tra l'intestazione di una tabella (ovvero il nome delle colonne), e il suo contenuto (ovvero i record). Anche un database formato da una sola tabella, in questo senso, può essere considerato un database *relazionale*.

Il più noto sistema di gestione dei database relazionali di Microsoft (RDBMS) si chiama Microsoft SQL Server, e offre una vasta gamma di funzionalità per la gestione, l'archiviazione, l'interrogazione e la protezione dei dati. In queste dispense imparere-

mo ad utilizzare le più importanti funzionalità di SQL Server attraverso l'utilizzo del suo linguaggio di interrogazione, il T-SQL, in **SQL Server Management Studio**.

1.3 SQL Server Management Studio

SQL Server Management Studio (SSMS) è un ambiente integrato di sviluppo e amministrazione utilizzato per gestire e interagire con i database di Microsoft SQL Server. È un'applicazione grafica avanzata che fornisce una vasta gamma di strumenti per la creazione, l'amministrazione, il monitoraggio e la gestione dei database SQL Server.

Per prima cosa occorre aprire SQL Server Management Studio, inserire il nome dell'istanza a cui ci si vuole collegare, e scegliere un tipo di autenticazione.

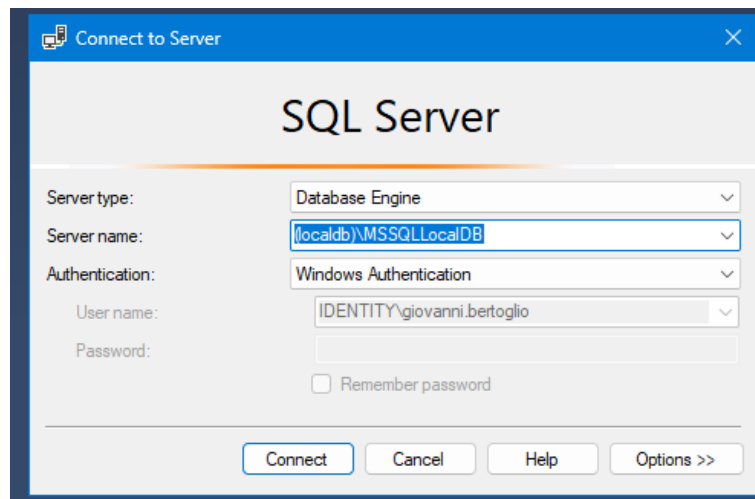


Figura 1.1: Accesso a SSMS

In alternativa, ci si potrà connettere selezionando **Object Explorer > Connect > Database Engine**.

Per verificare che la connessione al server sia riuscita, espandi ed esplora gli oggetti all'interno dell'Esplora oggetti dove sono visualizzati il nome del server, la versione di SQL Server e il nome utente. Questi oggetti sono differenti a seconda del tipo di server.

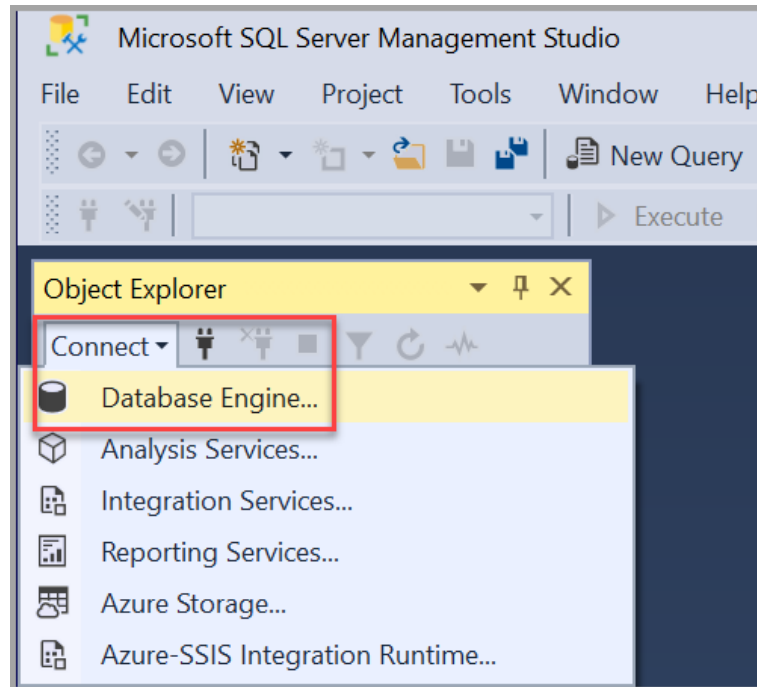


Figura 1.2: Accesso a SSMS

1.4 Creare un database

Fai clic con il pulsante destro del mouse sull'istanza del tuo server in Esplora oggetti, quindi seleziona "New Query":

Ora creiamo un database chiamato Academy seguendo i passaggi seguenti:

Incolla il seguente frammento di codice T-SQL nella finestra di query:

```
1 USE [master];
2 GO
3 IF NOT EXISTS (
4     SELECT name
5     FROM sys.databases
6     WHERE name = N'Academy')
7
8     CREATE DATABASE Academy;
9 GO
```

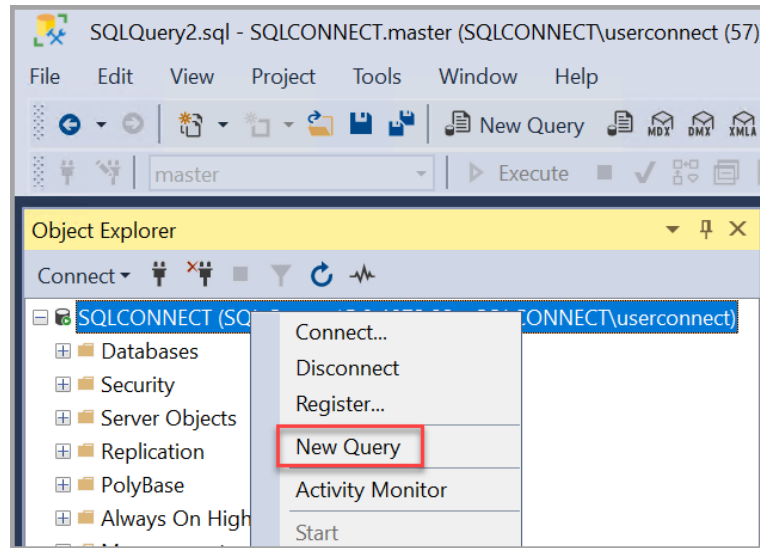


Figura 1.3: Crea una nuova query

Esegui la query cliccando su Esegui; se siete dei keyboard guys, potete ottenere lo stesso risultato digitando F5 oppure CTRL + E.

1.5 DDL e DML

Possiamo distinguere due principali categorie di azioni che possiamo compiere all'interno di un database, ovvero **DDL** (Data Definition Language) e **DML** (Data Modification Language).

Con DDL intendiamo l'insieme delle operazioni che vengono utilizzate per creare e modificare le strutture degli oggetti del database come tabelle, viste, schemi, indici etc

- **CREATE**: Crea una nuova tabella, una vista di una tabella o un altro oggetto nel database
- **ALTER**: Modifica un oggetto esistente nel database, come una tabella
- **DROP**: Elimina un'intera tabella, una vista di una tabella o altri oggetti nel database
- **TRUNCATE**: Svuota completamente una tabella in un'unica operazione

Con DML si intende invece l'insieme delle operazioni che vengono utilizzate per aggiungere, eliminare e modificare dati in un database

- **SELECT:** Recupera determinati record da una o più tabelle
- **INSERT:** Crea uno o più record in una tabella
- **UPDATE:** Modifica i record in una tabella
- **DELETE:** Cancella i record in una tabella

Ciascuna di queste operazioni verrà poi analizzata in dettaglio nei capitoli seguenti.

Capitolo 2

Primi passi nel mondo delle query

In questo capitolo verrà introdotto il linguaggio T-SQL, utilizzato per scrivere query e manipolare dati su SQL Server, verranno illustrate le sue somiglianze e differenze con il linguaggio SQL e verranno introdotti alcuni fondamentali concetti sul database.

2.1 Standard e non standard

SQL (si pronuncia *see·kw1*) è un acronimo che sta per **Structured Query Language**, ovvero linguaggio di interrogazione strutturata. Si tratta di un linguaggio di programmazione utilizzato per gestire i dati nei database relazionali come MySQL, MS Access, Oracle, e, ovviamente, SQL Server. SQL è stato sviluppato da IBM negli anni '70 ed è sia uno standard ISO (International Organization for Standards), sia uno standard ANSI (American National Standards Institute); ciascuna azienda ha poi sviluppato il proprio linguaggio di programmazione, che per il RDBMS di Microsoft, è per l'appunto T-SQL, in cui la T sta per **Transact**.

Se SQL è un linguaggio, il T-SQL si può definire come un suo dialetto, che lo amplia e lo estende, ma a differenza di SQL, T-SQL non è uno standard. Per questo motivo, quando siamo di fronte a due modi per fare la stessa cosa, uno tipico del solo T-SQL e un altro che è uno standard SQL, è sempre preferibile scegliere il secondo. Banalmente, in T-SQL ci sono due modi per scrivere l'operatore "diverso", ovvero `<>`, oppure `!=`, ma, mentre il primo corrisponde allo standard, il secondo no, quindi il primo è sempre il modo preferibile.

Un'altra buona pratica di scrittura del codice T-SQL consiste nel terminare le istruzioni con il punto e virgola: se non lo si fa non viene sollevato nessun errore e il codice funzionerà ugualmente, ma anche in questo caso è buona pratica attenersi allo standard, specie perché in alcune situazioni saremo costretti ad usarlo (vedi, più avanti, la sezione 8.4, dedicata alle CTE), quindi tanto vale abituarsi fin da subito.

2.2 SELECT e FROM

Vediamo con un esempio pratico la query su una tabella di anagrafica: potete crearla facilmente nel database "Academy", eseguendo lo script a questo link.

Tabella 2.1: Tabella "Hobbits"

Nome	Cognome
Bilbo	Baggins
Frodo	Baggins
Sam	Gamgee
Merry	Brandybuck
Pippin	Tuck

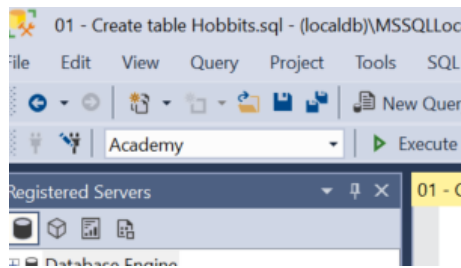
la nostra prima query tira fuori in modo molto banale l'intero contenuto della tabella "Hobbits"

```
1 USE Academy;  
2  
3 SELECT *  
4 FROM Hobbits;
```

Quick tip: sia i comandi sia i nomi degli oggetti sono case insensitive

La prima riga specifica semplicemente il database sul quale vogliamo eseguire la query. In mancanza di indicazioni, probabilmente il default sarà il database **master**, che è un database di sistema, ed è poco consigliabile (quando non impossibile) crearvi tabelle. Per vedere in qualsiasi momento su quale database stiamo lavorando possiamo guardare in alto a sinistra sulla nostra finestra di SSMS. Un altro metodo per

cambiare il database è proprio scriverlo nel box o sceglierlo dalla lista che compare cliccando sulla freccia in basso, a lato del box.



Quick tip: Per i keyboard guys, potete raggiungere il box con il nome del database con il comando **CTRL + U**.

Una volta scelto il database su cui vogliamo lavorare in uno dei modi possibili appena citati, non sarà più necessario specificarlo nelle query successive.

Tornando alla nostra query, possiamo fin da subito notare una particolarità, ovvero l'uso di *. Nell'informatica, come nella vita, se una cosa è particolarmente comoda, probabilmente non è opportuno usarla, e questo caso non fa eccezione. Dopo il comando **SELECT**, occorre inserire l'elenco delle colonne che si vogliono visualizzare dalla tabella indicata dopo il **FROM**. Quando usiamo l'asterisco stiamo implicitamente dicendo "Dammi *tutte* le colonne dalla tabella X". Molto più corretto (anche se leggermente più noioso), sarebbe scrivere in questo modo:

SELECT Nome, Cognome

FROM Hobbits

quindi, esplicitando il nome di ciascuna colonna, separandoli con delle virgole. Il risultato dell'interrogazione è assolutamente identico, ma la query è decisamente più leggibile e manutenibile.

In dettaglio:

Quando si utilizza **SELECT ***, il database deve recuperare tutte le colonne dalla tabella, anche quelle che potrebbero non essere necessarie per la query in que-

stione. Questo può causare un maggiore utilizzo di risorse del server e rallentare le prestazioni, specialmente su tabelle con molte colonne o su reti con larghezza di banda limitata.

Se il nome delle colonne cambia o vengono aggiunte nuove colonne alla tabella nel tempo, le query che utilizzano **SELECT *** potrebbero restituire risultati imprevisti o errati senza che l'utente se ne accorga. Questo può portare a errori difficili da individuare.

Specificare esplicitamente le colonne desiderate rende il codice più leggibile e comprensibile per gli sviluppatori e per chiunque debba mantenere o modificare il codice in futuro. Aiuta anche a evitare la duplicazione di colonne se più tabelle nel database hanno nomi di colonne simili o uguali.

In alcuni scenari di sicurezza, potrebbe essere necessario nascondere o proteggere alcune colonne dei dati. Utilizzando **SELECT ***, si potrebbe incorrere nell'esposizione di informazioni sensibili o riservate.

2.3 La clausola WHERE

Quello che abbiamo fatto con la query precedente è stato richiedere l'intero contenuto di un elenco di colonne, ma come potremmo fare se volessimo visualizzare solo alcune righe, per esempio conoscere il nome di battesimo dell'hobbit con cognome 'Gamgee'?

In questo caso la query assumerebbe questo aspetto

```
SELECT Nome
FROM Hobbits
WHERE Cognome = 'Gamgee';
```

eseguendo la query, otterremo come risultato

<u>Nome</u>
Sam

Se quindi dal **FROM** stiamo indicando quale tabella desideriamo interrogare, dal **SELECT** quali colonne di questa tabella ci interessa vedere, con il **WHERE** stiamo imponendo una condizione sulle righe. Si può facilmente notare come sia possibile imporre nella **WHERE** il vincolo su di una colonna che non viene effettivamente restituita nei risultati.

L'insieme più grande è dato dalla tabella (incorniciata in verde), e il risultato che vogliamo ottenere è dato dall'incrocio delle colonne (incorniciate in blu) e delle righe (incorniciate in rosso).

Nome	Cognome
Bilbo	Baggins
Frodo	Baggins
Merry	Brandvuck
Sam	Gamgee
Pippin	Took

La query che abbiamo appena eseguito estrae una sola riga e una sola colonna, ma ovviamente non c'è limite alla fantasia, e siamo liberi di selezionare tutte le colonne che vogliamo e imporre tutti i vincoli che riteniamo opportuni.

Notare come la stessa query di prima, con **'Baggins'** nella condizione di **WHERE**, avrebbe tirato fuori due righe, cioè **'Bilbo'** e **'Frodo'** anziché una.

In dettaglio:

Il T-SQL è un linguaggio di programmazione dichiarativo. Un linguaggio dichiarativo è un tipo di linguaggio di programmazione in cui gli sviluppatori specificano ciò che dev'essere fatto, ma non come farlo.

Nella query che abbiamo appena creato, abbiamo fatto una richiesta *senza specificare come il database dovesse effettivamente eseguire questa selezione*. Il database è libero di utilizzare l'approccio più efficiente per ottenere i risultati desiderati,

ad esempio eseguendo una scansione della tabella o utilizzando un indice, senza che l'utente debba preoccuparsi dei dettagli implementativi.

2.4 La clausola ORDER BY

Prima di cominciare a complicare un po' le cose, è importante chiarire bene un concetto: se non specifichiamo esplicitamente nella query in quale ordine desideriamo visualizzare i risultati, il database non è tenuto a restituirli seguendo un criterio particolare. Anche eseguendo due volte la stessa query non abbiamo *nessuna* garanzia di vedere le righe nello stesso ordine entrambe le volte. Il database può decidere di restituire i dati nell'ordine in cui sono fisicamente memorizzati o in qualsiasi altro ordine considerato ottimale per l'esecuzione della query.

Se vogliamo che i dati vengano visualizzati in un ordine particolare, dobbiamo specificare la clausola `ORDER BY` alla fine della query, dopo il `WHERE`.

```
SELECT Nome, Cognome
FROM Hobbits
WHERE Cognome <> 'Gamgee'
ORDER BY Nome;
```

Se eseguiamo questa query, troveremo come risultato

Tabella 2.2: Tabella "Hobbits"

Nome	Cognome
Bilbo	Baggins
Frodo	Baggins
Merry	Brandybuck
Pippin	Took

Si può notare come, se non specifichiamo nulla dopo la clausola `ORDER BY`, l'ordine di default sia quello ascendente. Per evitare qualunque tipo di ambiguità è sempre consigliabile specificare l'ordine desiderato, in questo modo

```
ORDER BY Nome ASC;
```

oppure

```
ORDER BY Nome DESC;
```

Se vogliamo che l'ordine vada dal valore più grande al più piccolo.

In dettaglio:

È già arrivato il momento di inserire una nuova sintassi tanto comoda quanto deprecata: la query di prima, scritta in questo modo

```
SELECT Nome, Cognome
FROM Hobbits
WHERE Cognome <> 'Gamgee'
ORDER BY 1;
```

restituisce esattamente il risultato desiderato, dato che, con il comando `ORDER BY 1`, stiamo dicendo al database "ordina rispetto alla prima colonna della select". Allo stesso modo avremmo potuto indicare `ORDER BY 2`, se avessimo voluto ordinare per cognome.

Come si può facilmente intuire, è possibile inserire criteri di ordinamento anche su altre colonne: in questo caso, infatti, a parità di cognome 'Baggins', abbiamo due record, 'Frodo' e 'Bilbo'. Esattamente come succedeva quando non avevamo nessuna condizione di ordinamento, se abbiamo una colonna per cui non abbiamo specificato l'ordine, a parità di risultati sulla colonna ordinata, i risultati sulla colonna non ordinata saranno esposti secondo un ordine non deterministico. Se ordiniamo per cognome come in questo caso, quindi, potremmo visualizzare come primo risultato 'Frodo' o 'Bilbo' indifferentemente. Per ovviare questo problema si può aggiungere un'ulteriore condizione di ordinamento, ottenendo una query con questo aspetto:

```
SELECT Nome, Cognome
FROM Hobbits
WHERE Cognome <> 'Gamgee'
ORDER BY Cognome ASC, Nome DESC;
```

ottenendo come risultato

Nome	Cognome
Frodo	Baggins
Bilbo	Baggins
Merry	Brandybuck
Pippin	Took

Ovviamente, se ordiniamo per una colonna che abbiamo già la garanzia contenga solo risultati univoci, l'ordinamento per altre colonne sarà del tutto superfluo.

2.5 Limitare il numero di risultati

Un comando che viene spesso utilizzato in combinazione con la `ORDER BY` è il comando `TOP`: usato in una query serve a limitare il numero di risultati che la query presenterà, ad esempio

```
SELECT TOP (3) Nome, Cognome
FROM Hobbits
ORDER BY Cognome ASC;
```

darà questo output

Nome	Cognome
Frodo	Baggins
Bilbo	Baggins
Merry	Brandybuck

Per ragioni di retrocompatibilità è consentito anche scrivere

```
SELECT TOP 3 Nome, Cognome ...
```

, ma la sintassi corretta è quella con le parentesi.

In dettaglio: al posto del numero definito delle righe, possiamo anche chiedere che ci venga restituita una determinata percentuale delle righe totali, in questo modo

```
SELECT TOP (10) PERCENT Nome, Cognome ...
```

, dove il valore tra parentesi è compreso tra 0 e 100. Il numero di righe presentato è sempre arrotondato per eccesso. Ad esempio, se la query senza il comando **TOP** restituisse 710 righe totali, e usassi **SELECT TOP (1) PERCENT**, otterrei 7.1 righe, che verrebbero arrotondate a 8.

Sempre per il fatto che, in assenza della clausola **ORDER BY**, i risultati di una query sono espressi in un ordine non deterministico, diventa cruciale specificarla in casi come questo (benché non sia obbligatorio), per evitare di ottenere risultati casuali. Sempre per questo motivo, è opportuno eseguire l'ordinamento su colonne significative: una query come questa:

```
SELECT TOP (1) Nome  
FROM Hobbits  
ORDER BY Cognome ASC;
```

Potrebbe restituire una volta "Frodo" e un'altra volta "Bilbo".

SQL Server offre anche un altro modo per aggirare il problema, ovvero la clausola **WITH TIES**: utilizzando questa clausola nella query di prima, nonostante abbiamo indicato una **TOP (1)**, otterremo due risultati, dato che i primi due cognomi della nostra tabella in ordine alfabetico sono uguali.

```
SELECT TOP (1) WITH TIES Nome  
FROM Hobbits  
ORDER BY Cognome ASC;
```

Cognome

Bilbo

Frodo

La clausola **WITH TIES** in T-SQL deve essere necessariamente utilizzata insieme alla clausola **TOP** in una query. La conseguenza è che verranno incluse tutte le righe che soddisfano il criterio di ordinamento massimo (o minimo) definito, anche se ciò

comporta il recupero di più righe rispetto a quelle specificate nella clausola **TOP**. In altre parole, **WITH TIES** garantisce che vengano restituite tutte le righe che hanno lo stesso valore di ordinamento della riga "limite" definita da **TOP**.

2.6 Lo schema di una tabella

Se il nome di una tabella è paragonabile al suo nome di battesimo, il suo schema è paragonabile al cognome. Si tratta di un metodo semplice ed efficace di organizzare le tabelle, raggruppandole sotto le stesse aree tematiche. Se non viene specificato nessuno schema, la tabella viene automaticamente creata sotto lo schema **dbo**, che sta per "database object", un nome che più generico di così non si può. Se proviamo ad eseguire questa query

```
SELECT Nome, Cognome  
FROM dbo.Hobbits;
```

troviamo esattamente gli stessi risultati che abbiamo ottenuto senza specificare lo schema, proprio perché **dbo** è lo schema di default (default che volendo può essere anche cambiato). Inutile dire che è buona pratica creare degli schema chiari e che definiscano bene le tabelle sotto di loro, nonché creare tabelle sotto questi schema in modo coerente. È altrettanto consigliabile indicare sempre lo schema di una tabella quando la si interroga, ma la cosa dovrebbe venire da sé, dato che, se una tabella appartiene ad uno schema diverso da **dbo**, specificare lo schema è obbligatorio, e abbiamo visto che creare tabelle sotto lo schema **dbo** non è una buona pratica.

Lo schema può essere utilizzato per applicare controlli di accesso granulari alle tabelle: è possibile concedere o negare l'accesso a uno schema specifico alle diverse categorie di utenti. Questa prassi è utile per implementare una sicurezza basata sui ruoli.

Un altro vantaggio di un utilizzo accorto degli schema è per evitare conflitti tra i nomi delle tabelle: se ho due tabelle con lo stesso nome, inserirle sotto due schemi diversi risolve il problema.

Vedremo più avanti la sintassi sia per creare uno schema, sia per creare una tabella all'interno di uno schema.

2.7 Gli alias

Gli alias in T-SQL sono utilizzati per fornire nomi alternativi temporanei alle colonne e alle tabelle all'interno di una query. Gli alias rendono il codice più leggibile, consentendo agli sviluppatori di assegnare nomi più brevi o descrittivi alle colonne e alle tabelle senza dover modificare gli schemi di database sottostanti.

Vediamo come applicare questo concetto ad una delle query che abbiamo incontrato in precedenza:

```
SELECT Nome AS NomeHobbit, Cognome
FROM dbo.Hobbits
WHERE Cognome = 'Baggins';
```

Ciò che abbiamo fatto è stato dare un nome diverso alle colonne che mostrano i risultati della nostra query, ottenendo questo

NomeHobbit	Cognome
Bilbo	Baggins
Frodo	Baggins

È possibile snellire la sintassi omettendo la parola **AS**, ma è una pratica che non aiuta la lettura e presta il fianco ad errori. Se per esempio ci dimentichiamo di mettere una virgola tra una colonna e l'altra, otterremo un risultato errato rispetto a quello che ci aspettiamo, ovvero una sola colonna anziché due, e per di più con un nome che non ne rappresenta il contenuto. Questa query, infatti

```
SELECT Nome Cognome
FROM dbo.Hobbits
WHERE Cognome = 'Baggins';
```

dà come risultato

Cognome
Bilbo
Frodo

Se abbiamo l'abitudine di separare il nome della colonna dal suo alias con l'**AS**, errori di questo genere diventano molto più facili da individuare.

Quick tip: se proviamo a mettere nella condizione di **WHERE** l'alias della colonna, anziché il nome della colonna, in questo modo

```
SELECT Nome AS NomeHobbit, Cognome
FROM dbo.Hobbits
WHERE NomeHobbit = 'Merry';
```

riceveremo questo errore:

```
Msg 207, Level 16, State 1, Line 1
Invalid column name 'NomeHobbit'.
```

Questo perché, nell'esecuzione di una query, il database "vede" la clausola **WHERE** *prima* di prendere in considerazione quali colonne vogliamo ottenere dalla **SELECT**.

Ma vedremo questa particolarità in dettaglio più avanti.

L'assegnazione degli alias al nome di tabella (per quanto, al momento attuale, sembra un esercizio un po' sterile), funziona più o meno nello stesso modo

```
SELECT Nome, Cognome
FROM dbo.Hobbits AS Hb;
```

E anche in questo caso è possibile omettere la parola **AS**. A differenza di quanto accadeva per i nomi delle colonne, l'alias della tabella non ha alcun effetto nel risultato della query, tanto che ci si potrebbe chiedere quale utilità possa avere. L'utilità sarà

evidente nel momento in cui affronteremo query che attingeranno a più tabelle contemporaneamente, ma per il momento basti sapere che è sempre possibile specificare da quale tabella proviene la colonna di una query, inserendo l'alias di tale tabella prima del nome della colonna, in questo modo:

```
SELECT Hb.Nome, Hb.Cognome  
FROM dbo.Hobbits AS Hb;
```

Questo modo di scrivere una query è possibile anche in assenza dell'alias della tabella, ovvero

```
SELECT Hobbits.Nome, Hobbits.Cognome  
FROM dbo.Hobbits;
```

ma è un tipo di scrittura che risulta sicuramente più pesante rispetto all'utilizzare gli alias.

Quick tip: se proviamo ad utilizzare il nome di tabella originale prima delle colonne, ma abbiamo dato un alias alla tabella, riceveremo un errore. Una volta dato un alias a una tabella, è come se il database, per quella query, sostituisse completamente il nome originale.

Se proviamo ad eseguire la query seguente:

```
SELECT Hobbits.Nome, Hb.Cognome  
FROM dbo.Hobbits AS Hb;
```

otterremo questo errore:

Msg 4104, Level 16, State 1, Line 1

The multi-part identifier "Hobbits.Nome" could not be bound.

Proprio perché il nome tabella "Hobbits", per il database, dopo che le è stato assegnato un alias, non ha più significato.

2.8 Delimitare gli identificatori

Sempre parlando di alias, come potremmo fare nel caso in cui ci venisse chiesto di scrivere una query che mostrasse la colonna "Cognome" con l'alias "Cognome Hobbit" (quindi con uno spazio tra le due parole)?

L'esecuzione della query seguente

```
SELECT Cognome AS Cognome Hobbit
FROM dbo.Hobbits;
```

darebbe luogo ad un errore:

```
Msg 102, Level 15, State 1, Line 1
Incorrect syntax near 'Hobbit'.
```

In questo caso dobbiamo ricorrere all'uso delle parentesi quadre, che indicano come i due oggetti separati da spazio siano in realtà un oggetto solo

```
SELECT Cognome AS [Cognome Hobbit]
FROM dbo.Hobbits;
```

Scritta in questo modo, la query mostra il risultato desiderato, senza errori.

Allo stesso modo le parentesi possono essere utilizzate per "neutralizzare" nomi che corrispondono a parole riservate del linguaggio SQL. Se ricevessimo il compito di visualizzare una colonna in una query che abbia come alias "FROM", dovremmo mettere anch'esso tra parentesi, per evitare di incorrere in errori.

```
SELECT Cognome AS [FROM]
FROM dbo.Hobbits;
```

Questa pratica si applica anche ai nomi "veri" di tabelle o di colonne: se per esempio avessimo una tabella, o una colonna, chiamata "Order", dovremmo applicare le quadre anche in questo caso.

```
SELECT [Order]
FROM dbo.Tabella;
```

oppure

```
SELECT Colonna
FROM dbo.[Order];
```

Un altro contesto in cui le parentesi quadre evitano problemi è quando l'oggetto in questione non rispetta una delle regole standard per la nomenclatura di T-SQL, ad esempio, cominciare con un numero o contenere caratteri speciali.

```
SELECT Cognome AS [1name]
FROM dbo.Hobbits;
```

oppure

```
SELECT Cognome AS [Nome?]
FROM dbo.Hobbits;
```

Nulla vieta, se si vuole, di mettere tra parentesi quadre anche ad oggetti che normalmente non lo richiederebbero. Alcuni sviluppatori adottano questo metodo come standard, ma personalmente lo uso solamente quando è necessario, per evitare di vedere cose di questo tipo, probabilmente un po' esagerate, anche se perfettamente lecite.

```
SELECT [Hb].[Cognome] AS [Cognome_Hobbit]
FROM [dbo].[Hobbits] AS [Hb];
```

In ogni caso il suggerimento è sempre quello di evitare nomi particolarmente astrusi.

Quick tip: Se tornate alla sezione 1.4, potrete vedere che le parentesi quadre sono già comparse nello script utilizzato per creare il database "Academy".

2.9 La DISTINCT

Poniamo di voler avere un elenco di tutti i cognomi della nostra tabella "Hobbits", sembra un compito banale:

```
SELECT Cognome  
FROM dbo.Hobbits;
```

Otterremo:

<u>Cognome</u>
Baggins
Baggins
Gamgee
Brandybuck
Took

Ma il risultato non è ottimale, visto che c'è la ripetizione del cognome "Baggins", di fatto inutile: affermare due volte una cosa vera non la rende più vera. Come potrei fare quindi per ottenere tutti i risultati distinti di una query? Posso aggiungere il comando DISTINCT in questo modo:

```
SELECT DISTINCT Cognome  
FROM dbo.Hobbits;
```

Che restituirà:

<u>Cognome</u>
Baggins
Gamgee
Brandybuck
Took

Il comando `DISTINCT` elimina i valori duplicati su *tutte* le colonne esposte da una query, difatti, eseguendo questo comando

```
SELECT DISTINCT Nome, Cognome
FROM dbo.Hobbits;
```

otterremo lo stesso risultato che avremmo ottenuto eseguendo la query senza la `DISTINCT`, dato che ciascuna coppia dei valori (Nome, Cognome) è univoca all'interno della tabella.

Quick tip: la `DISTINCT` è un'operazione potenzialmente molto costosa in termini di consumo delle risorse, specie se parliamo di query che restituiscono un numero elevato di righe; pertanto va utilizzata con parsimonia, e solo quando è davvero necessaria.

2.10 `DISTINCT` e `ORDER BY`, un caso delicato

Proviamo ad eseguire questa query, in cui abbiamo utilizzato la clausola `DISTINCT` in combinazione con `ORDER BY`:

```
SELECT DISTINCT Cognome
FROM dbo.Hobbits
ORDER BY Nome ASC;
```

All'apparenza sembra del tutto innocua: stiamo semplicemente chiedendo i valori distinti della colonna cognome, ordinando i risultati per la colonna nome. Eppure, se la eseguiamo, riceviamo un messaggio d'errore:

```
Msg 145, Level 15, State 1, Line 1
```

```
ORDER BY items must appear in the select list if
SELECT DISTINCT is specified.
```

Cerchiamo di capire qual è il motivo dell'errore: nella nostra tabella `Hobbits`, ci sono Frodo e Bilbo che hanno lo stesso cognome, cioè `Baggins`. Poniamo di aver inserito

un nuovo record, ovvero Daisy Boffin. Questo record, nell'ordinamento per nome, si troverà tra i due.

NomeHobbit	Cognome
Bilbo	Baggins
Daisy	Boffin
Frodo	Baggins
Merry	Brandybuck
Pippin	Took
Sam	Gamgee

Se ora però volessimo fare una `distinct` ordinata dei soli cognomi, cosa vedremmo? La sequenza Baggins, Boffin, Brandybuck etc. oppure Boffin, Baggins, Brandybuck etc.? Il database non si prende questa responsabilità, e pertanto solleva l'eccezione.

È interessante notare che, anche se nella tabella `Hobbit` avessimo avuto tutti cognomi diversi, la query avrebbe dato ugualmente errore. Semplicemente, in presenza di una `DISTINCT`, è sempre richiesto di visualizzare nella `SELECT` la colonna per cui si fa l'ordinamento.

Ciò non toglie che invece una query scritta in questo modo non dia problemi di sorta:

```
SELECT Cognome
FROM dbo.Hobbits
ORDER BY Nome ASC;
```

, mostrando come risultato:

Cognome

Baggins
Boffin
Baggins
Brandybuck
Took
Gamgee

(notare il record duplicato)

2.11 NULL: un altro caso delicato

Il **NULL** in T-SQL rappresenta l'assenza di un valore in una colonna di database, può essere applicato a qualsiasi tipo di dato, inclusi interi, decimali, date, stringhe, etc. È importante comprendere che **NULL** non è la stessa cosa di una stringa vuota o di un valore numerico zero. Ad esempio, se una colonna **Nome** contiene una stringa vuota, significa che il nome è stato fornito come vuoto, mentre se contiene **NULL**, potrebbe indicare che il nome non è stato fornito o è sconosciuto. La differenza è sottile ma molto significativa.

Possiamo interpretare il **NULL** come "sconosciuto", e in quanto tale non è possibile paragonare due valori sconosciuti tra di loro. Sarebbe come dire: "Ho ricevuto una telefonata da un numero sconosciuto" e poi "L'autore dell'omicidio è sconosciuto" e arrivare alla conclusione: "Ho ricevuto una telefonata dall'autore dell'omicidio". Se mettiamo a confronto un **NULL** con un altro **NULL**, l'esito dà sempre "diverso".

È possibile specificare, in fase di creazione di una tabella, se una determinata colonna accetta o no la presenza di valori nulli. Di default, se non viene specificato, questi sono consentiti. Nel caso della nostra tabella **Hobbits** possiamo difatti inserire dei valori nulli sia nel nome, sia nel cognome.

Poniamo caso di aver inserito nella tabella un record in cui abbiamo solo il nome, ad es. "Gollum", senza specificare il cognome.

```
SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE Nome = 'Gollum';
```

l'output è

Nome	Cognome
Gollum	NULL

Ma cosa succede se proviamo a cercare di estrarre i record che hanno cognome NULL con questa query?

```
SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE Cognome = NULL;
```

Scopriamo che l'esecuzione non dà nessun errore, ma al tempo stesso non tira fuori nemmeno un valore. Questo perché gli operatori di confronto '=', '<>', '>', '<', etc. non possono essere utilizzati quando in gioco c'è NULL. Allora come possiamo modificare la query di prima in modo che tiri fuori tutti i record che hanno cognome NULL? Bisogna utilizzare funzioni specifiche come IS NULL oppure IS NOT NULL.

Se infatti eseguiamo

```
SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE Cognome IS NULL;
```

otterremo proprio il risultato che stavamo cercando.

Quick tip: Come viene trattato il NULL in un ordinamento? Proprio in virtù della sua natura ambigua, non c'è un modo giusto o sbagliato per gestirlo: in

SQL Server è stato stabilito arbitrariamente che i NULL siano messi tutti quanti prima dei valori NOT NULL.

Questa query, infatti,

```
SELECT TOP (2) Nome, Cognome
FROM dbo.Hobbits
ORDER BY Cognome ASC;
```

dà come risultato

Nome	Cognome
Gollum	NULL
Frodo	Baggins

Quick Tip: Anche se siamo appena all'inizio del nostro percorso, abbiamo già visto diversi esempi di come il codice può essere scritto in maniera ottimale oppure no. Nella query che segue ci sono ben sette aspetti che potrebbero essere migliorati, riesci a individuarli tutti?

```
SELECT *, Nome NomeHobbit
FROM Hobbits
WHERE Cognome != 'Gamgee'
ORDER BY 1
```

*Soluzione: l'utilizzo di * invece che i nomi estesi delle colonne, aver messo != anziché <>, l'uso della sintassi ORDER BY 1 al posto del nome di colonna, l'assenza dell'indicazione ASC al fondo, la mancanza del punto e virgola alla fine della query, l'assenza dell'indicazione dello schema della tabella e di AS prima dell'alias*

Capitolo 3

Query: uno sguardo più approfondito

In questo capitolo andremo un po' più in profondità nello scrivere le query: scopriremo come combinare condizioni tra di loro, raggruppare risultati, filtrare i dati in modo un po' più sofisticato, e impareremo a conoscere alcune funzioni di SQL che ci torneranno estremamente utili. A conclusione del capitolo produrremo le nostre prime interrogazioni su più tabelle.

3.1 Combinare condizioni nella clausola WHERE

Finora abbiamo sempre interrogato la tabella `Hobbits` ponendo una sola condizione: trova i record in cui la colonna "Nome" è uguale a, trova i record in cui la colonna "Cognome" è diversa da etc.

Sicuramente, se ci fermassimo qui, il nostro raggio d'azione sarebbe molto limitato: fortunatamente in SQL, la clausola WHERE può essere estesa per includere più condizioni utilizzando operatori logici come AND e OR, che possono essere combinati tra di loro utilizzando opportunamente le parentesi.

Per esempio, se volessi ottenere la lista degli hobbit con nome "Merry" e cognome "Baggins", dovrei scrivere in questo modo:

```

SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE (Nome = 'Merry') AND (Cognome = 'Baggins');

```

Non stupitevi se la query così composta non tira fuori nessun risultato, dato che non esistono Hobbits (almeno nella nostra tabella), che si chiamino Merry Baggins.

Potrebbe anche venirci la curiosità di cercare tutti gli hobbits che si chiamano Sam *oppure* che hanno cognome Took: è facile dedurre che la query sarà scritta in questo modo:

```

SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE (Nome = 'Sam') OR (Cognome = 'Took');

```

Nome	Cognome
Sam	Gamgee
Pippin	Took

Per vedere infine un esempio con le parentesi, possiamo richiedere tutti i record che abbiano cognome: "Baggins", oppure che abbiano nome "Pippin" e cognome "Brandybuck"

```

SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE (Cognome = 'Baggins') OR
      ((Nome = 'Pippin') AND (Cognome = 'Brandybuck'));

```

Il risultato è

Nome	Cognome
Frodo	Baggins
Bilbo	Baggins

dato che la prima parte della condizione restituisce due valori, la seconda nessuno.

3.2 IN e NOT IN

Se volessimo estrarre un lungo elenco di nomi dalla nostra tabella, ci viene in aiuto l'operatore `IN` il quale è seguito da una parentesi che racchiude l'elenco dei valori che desideriamo estrarre.

```
SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE Nome IN ('Frodo', 'Bilbo', 'Took');
```

che è equivalente a scrivere in questo modo:

```
WHERE ((Nome = 'Frodo') OR (Nome = 'Bilbo') OR (Nome = 'Took'))
```

, decisamente più pesante.

Si può facilmente notare che inserire nell'elenco un nome che non esiste in tabella (nel nostro caso, "Took") non genera nessun errore: semplicemente quel particolare valore non potrà essere estratto.

Non occorre avere una fantasia scatenata per immaginare quale funzione possa avere l'operatore `NOT IN`, la cui sintassi è del tutto analoga al suo corrispettivo, e la cui funzione equivale a scrivere in questo modo:

```
WHERE ((Nome <> 'Frodo') AND (Nome <> 'Bilbo') AND (Nome <> 'Took'))
```

Quick Tip: Come vengono gestiti i `NULL` dall'operatore `NOT IN`? Se ricordate, nella nostra tabella c'è un record con cognome `NULL`, ovvero Gollum: riuscite a immaginare quale risultato dà la query seguente?

```
SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE Cognome NOT IN ( 'Baggins', 'Gamgee', 'Brandybuck' );
```

Il risultato è

Nome	Cognome
Pippin	Took
Daisy	Boffin

Il record con Cognome NULL *non viene estratto*. Il motivo di questo comportamento si spiega facilmente, se ci si ricorda come viene gestito il NULL, che dà sempre risultato negativo quando comparato a qualsiasi altro valore con i normali operatori di comparazione.

3.3 <, >, loro varianti e BETWEEN

Dopo aver visto gli operatori <> e =, in T-SQL possiamo utilizzare <, >, <=, >=, che possono essere utilizzati indistintamente sia con valori numerici, sia con valori testuali o date. In questo modo potremo scrivere senza nessun problema

```
SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE Nome > 'Merry';
```

con cui otteniamo

Nome	Cognome
Sam	Gamgee
Pippin	Took

In T-SQL esiste un modo alternativo per trovare i record compresi tra due valori (estremi compresi); anziché scrivere in questo modo, infatti

```
SELECT Nome, Cognome
FROM dbo.Hobbits
```

```
WHERE Cognome >= 'Brandybuck' AND Cognome <= 'Took';
```

possiamo utilizzare il `BETWEEN`, con cui si ottiene lo stesso identico risultato

```
SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE Cognome BETWEEN 'Brandybuck' AND 'Took';
```

più snella e leggibile.

3.4 La funzione ISNULL

Anche se abbiamo iniziato da poco a lavorare con le query, abbiamo già notato come la gestione dei `NULL` possa essere molto insidiosa. In T-SQL abbiamo però alcuni modi per affrontare questo problema, il più immediato è la funzione `ISNULL`. Questa funzione, quando viene applicata a un valore, se questo è `NOT NULL`, restituisce il valore stesso, al contrario, restituisce un valore a nostra scelta in sostituzione di `NULL`. Questo ci garantisce di avere sempre a che fare con valori non nulli.

```
SELECT Nome, ISNULL(Cognome, 'Unknown')
FROM dbo.Hobbits
WHERE Nome = 'Gollum';
```

Nome	(No column name)
Gollum	Unknown

Salta subito all'occhio un particolare interessante, ovvero la colonna a cui abbiamo applicato la funzione `ISNULL` non riporta nessun nome, questo perché ogni volta che operiamo una qualsiasi modifica su di una colonna in fase di `SELECT`, il database sa che non potrà dare a quella colonna lo stesso nome della colonna originale.

L'unico modo per uscire da questa situazione è, ovviamente, utilizzare l'alias opportuno (che potrebbe anche essere il nome della colonna originale).

3.5 La funzione COALESCE

Anche questa funzione rientra nella categoria delle funzioni che ci aiutano a trattare quella bestia strana che è NULL. Il suo scopo è quello di valutare uno per uno gli argomenti che le vengono passati, dal primo all'ultimo e restituisce il valore corrente della prima espressione che inizialmente non restituisce NULL. Ad esempio,

```
SELECT COALESCE(NULL, NULL, 'terzo_valore', 'quarto_valore')
```

restituisce il terzo valore perché il terzo valore è il primo NOT NULL.

È importante sottolineare che la funzione COALESCE deve avere al suo interno almeno un valore non nullo, tant'è vero che eseguendo questa query

```
SELECT COALESCE (NULL, NULL, NULL)
```

otteniamo il seguente errore: **Msg 4127, Level 16, State 1, Line 1**

At least one of the arguments to COALESCE must be an expression that is not the NULL constant.

Nella sua forma più semplice, ovvero

```
SELECT COALESCE(expr1, 'Placeholder')
```

, la funzione COALESCE è del tutto equivalente a questa:

```
SELECT ISNULL(expr1, 'Placeholder')
```

Quick Tip: Questa è anche la prima volta che vediamo il comando **SELECT** utilizzato senza la clausola **FROM**: è una sintassi del tutto lecita in T-SQL, anche se non è uno standard SQL. Dato che non stiamo estraendo dati da una tabella, il risultato sarà sempre una sola riga, e altrettanto ovviamente, se non viene specificato nessun alias, il nome della colonna sarà (**No column name**).

3.6 Il comando LIKE

Il comando `LIKE` è utile per selezionare righe che corrispondono a pattern di stringhe specifici e offre una grande flessibilità nella ricerca dei dati basata su modelli. Se infatti volessimo estrarre tutti i record dalla tabella `Hobbits` col cognome che comincia per "Ba", potremmo scrivere una query di questo tipo:

```
SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE Cognome LIKE 'Ba%'
```

Potrei allo stesso modo volere i cognomi che finiscono per "E": in questo caso il simbolo `'%'` va spostato all'inizio della stringa. In modo del tutto analogo, se volessi cercare i caratteri `'amg'` in qualsiasi punto di una stringa, basterebbe scrivere in questo modo `'%amg%'`, dopo il `LIKE`.

Il comando `LIKE` è molto flessibile, e può essere usato in parecchi altri modi, eccoli elencati qui di seguito:

Wildcard	Significato	Esempio
<code>%</code> percento	Qualunque stringa, compresa stringa vuota	<code>'R%'</code> : una stringa che comincia per R
<code>_</code> (underscore)	Singolo carattere	<code>'_R%'</code> : una stringa in cui il secondo carattere è R
<code>[<lista di caratteri>]</code>	Il singolo carattere da una lista	<code>'[AC]%'</code> : una stringa il cui primo carattere è A o C
<code>[<range di caratteri>]</code>	Il singolo carattere da un range	<code>'[0-9]%'</code> : una stringa che abbia come primo carattere un numero
<code>[^<range o lista di caratteri>]</code>	Un singolo carattere che non è nella lista o range	<code>'[^0-9]%'</code> : una stringa che non abbia come primo carattere un numero

3.7 Il costrutto CASE... WHEN

Il costrutto CASE WHEN in SQL è utilizzato per eseguire valutazioni condizionali all'interno di una query e restituire valori diversi in base alle condizioni specificate. È simile alla struttura di controllo switch in altri linguaggi di programmazione.

La sintassi generale del costrutto CASE WHEN è la seguente:

```
CASE
    WHEN condizione1 THEN risultato1
    WHEN condizione2 THEN risultato2
    ...
    ELSE default_result
END
```

Nota bene: è buona pratica (anche se non obbligatorio), inserire la condizione di ELSE alla fine; se non lo facciamo e nessuna delle condizioni che abbiamo imposto si verifica, otterremo il famigerato NULL. Quindi, o siamo sicuri che le nostre condizioni coprano tutti i casi possibili, altrimenti scriviamo l'ELSE: potrebbe salvarci la vita.

Applicando il costrutto al nostro mini database da una sola tabella, potremmo voler scrivere il periodo in cui gli hobbit hanno portato l'anello, troveremo perciò

```
SELECT Nome,
        Cognome,
        CASE
            WHEN Nome = 'Bilbo' THEN
                '60 anni'
            WHEN Nome = 'Frodo' THEN
                '17 anni'
            WHEN Nome = 'Gollum' THEN
                '478 anni'
            WHEN Nome = 'Sam' THEN
```



```

        'due giorni'
    ELSE
        '-'
    END AS [Portatore per]
FROM dbo.Hobbits;

```

il risultato ottenuto sarà

Nome	Cognome	Portatore per
Bilbo	Baggins	60 anni
Frodo	Baggins	17 anni
Sam	Gamgee	due giorni
Merry	Brandybuck	-
Pippin	Took	-
Daisy	Boffin	-
Gollum	NULL	478 anni

Quick Tip: Se consideriamo la funzione **COALESCE** incontrata poco fa, possiamo provare ad esprimerla proprio attraverso il costrutto **CASE... WHEN** in questo modo. La sintassi:

```
SELECT COALESCE(val1, val2, val3, val4, ... valN)
```

equivale a questa:

```

CASE
    WHEN val1 IS NOT NULL THEN val1
    WHEN val2 IS NOT NULL THEN val2
    ...
    WHEN valN IS NOT NULL THEN valN
    ELSE NULL
END

```

Il costrutto `CASE... WHEN` è molto versatile, tanto che, volendo, potremmo specificare nelle `WHEN` anche condizioni su colonne diverse, come ad esempio

```
SELECT Nome,  
       Cognome,  
       CASE  
         WHEN Cognome = 'Baggins' THEN  
           'Buono'  
         WHEN Nome = 'Sam' THEN  
           'Buonissimo'  
         WHEN Nome = 'Gollum' THEN  
           'Cattivo'  
         ELSE  
           'Buono'  
       END AS Classificazione  
FROM dbo.Hobbits;
```

3.8 La funzione IIF

T-SQL ci fornisce un'altra funzione che possiamo utilizzare al posto del costrutto `CASE... WHEN`, ovvero la funzione `IIF`, che è un po' più semplice e meno versatile.

Anche in questo caso si tratta di valutare una condizione: se questa è verificata, allora viene restituito un valore, se non lo è, ne viene restituito un altro. La differenza fondamentale rispetto al costrutto `CASE... WHEN`, è che la condizione da valutare sarà una e una soltanto.

La sua sintassi è la seguente:

```
IIF(Condizione, ValoreQuandoVero, ValoreQuandoFalso)
```

che equivale a:

CASE

WHEN Condizione **THEN** ValoreQuandoVero

ELSE ValoreQuandoFalso

END

Potremmo affermare che ISNULL sta a COALESCE come IIF sta a CASE... WHEN.

Vediamo ora un esempio dell'uso che se ne può fare

SELECT TOP (4)

Nome,

Cognome,

IIF(Cognome = 'Baggins', 1, 0) **AS** IsBaggins

FROM dbo.Hobbits

ORDER BY Cognome **ASC**;

Che ci dà in output

Nome	Cognome	IsBaggins
Gollum	NULL	0
Frodo	Baggins	1
Bilbo	Baggins	1
Daisy	Boffin	0

3.9 Le query raggruppate - GROUP BY

Può essere molto utile, a volte, utilizzare qualche tipo di raggruppamento sui risultati di una query. Una query è raggruppata quando si usa una funzione di raggruppamento, una GROUP BY, o entrambe.

Guardiamo un esempio molto semplice, ovvero

SELECT COUNT(*) AS NumHobbits

FROM dbo.Hobbits;

Il risultato è

NumHobbits

7

Lo scopo di questa query è semplicemente contare quanti sono i record totali nella tabella. Aggiungendo alla query una clausola **WHERE**, troveremmo il numero dei risultati a seguito dell'applicazione del filtro.

In assenza di una **GROUP BY**, che differenzia i risultati per categorie all'interno di una tabella (lo vedremo fra un attimo), il risultato di una funzione di raggruppamento sarà sempre e solo una riga.

Se proviamo infatti ad eseguire questa query

```
SELECT Cognome, COUNT(*) AS NumCognome  
FROM dbo.Hobbits  
GROUP BY Cognome;
```

otterremo questo output

Cognome	IsBaggins
NULL	1
Baggins	2
Brandybuck	1
Gamgee	1
Took	1

, ovvero il numero totale di occorrenze per ciascun cognome presente.

Possiamo provare a complicare un po' le cose, scrivendo una query che restituisca il primo nome in ordine alfabetico a parità di cognome

```
SELECT MIN(Nome) AS MinNome, Cognome
FROM dbo.Hobbits
GROUP BY Cognome;
```

Troveremo infine questi risultati:

MinNome	Cognome
Gollum	NULL
Bilbo	Baggins
Daisy	Boffin
Merry	Brandybuck
Sam	Gamgee
Pippin	Took

Notiamo come i due risultati corrispondenti al cognome Baggins, ovvero Frodo e Bilbo, siano stati ridotti a uno, ovvero Bilbo, che precede Frodo in ordine alfabetico.

Passiamo ora in rassegna con una carrellata veloce le principali funzioni di raggruppamento che troviamo in T-SQL.

- **MIN:** Trova il valore minimo in una lista di valori
- **MAX:** Trova il valore massimo in una lista di valori
- **AVG:** Trova la media in una lista di valori
- **SUM:** Trova la somma di una lista di valori
- **COUNT:** Conteggia il numero di occorrenze

3.10 Le query raggruppate - HAVING

Se ve lo siete chiesti, sì, è possibile filtrare i risultati a seguito di una **GROUP BY**; se non ve lo siete chiesti non importa, la spiegazione arriverà lo stesso.

Non è difficile immaginare che non si possa utilizzare la classica clausola **WHERE**, dato che questa agisce sulle righe, mentre noi abbiamo bisogno di qualcosa che agisca sulle righe raggruppate. A questo proposito possiamo utilizzare la clausola **HAVING**.

Come dovrei fare, ad esempio, per trovare tutti i cognomi della nostra tabella che hanno più di una occorrenza? Potrei scrivere una query così:

```
SELECT Cognome, COUNT(*) AS NumCognome
FROM dbo.Hobbits
GROUP BY Cognome;
HAVING COUNT(*) > 1;
```

Il risultato già lo sappiamo, perché solo il cognome "Baggins" ha più di un'occorrenza.

Cognome	IsBaggins
Baggins	2

Cosa succederebbe se provassimo ad eseguire questa query?

```
SELECT Nome, Cognome, COUNT(*) AS Conteggio
FROM dbo.Hobbits
GROUP BY Cognome;
```

Riceveremmo un messaggio di errore

Msg 8120, Level 16, State 1, Line 1

Column 'dbo.Hobbits.Nome' is invalid in the select list because it is not contained in either an aggregate function or the GROUP BY clause.

Questo ci dice che ogni colonna che non subisce l'azione di una funzione di raggruppamento, e che non sia una costante, deve essere indicata esplicitamente nella **GROUP BY**.

Ci sono diversi modi per evitare l'errore di prima, il più immediato è aggiungere anche il cognome nella **GROUP BY**, ottenendo:

```
GROUP BY Nome, Cognome;
```

In questo caso la soluzione è un po' banale, dato che abbiamo solo valori distinti della coppia Nome / Cognome, e quindi troveremo sempre un conteggio uguale a 1.

L'altra soluzione è applicare una funzione di raggruppamento sul campo non presente nella GROUP BY, in modo simile a quanto avevamo visto all'inizio di questo stesso paragrafo:

```
SELECT MIN(Nome) AS Nome, Cognome, COUNT(*) AS Conteggio
FROM dbo.Hobbits
GROUP BY Cognome;
```

3.11 La COUNT, qualche dettaglio in più

```
SELECT COUNT(Cognome)
FROM dbo.Hobbits;
```

Possiamo usare la DISTINCT anche in combinazione con COUNT; se volessimo contare quanti cognomi distinti esistono nella tabella `Hobbits`, ci basterà scrivere in questo modo:

```
SELECT COUNT(DISTINCT Cognome) AS NumCognomi
FROM dbo.Hobbits;
```

Il risultato è 5 perché, anche in questo caso, il NULL non viene conteggiato, in quanto non viene nemmeno considerato come cognome.

3.12 Siamo uomini o macchine?

Se vi chiedessi di esprimere nel modo più semplice possibile un comando in cui spiegate a una persona priva di immaginazione di comprare qualcosa, quale opzione scegliereste?

1. Vai a prendere un litro di latte al supermercato

2. Vai al supermercato a prendere un litro di latte

Vi do una notizia: se avete scelto la prima opzione probabilmente siete umani, se avete scelto la seconda, invece, c'è il sospetto che siate dei computer mascherati.

Spiego meglio questa affermazione piuttosto forte: è tipicamente umano ragionare dal piccolo al grande (difatti diciamo prima il giorno, poi il mese e l'anno), mentre è molto più semplice per un computer fare l'inverso (non a caso il modo migliore per scrivere una data in un codice qualunque, generalmente è YYYYMMDD). Al netto di queste considerazioni spannometriche, vorrei che passasse un concetto importante: la sintassi dei comandi di una query segue un ordine che non è lo stesso con cui questa query viene eseguita.

Tutte le query che abbiamo visto finora avevano i comandi espressi in quest'ordine:

1. **SELECT**: Indica quali colonne vogliamo visualizzare
2. **FROM**: Indica da quale tabella vogliamo prenderle
3. **WHERE**: Impone un filtro a livello di riga
4. **GROUP BY**: Raggruppa le righe secondo un determinato criterio
5. **HAVING**: Impone un filtro a livello di righe raggruppate
6. **ORDER BY**: Ordina i risultati

e sarà sempre così. Abbiamo visto come alcuni di questi comandi (tutti quanti tranne la **SELECT**, in realtà) si possano omettere, ma ogni volta che scriveremo una query dovremo rispettare quest'ordine. D'altronde è abbastanza intuitivo: prima dico cosa voglio ottenere, poi con quali criteri di filtraggio, eventualmente eseguo un raggruppamento, poi con quali criteri di filtraggio sulle righe raggruppate, e, come ultima azione, ho la possibilità di ordinare i risultati. Ma il database ragiona davvero così? La risposta è no.

Il modo con cui la query viene processata effettivamente è il seguente

1. **FROM**: Indica da quale tabella vogliamo prenderle

2. **WHERE**: Impone un filtro a livello di riga
3. **GROUP BY**: Raggruppa le righe secondo un determinato criterio
4. **HAVING**: Impone un filtro a livello di righe raggruppate
5. **SELECT**: Indica quali colonne vogliamo visualizzare
6. **ORDER BY**: Ordina i risultati

Ogni fase opera su una tabella (o più tabelle) e restituisce una tabella virtuale come output, che diventa l'input per il comando immediatamente successivo.

Consideriamo la seguente query:

```
SELECT Cognome
FROM dbo.Hobbits
WHERE Cognome <> 'Took'
GROUP BY Cognome
HAVING COUNT(*) > 1
ORDER BY Cognome ASC;
```

Chiediamoci qual è l'insieme più grande che viene menzionato nella query: sicuramente la tabella non filtrata. Data la nostra query, tutti e soli i valori che vedremo, apparterranno a questa tabella. Solitamente una tabella ha più righe o più colonne? Nella stragrande maggioranza dei casi ha più righe: ho visto più di una volta tabelle con miliardi di record, ma per fortuna ancora nessuna tabella con un miliardo di colonne, quindi dal punto di vista dell'efficienza è più conveniente liberarsi il prima possibile delle righe in eccesso, ed ecco spiegato il motivo per cui il secondo step è la **WHERE**.

Nella figura seguente possiamo vedere, incorniciata in verde, la tabella virtuale di output come esce dopo la **FROM**; incorniciata in blu, invece, abbiamo la tabella come compare dopo essere passata attraverso il filtro della **WHERE**. (Non mi stancherò mai di far notare che l'operatore <> esclude anche il record che ha cognome **ISNULL**).

Nome	Cognome
Gollum	NULL
Bilbo	Baggins
Frodo	Baggins
Daisy	Boffin
Merry	Brandybuck
Sam	Gamgee
Pippin	Took

Utilizzare il **GROUP BY** accorpa tutti i record con cognome uguale, in questo modo

Nome	Cognome
Bilbo	Baggins
Frodo	Baggins
Daisy	Boffin
Merry	Brandybuck
Sam	Gamgee

e **HAVING**, esattamente come era accaduto prima per la **WHERE**, filtra le righe accorpate.

Cognome	Conteggio
Baggins	2
Boffin	1
Brandybuck	1
Gamgee	1

La penultima azione che viene compiuta, infine, è selezionare le colonne che vogliamo:

Cognome	Conteggio
Baggins	2

Una volta che abbiamo l'elenco di tutti i record che ci interessano, possiamo eseguire l'ordinamento che, essendo un'operazione piuttosto costosa in termini di consumo

delle risorse, va fatta sul minor numero di dati possibili, ovvero solo dopo che tutti i filtri e la select sono state effettuate.

In questo caso, avendo un solo record che soddisfa tutte le caratteristiche della query, l'ordinamento non è particolarmente significativo, ma prima dell'esecuzione, il database non poteva di certo saperlo.

Possiamo quindi affermare che, eccezion fatta per i cognomi sconosciuti e per i membri della famiglia Took, solo "Baggins" è il cognome che appare nella nostra tabella per più di una volta. Un risultato eccezionale.

Soffermiamoci un attimo a ragionare su quanto abbiamo appena affermato, e capiremo alcuni comportamenti già incontrati in precedenza, ma che allora ci sembravano inspiegabili.

Il fatto che la **WHERE** venga eseguita logicamente prima della **SELECT**, giustifica il fatto che, se nel filtrare dei risultati proviamo a riferirci ad una colonna con il suo alias, riceviamo un errore. Nel momento in cui viene fatta la **WHERE**, il database non sa ancora come abbiamo riominato la colonna, perché non è ancora arrivato a quel punto.

```
SELECT Nome AS NomeHobbit
FROM dbo.Hobbits
WHERE NomeHobbit = 'Frodo';
```

Solleva questa eccezione: **Invalid column name 'NomeHobbit'**.

L'alias che invece è perfettamente conosciuto già al momento di processare la **WHERE** è quello della tabella, dato che siamo già passati dalla **FROM**. Una query scritta in questo modo, infatti

```
SELECT hob.Nome AS NomeHobbit
FROM dbo.Hobbits hob;
```

Non crea nessun problema.

Possiamo a questo punto giustificare un altro fatto, ovvero come mai l'operatore di `ORDER BY` si comporti diversamente, e possa riferirsi ad una colonna direttamente con il suo alias: essendo eseguito per ultimo, è già passato dalla `ORDER BY` e quindi sa già che nomi abbiamo dato alle colonne. Una query del genere, quindi

```
SELECT Nome AS NomeHobbit
FROM dbo.Hobbits
ORDER BY NomeHobbit ASC;
```

sarà eseguita senza alcun errore.

Quick tip: per lo stesso identico motivo, nella `ORDER BY`, possiamo riferirci alle colonne che vogliamo estrarre utilizzando il numero corrispondente al loro ordine nella `SELECT`.

```
SELECT Nome, Cognome
FROM dbo.Hobbits
ORDER BY 2 ASC;
```

Capitolo 4

Tipi di dati e loro manipolazione in SQL Server

Nella sezione che ci attende prenderemo confidenza con i tipi di dati che possiamo incontrare lavorando con SQL Server, come trasformarli da un tipo ad un altro, e impareremo ad usare alcune funzioni utili per la manipolazione delle stringhe e delle date.

4.1 Tipi di dati in SQL Server

Ogni volta che creiamo una tabella, una variabile, un parametro, veniamo messi di fronte alla scelta di quale tipo di variabile utilizzare. SQL Server fornisce un ventaglio di possibilità molto ampio: prendere la decisione giusta in questo senso può rivelarsi cruciale.

SQL Server supporta molte categorie diverse di dati, numerico esatto (`INT`, `NUMERIC`), stringhe di caratteri (`CHAR`, `VARCHAR`), Stringhe di caratteri Unicode (`NCHAR`, `NVARCHAR`), numerici approssimati (`FLOAT`, `REAL`), stringhe binarie (`BINARY`, `VARBINARY`), data e ora (`DATE`, `TIME`, `DATETIME2`, `SMALLDATETIME`, `DATETIME`, `DATETIMEOFFSET`), e altri.

L'aspetto più importante da tenere in considerazione è che scegliere un tipo di variabile impone la scelta di un dominio di valori che questa variabile potrà assumere. Se, ad esempio, scegliamo il tipo `DATE`, non potremo assegnarlo a valori come `'2024-99-99'`: se cerchiamo di farlo riceveremo un errore. Se però decidiamo di

allargare le maglie, per esempio decidendo di rappresentare una data con un tipo stringa, magari non otterremo errori di validazione di questo genere, ma ovviamente perderemo la garanzia che la data rappresentata sia una data valida. Il tipo di dato che scegliamo può avere conseguenze molto importanti anche su come questi dati si comportano: l'operatore `+`, ad esempio, se applicato a due stringhe le concatena, se applicato a due interi li somma. E se venisse applicato ad una stringa e un intero? In questo caso avremmo un errore di conversione, perché SQL Server non è in grado di scegliere a quale dei due tipi dare precedenza; questo non avviene tutte le volte che mettiamo a confronto due dati di tipo diverso: se questi sono in qualche modo compatibili, SQL Server tenta di compiere una conversione implicita. Mantenendo il caso dell'operatore `+`, non ci sono problemi a concatenare un tipo `CHAR` e un tipo `NVARCHAR`, ad esempio, come non ce ne sono a sommare un numero dichiarato `INT` e un `NUMERIC`. Sicuramente però è necessario prestare più attenzione per operazioni più complicate, come per esempio la divisione, dove una gestione disattenta dei decimali potrebbe generare risultati inattesi.

Quick tip: specialmente quando, più avanti, metteremo in rapporto tra loro colonne di tabelle diverse, utilizzando la `JOIN`, sarà molto importante che i tipi di dati delle due colonne confrontate siano consistenti tra di loro perché, sebbene permessa, la conversione implicita potrebbe avere impatti negativi sulle performance, ad esempio impedendo un uso ottimale degli indici.

Un'altra scelta determinante, che esula dal tipo di dato, ma che lo riguarda molto da vicino, è se questo possa accettare o meno il famigerato `NULL`.

Sicuramente nel decidere tra un tipo di dato "grande" (es. `NVARCHAR(MAX)`) e uno più piccolo `CHAR(5)`, la scelta più sicura è quella di tenersi larghi, ma anche tenersi *troppo* larghi potrebbe avere delle conseguenze nefaste, ad esempio sul consumo delle risorse: più grande è il tipo di dato, più storage viene utilizzato. Poniamo il caso di avere una colonna in cui viene inserita la data di nascita di una persona: il tipo `DATETIME`, che arriva alla precisione del millisecondo (8 byte di storage), è decisamente eccessivo:

in questo caso basterà utilizzare il più snello `DATE` (3 byte di storage), che contiene solo giorno, mese e anno. Il gioco consiste nello scegliere un tipo di dato abbastanza grande da contenere ragionevolmente tutti i valori possibili per quel dato, ma al tempo stesso senza appesantire troppo.

I tipi di dati che riguardano le stringhe si distinguono tra tipi fissi (`CHAR`, `NCHAR`, `BINARY`) e dinamici (`VARCHAR`, `NVARCHAR`, `VARBINARY`). Se per esempio creiamo una colonna di tipo `CHAR(10)`, i dati ivi contenuti, indipendentemente dalla loro lunghezza (che dovrà essere sempre minore o uguale a dieci, naturalmente), occuperanno lo spazio di archiviazione per dieci caratteri.

Con le stringhe di caratteri, c'è anche la questione dell'uso dei tipi di carattere regolari (`CHAR`, `VARCHAR`) rispetto ai tipi Unicode (`NCHAR`, `NVARCHAR`).

I tipi di caratteri regolari utilizzano 1 byte di archiviazione per carattere e supportano principalmente il set di caratteri ASCII, che include solo l'alfabeto inglese, numeri e caratteri speciali, sono perciò adatti per contenere dati in lingua inglese o altre lingue che utilizzano il set di caratteri ASCII.

I tipi Unicode, invece utilizzano 2 byte di archiviazione per carattere; supportano un'ampia gamma di lingue e caratteri, inclusi quelli non presenti nel set di caratteri ASCII, come caratteri accenti, simboli speciali, ideogrammi, ecc. Sono adatti per dati internazionali o applicazioni che devono supportare lingue diverse. Possono occupare più spazio di archiviazione rispetto ai tipi di caratteri regolari, ma offrono maggiore flessibilità per gestire una vasta gamma di lingue e caratteri.

Quick tip: Quando si utilizzano tipi che possono avere una lunghezza associata, come `CHAR` o `NVARCHAR`, e non si specifica tale lunghezza, T-SQL utilizza quella di default. È comunque considerata una pratica migliore indicarla esplicitamente, specificando sempre, ad es. `VARCHAR(255)` o `CHAR(8)`.

In T-SQL, il prefisso N davanti alle stringhe è utilizzato per indicare una stringa Unicode. Quando si utilizza N prima di una stringa letterale, si sta indicando al

sistema che la stringa dovrebbe essere trattata come una stringa Unicode anziché come una stringa non Unicode (o "ASCII").

Perciò, sempre per evitare conversioni implicite, dato che il default nella dichiarazione di una stringa è il `VARCHAR`, sarebbe sempre opportuno utilizzare questa sintassi, quando il confronto è con `NVARCHAR`, ad es.

```
SELECT Nome
FROM dbo.Hobbits
WHERE Cognome = N'Baggins';
```

4.2 Principali tipi di dati in SQL Server

Tipi di dati numerici:

1. `TINYINT`: Interi a 8 bit (da 0 a 255).
2. `SMALLINT`: Interi a 16 bit (da -32.768 a 32.767).
3. `INT`: Interi a 32 bit (da -2.147.483.648 a 2.147.483.647).
4. `BIGINT`: Interi a 64 bit (da -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807).
5. `DECIMAL(p,s)/NUMERIC(p,s)`: Numeri decimali con precisione fissa: il primo numero tra parentesi indica il numero di cifre totali (prima e dopo la virgola), il secondo indica le sole cifre dopo la virgola - `NUMERIC(10,5)` avrà quindi cinque cifre a destra e cinque a sinistra della virgola.
6. `FLOAT(n)/REAL(n)`: Numeri a virgola mobile a precisione singola o doppia.

Tipi di dati carattere:

1. `CHAR(n)`: Stringhe di lunghezza fissa fino a 8000 caratteri.
2. `VARCHAR(n)`: Stringhe di lunghezza variabile fino a 8000 caratteri.
3. `VARCHAR(MAX)`: Stringhe di lunghezza variabile fino a 1.073.741.824 caratteri.

4. **NCHAR(n)**: Stringhe Unicode di lunghezza fissa fino a 4000 caratteri.
5. **NVARCHAR(n)**: Stringhe Unicode di lunghezza variabile fino a 4000 caratteri.
6. **NVARCHAR(MAX)**: Stringhe Unicode di lunghezza variabile fino a 536.870.912 caratteri.
7. **TEXT**: 2GB di dati testuali.

Tipi di dati data e ora:

1. **DATE**: Archivia solo la data. Dal 1° gennaio dell'anno 1 al 31 dicembre 9999.
2. **TIME**: Archivia solo l'ora con un'accuratezza di 100 nanosecondi.
3. **DATETIME**: Dal 1° gennaio 1753 al 31 dicembre 9999 con un'accuratezza di 3,33 millisecondi.
4. **DATETIME2**: Dal 1° gennaio dell'anno 1 al 31 dicembre 9999 con un'accuratezza di 100 nanosecondi.
5. **SMALLDATETIME**: Dal 1° gennaio dell'anno 1 al 31 dicembre 1900 al 6 giugno 2079 con un'accuratezza di 1 minuto.
6. **DATETIMEOFFSET**: Lo stesso di datetime2 con l'aggiunta di un offset del fuso orario.

Tipi di dati booleani:

1. **BIT**: 0, 1 o NULL.

Altri tipi di dati:

1. **BINARY(n)**: Stringa binaria a lunghezza fissa.
2. **VARBINARY(n)**: Stringa binaria a lunghezza variabile.
3. **IMAGE**: Stringa binaria a lunghezza variabile.
4. **UNIQUEIDENTIFIER**: identificatore univoco globale (GUID).
5. **XML**: dati XML fino a un massimo di 2 GB.

In dettaglio: Proviamo a vedere cosa succede eseguendo la query seguente, in cui aggiungiamo due millisecondi ad una certa data:

```
SELECT DATEADD(ms,2,'2024-05-18 23:59:59.997');
```

Ci aspetteremmo di trovare 2024-05-18 23:59:59.999, invece il risultato, sorprendentemente, è 2024-05-19 00:00:00.000. Il motivo risiede nella precisione dei tipi di dati datetime in SQL Server.

Il tipo di dato DATETIME in SQL Server ha una precisione di 3.33 millisecondi, il che significa che i millisecondi vengono arrotondati al terzo decimale più vicino. Ne consegue che, nel nostro caso, aggiungendo 2 millisecondi al tempo 23:59:59.997, il risultato viene arrotondato a 00:00:00.000.

Se usiamo un tipo DATETIME non avremo quindi mai modo di vedere un orario del genere: 12:32:45.238 o 17:27:54.822. Se necessitiamo di una maggiore precisione, possiamo considerare l'utilizzo del tipo di dato DATETIME2, che offre una precisione fino a 100 nanosecondi.

4.3 Le funzioni di conversione

Ora che abbiamo visto per bene la zoologia dei tipi di variabile, possiamo esplorare come passare da un tipo di variabile ad un altro, tenendo ben presente che non tutti i passaggi sono consentiti.

La prima che affrontiamo, di queste, è la **CAST**. La sua sintassi è la seguente

```
SELECT CAST('123' AS INT);
```

, quindi prima di **AS** va inserito il valore da convertire, e dopo il tipo di dato in cui convertirlo.

Se provassimo ad applicare la funzione ad un valore non convertibile in intero, riceveremmo un errore:

```
SELECT CAST( 'a' AS INT) AS Conversione;
```

Msg 245, Level 16, State 1, Line 48

Conversion failed when converting the varchar value 'a'
to data type int.

Quick tip: bisogna sempre prestare molta attenzione ai tipi di dato approssimati, come il `FLOAT`: il vantaggio che ha è di occupare poco spazio, ma il fatto di essere approssimati, a volte porta a risultati bizzarri.

```
SELECT  
    CAST(  
        CAST('42128.022495' AS FLOAT)  
        AS NUMERIC(28, 14)) AS [value];
```

In questo caso abbiamo semplicemente convertito un numero da `FLOAT` a `NUMERIC(28, 14)`, ma il risultato di questa conversione dà un numero diverso dall'originale, ovvero 42128.02249499999743.

Molto simile è il comportamento della funzione `CONVERT`, che ha due principali differenze: è più versatile e non è uno standard SQL.

Nella sua forma base fa esattamente la stessa cosa della `CAST`, ovvero

```
SELECT CONVERT(INT, '123') AS Conversione;
```

ma consente di specificare uno stile di formattazione per la conversione di tipi di dati come date e orari. Ad esempio, avvalendoci della funzione `GETDATE()`, che restituisce la data corrente:

```
SELECT CONVERT(VARCHAR(20), GETDATE(), 1) AS DataTestuale;
```

porta a questo risultato: 05/28/24, mentre scrivendo in quest'altro modo

```
SELECT CONVERT(VARCHAR(20), GETDATE(), 25) AS DataTestuale;
```

otteniamo 2024-05-28 11:59:27.

Per avere l'elenco totale dei codici dei formati possibili (sono davvero tanti!) potete fare riferimento alla documentazione Microsoft.

Quick tip: Quando si tratta di manipolare dei numeri, è di fondamentale importanza utilizzare il tipo di dati corretto, come anche utilizzare l'eventuale funzione di conversione nel modo giusto. Se infatti scrivo

```
SELECT 1 / 2
```

otterrò come risultato: 0, che probabilmente non è quello che mi aspetto. Trattandosi di un'operazione tra due numeri che vengono interpretati come interi, SQL Server restituirà un numero intero, che per approssimazione, è 0.

Quello che serve fare a questo punto è applicare una funzione di conversione, ovvero

```
SELECT CAST(1 / 2 AS NUMERIC(10,5))
```

Giusto? No. Anche applicando l'operatore di conversione, ottengo sempre 0, ma con più cifre decimali, cioè 0.00000.

Questo perché la conversione viene eseguita solo dopo che l'operazione tra interi ha dato il suo risultato, cioè 0.

L'unico modo che abbiamo per ottenere un risultato decimale da questa operazione tra interi è farli trattare da SQL Server come decimali, ovvero

```
SELECT 1.0 / 2.0
```

che infine dà il risultato sperato, ovvero 0.500000.

Oppure, se vogliamo avere il pieno controllo sulle cifre decimali, in modo sicuramente più barocco,

```
SELECT CAST(1 AS numeric(7,2))/ CAST(2 AS numeric(7,2))
```

che dà 0.500000000000000.

Anche la funzione `PARSE` fa praticamente la stessa cosa, consentendo infatti di convertire in un formato basato su una specifica cultura. Eccone un esempio:

```
SELECT PARSE('12/16/2024' AS DATETIME USING 'en-US') AS Risultato;
```

da cui otteniamo: 2024-12-16 00:00:00.000.

Di grande utilità è anche la funzione `FORMAT`, che invece converte da data a stringa, esplicitando il formato;

```
SELECT FORMAT(GETDATE(), 'dd/MM/yyyy') AS Risultato;
```

da cui, come è facilmente intuibile, otteniamo: 28-05-2024.

Abbiamo visto prima che, se tentiamo di convertire 'a' in un intero, riceviamo un errore: per fortuna esistono tre funzioni, `TRY_CAST`, `TRY_CONVERT`, `TRY_PARSE`, dalla sintassi del tutto analoga alle loro corrispettive `CAST`, `CONVERT`, `PARSE`. Queste funzioni, anziché restituire errore in caso di conversione "impossibile", restituiscono `NULL`.

```
SELECT TRY_CAST('a' AS INT) AS Risultato;
```

Risultato
NULL

Non si tratta propriamente di una funzione di conversione, ma è utile citarla, la funzione `ISNUMERIC`, che restituisce 1 quando l'espressione di input è di tipo numerico; in caso contrario, restituisce 0.

Quick tip: Se eseguiamo questa query

```
SELECT ISNUMERIC('4E4');
```

potremmo aspettarci che restituisca 0, dato che 4E4 non sembra affatto un numero; invece SQL Server considera anche la notazione scientifica e pertanto valuta 4E4 come un numero valido.

Allo stesso identico modo lavora la funzione ISDATE

```
SELECT ISDATE('2024-05-31');
```

```
SELECT ISDATE(20241212);
```

```
SELECT ISDATE('2024/06/30');
```

```
SELECT ISDATE('01-15-2024');
```

queste select danno come risultato 1, quindi tutte le date vengono riconosciute come valide.

```
SELECT ISDATE('15-01-2024');
```

```
SELECT ISDATE('2024/30/06');
```

Per contro, queste due select danno come risultato 0, dato che SQL Server non riesce a riconoscere le stringhe in questione come date.

Quick tip: datetime vs date

Capitolo 5

Una carrellata sulle funzioni di SQL Server

Abbiamo già incontrato parecchie funzioni durante il nostro percorso, solo per citarne alcune, `MIN`, `MAX`, `CAST`, `TRY_CONVERT` etc.

In questo capitolo ne affronteremo delle altre, orientate soprattutto alla gestione delle date e delle stringhe.

5.1 Funzioni per la gestione delle date

5.1.1 Funzioni per il recupero della data corrente

Abbiamo visto nella sezione 4.3 l'utilizzo della funzione `GETDATE()`, più nello specifico, questa funzione restituisce l'istante temporale di sistema corrente del database come un valore datetime senza l'offset del fuso orario del database. Questo valore è derivato dal sistema operativo del computer su cui è in esecuzione l'istanza di SQL Server.

Esattamente analoga è la funzione `CURRENT_TIMESTAMP`, che è standard, e quindi preferibile rispetto a `GETDATE()`.

L'altra funzione più diffusa per ottenere la data corrente è la `GETUTCDATE()`, che restituisce un valore datetime contenente la data e l'ora del computer su cui è in

esecuzione l'istanza di SQL Server. La funzione restituisce i valori della data e dell'ora come tempo UTC (Tempo Universale Coordinato).

Qui di seguito tre esempi di esecuzione delle funzioni appena illustrate:

```
SELECT GETDATE();
```

```
SELECT CURRENT_TIMESTAMP;
```

```
SELECT GETUTCDATE();
```

```
2024-05-28 18:21:11.713
```

```
2024-05-28 18:21:11.713
```

```
2024-05-28 13:21:11.713
```

Quick tip: quando si tratta di esprimere date come testo, ad es per valorizzare il parametro di una funzione, in SQL Server, sono permesse tutte queste forme:

```
'20240528' , '2024-05-28' , '2024/05/28'
```

Lo stesso vale se vogliamo specificare anche l'ora: sono interpretate correttamente come date tutte le stringhe seguenti

```
'2024-05-28 18:21:11.713'
```

```
'20240528 18:21:11'
```

```
'2024/05/28 18:21'
```

5.1.2 Funzioni per estrarre elementi da una data o comporla

La funzione DATEPART estrae da una data le diverse parti da cui è composta, mentre la DATEFROMPARTS fa l'opposto, ovvero compone una data a partire dalle sue parti.

La sintassi è DATEPART(*partedelladata*, *data*), dove la parte da estrarre deve essere sostituita da uno di questi valori

Il che vuol dire, che se vogliamo estrarre l'anno dalla data corrente, potremo scrivere indifferentemente

Parte della data	abbreviazione
year	yy, yyyy
quarter	qq, q
month	mm, m
dayofyear	dy, y
day	dd, d
week	wk, ww
weekday	dw
hour	hh
minute	mi, n
second	ss, s
millisecond	ms
microsecond	mcs
nanosecond	ns

```
SELECT DATEPART(yy, GETDATE());
```

```
SELECT DATEPART(yyyy, GETDATE());
```

```
SELECT DATEPART(year, GETDATE());
```

Il risultato sarà, in tutti e tre i casi, 2024 (almeno finché saremo nel 2024).

Una funzione simile, che però ricava una stringa anziché un intero è la DATENAME, che, utilizzata in questo modo,

```
SELECT DATENAME(month, '20243110');
```

restituisce October

Ovviamente si tratta di una funzione che dipende dal linguaggio della nostra sessione, se infatti utilizziamo il comando

```
SET LANGUAGE italian;
```

otterremo: ottobre.

SQL Server ci fornisce anche un altro modo per ricavare anno, mese o giorno (come interi) per una determinata data, ovvero le funzioni `YEAR`, `MONTH`, e `DAY`, dalla sintassi immediata:

```
SELECT YEAR(GETDATE());
```

```
SELECT MONTH(GETDATE());
```

```
SELECT DAY(GETDATE());
```

Come anticipato prima, la `DATEFROMPARTS` fa esattamente l'opposto, ovvero compone una data a partire dai suoi elementi:

```
SELECT DATEFROMPARTS(2024,12,31)
```

Il risultato è espresso in formato `DATE`, ovvero senza l'orario. Ovviamente esiste una funzione per ciascun tipo di variabile data, avremo quindi `DATETIMEFROMPARTS` che dà in output un tipo `DATETIME`, `SMALLDATETIMEFROMPARTS` che dà in output un tipo `SMALLDATETIME`, e così via.

5.1.3 Funzioni per aggiungere periodi di tempo ad una data

T-SQL supporta le funzioni di aggiunta e differenza di data e ora chiamate `DATEADD` e `DATEDIFF`.

La `DATEADD` serve per aggiungere o togliere un determinato numero ad una determinata parte della data. Se volessi perciò aggiungere tre mesi al 28 maggio del 2024, dovrei scrivere in questo modo

```
SELECT DATEADD(month, 3, '20240528');
```

Per andare indietro nel tempo mi basterà mettere il segno meno al secondo parametro.

La funzione `DATEDIFF` è un'altra funzione comunemente utilizzata; restituisce la differenza in termini di una parte richiesta tra due valori di data e ora. Ad esempio, l'espressione

```
SELECT DATEDIFF(day, '20240512', '20240524');
```

calcola la differenza in giorni tra il 12 maggio 2024 e il 24 maggio 2024, restituendo il valore 10.

5.2 Funzioni per la gestione delle stringhe

5.2.1 Concatenazione di stringhe

Una delle funzioni più basilari che coinvolgono la manipolazione delle stringhe è la concatenazione. In SQL Server ci sono due modi leggermente differenti per farlo, il `+` e la funzione `CONCAT`.

Una query così composta,

```
SELECT TOP (3) Nome + ' ' + Cognome AS NomeCompleto
FROM dbo.Hobbits
ORDER BY Cognome ASC ;
```

NomeCompleto

NULL

Frodo Baggins

Bilbo Baggins

Tira fuori una sola colonna per nome e cognome, separati da uno spazio. Occorre notare che l'operatore `+`, quando si concatena una o più stringhe non nulle con un `NULL`, dà sempre origine a `NULL`; nel nostro caso, infatti, il primo risultato corrisponderebbe a "Gollum", che avendo però cognome `NULL`, dà come risultato finale proprio `NULL`.

La funzione `CONCAT`, al contrario, sostituisce gli eventuali valori nulli con una stringa vuota. La stessa query di prima, infatti, scritta in questo modo

```
SELECT TOP (3)
    CONCAT(Nome, ' ', Cognome) AS NomeCompleto
FROM dbo.Hobbits
ORDER BY Cognome ASC;
```

restituisce

NomeCompleto

Gollum

Frodo Baggins

Bilbo Baggins

Un'altra particolarità che rende la **CONCAT** più duttile, e preferibile rispetto all'operatore **+**, è che permette la concatenazione di tipi diversi (es. **NVARCHAR** e **INT**) senza dover esplicitare una funzione di conversione.

5.2.2 Estrazione e ricerca di una stringa

In questa sezione vedremo come estrarre una stringa più piccola da un'altra stringa, e come ricercare una stringa all'interno di un'altra stringa.

Per il primo scopo possiamo utilizzare la funzione **SUBSTRING**, che rispetta questa sintassi:

```
SUBSTRING(StringaDaTagliare,PosizioneIniziale,NumeroDiLettere)
```

Il primo argomento è la stringa che si vuole tagliare, mentre il secondo indica la posizione da cui iniziare a tagliare, e quante delle lettere successive considerare, tenendo presente che la prima lettera di una stringa è considerata essere in posizione 1. Da una query di questo genere, infatti,

```
SELECT SUBSTRING('Seggiovia', 1, 6);
```

otteniamo **Seggio**.

Inserendo come secondo argomento un numero maggiore rispetto alla lunghezza della stringa non otterremo nessun messaggio di errore, ma solamente una stringa vuota. Anche nel caso in cui la lunghezza della stringa che vogliamo estrarre eccedesse la lunghezza totale della stringa originale, non viene sollevata nessuna eccezione.

Se vogliamo tagliare una stringa a partire dall'inizio o a partire dal fondo, abbiamo la possibilità di utilizzare le due funzioni `LEFT` e `RIGHT`, che possiamo usare in questo modo:

```
SELECT LEFT('Seggiovvia', 6);
```

ottenendo lo stesso risultato di prima. In modo analogo, la `RIGHT`

```
SELECT RIGHT('Seggiovvia', 3);
```

restituisce via.

Scopo della funzione `CHARINDEX` è quello di individuare la prima occorrenza di un carattere (o di una stringa), all'interno di un'altra stringa, ad esempio

```
SELECT CHARINDEX('non', 'Chi non risica non rosica');
```

dà 5 come risultato, perché la parola "non" si trova al quinto posto; nel caso la parola non venga trovata, la funzione restituisce 0. La funzione `CHARINDEX` inoltre ammette anche un terzo parametro, che indica la posizione da cui cominciare la ricerca. Se scriviamo in questo modo, infatti,

```
SELECT CHARINDEX('non', 'Chi non risica non rosica', 6);
```

avremo 16 come risultato, ovvero la posizione del secondo "non" nella frase, dato che il terzo parametro ha imposto l'inizio della ricerca all'interno della frase oltre la posizione del primo "non".

Il T-SQL supporta anche l'utilizzo della funzione `PATINDEX`, che permette la ricerca di pattern all'interno di una stringa, in modo simile a come opera la funzione `LIKE`.

```
SELECT PATINDEX('%risi_a%', 'Chi non risica non rosica');
```

5.2.3 Lunghezza di una stringa

Per ottenere la lunghezza di una stringa possiamo far uso della funzione `LEN`, che conta il numero di caratteri presenti

```
SELECT LEN ('Frodo ')
```

Restituisce 4, nonostante la stringa sia lunga cinque caratteri, questo perché se la stringa in questione termina con uno spazio, questo non viene conteggiato.

DATALENGTH invece restituisce la lunghezza in termini di byte. Se la stringa in questione è Unicode, saranno contati 2 byte per carattere

```
SELECT DATALENGTH(N'Frodo ');
```

Dà come risultato 10, perché in questo caso lo spazio viene conteggiato.

5.2.4 Modifica di una stringa

In questa sezione faremo la conoscenza con quattro funzioni, REPLACE, SPACE, REPLICATE e STUFF.

La prima sostituisce una sequenza di caratteri con un'altra all'interno di una stringa, utilizzando questa sintassi

```
REPLACE ( StringaOriginale , StringaDaTrovare , StringaSostitutiva ).
```

Vediamone velocemente un esempio:

```
SELECT REPLACE( 'Correre è brutto', 'brutto', 'bello');
```

restituisce la frase `Correre è bello`.

La funzione REPLICATE ripete una determinata stringa per il numero di volte indicato dal secondo parametro, cioè

```
REPLICATE(StringaDaRipetere , NumeroDiRipetizioni).
```

Ed ecco una veloce query di esempio, che a partire da questo comando:

```
SELECT REPLICATE ( 'Yo' , 2 );
```

restituisce `YoYo`.

Volendo replicare solo spazi, anziché REPLICATE(' ',n), possiamo utilizzare la funzione SPACE, che accetta un solo argomento, numerico, che indica per quante volte replicare lo spazio.

Leggermente più complessa la funzione **STUFF**, che "infilà" una stringa in un'altra stringa.

La sintassi è questa

```
STUFF(StringaDaModificare , PosizioneIniziale, NumeroDiCaratteriDaSostituire,  
StringaSostitutiva ).
```

Avremo, in questo modo

```
SELECT STUFF ( 'Pasta senza glutine' , 7, 5, 'con' );
```

che restituisce `Pasta con glutine`, avendo sostituito i cinque caratteri successivi al settimo con la stringa "con".

5.2.5 Formattazione di una stringa

Terminiamo questa carrellata di funzioni stringa con alcune funzioni utili, **UPPER**, **LOWER**, **LTRIM**, **RTRIM**. Il loro scopo è immediato, come la loro sintassi. Ciascuna di esse ha come input un solo parametro, ovvero la stringa che si vuol modificare.

La prima e la seconda servono rispettivamente a rendere tutta maiuscola una determinata stringa, o a renderla tutta minuscola.

Ad esempio,

```
SELECT UPPER ( 'nasa' );
```

resituisce `NASA`.

LTRIM ed **RTRIM** servono invece per liberarsi rispettivamente degli spazi a sinistra o a destra di una stringa. Se questa stringa non ha spazi in testa e viene applicata la **LTRIM**, la stringa non viene modificata.

```
SELECT LTRIM ( ' Frodo' )
```

resituisce `"Frodo"`

5.3 Altre funzioni utili

5.3.1 La funzione CHOOSE

A chiusura di questo capitolo vorrei menzionare ancora qualche funzione che potrebbe tornare comoda, ad esempio la funzione `CHOOSE`, vediamo un esempio:

```
SELECT CHOOSE (3,  
              'PrimoValore',  
              'SecondoValore',  
              'TerzoValore',  
              'QuartoValore' )
```

ci dà `TerzoValore`, dato che il primo argomento indica la posizione del valore da estrarre dall'elenco degli `n` valori successivi.

5.3.2 La funzione STRING_AGG

Può essere utile anche sapere dell'esistenza della funzione `STRING_AGG`, usata in combinazione con la clausola `GROUP BY`, che raggruppa i valori di una colonna in valori separati da virgole a seconda del loro raggruppamento:

```
SELECT STRING_AGG(Nome, ', ') AS ListaNomi,  
       Cognome  
FROM dbo.Hobbits  
GROUP BY Cognome;
```

dà infatti origine a:

Nome	Cognome
Bilbo, Frodo	Baggins
Merry	Brandybuck
...	...

5.3.3 La funzione `STRING_SPLIT`

Se abbiamo una stringa che contiene un elenco di valori uno di seguito all'altro, separati da un determinato separatore, ma volessimo visualizzarli come se ciascuno fosse la riga di una tabella, possiamo usare la funzione `STRING_SPLIT`, che accetta due argomenti, di cui il primo è la stringa contenente i valori che vogliamo separare, il secondo invece ci chiede di specificare il separatore.

Se scriviamo in questo modo, quindi

```
SELECT *  
FROM STRING_SPLIT('Bilbo,Frodo,Merry,Sam,Pippin', ',');
```

otterremo

<u>value</u>
Bilbo
Frodo
Merry
Sam
Pippin

Notare che il valore di default del nome della tabella è `value`.

Volessimo dare un alias a questa colonna, ci basterebbe scrivere in questo modo:

```
SELECT [value] AS Nomi  
...
```

Notare che `value` è stato inserito tra parentesi quadre, trattandosi di una parola riservata.

5.3.4 La funzione `NULLIF`

Scopo della funzione `NULLIF` è prendere due argomenti che vengono confrontati fra loro: se uguali viene restituito `NULL`, se diversi, viene restituito il primo valore.

```
SELECT NULLIF ( 'Frodo', 'Frodo' )
```

```
SELECT NULLIF ( 'Frodo', 'Bilbo' )
```

Nel primo caso, avremo NULL, nel secondo, Frodo.

Se volessimo esprimerla attraverso un costrutto CASE ... WHEN, potremmo scriverla così

```
SELECT CASE WHEN valore1 = valore2 THEN NULL
        ELSE valore1
END
```

Quick tip: Come si comporta la funzione NULLIF, se mettiamo un NULL tra i suoi parametri?

```
SELECT NULLIF('Bilbo', NULL)
```

In questo caso avremo come risultato della select Bilbo, mentre se proviamo ad eseguire un comando del genere

```
SELECT NULLIF ( NULL, 'Bilbo' )
```

Otterremo questo brutto errore:

```
Msg 4151, Level 16, State 1, Line 1
```

```
The type of the first argument to NULLIF cannot be the NULL
constant because the type of the first argument has to be known.
```

Ovviamente tutti gli esempi mostrati finora facevano utilizzo di stringhe fisse, ma tutte queste funzioni possono essere applicate in delle normali SELECT su colonne di una tabella, es.

```
SELECT Nome,
        NULLIF ( Cognome, 'Brandybuck' ) AS Cognome
FROM dbo.Hobbits
ORDER BY Cognome ASC;
```

restituisce

Nome	Cognome
Merry	NULL
Gollum	NULL
Frodo	Baggins
Bilbo	Baggins
...	...

Quick tip: Soffermiamoci per un attimo a notare una particolarità della query appena vista. Abbiamo già citato il fatto che la clausola **ORDER BY** viene applicata dopo la **SELECT**, e pertanto può riferirsi alle colonne con i loro alias. In questo caso stiamo ordinando per la colonna **Cognome**, ma "Cognome" è sia il nome originale della colonna, sia il suo alias: quale dei due sceglie SQL Server? La risposta è: il secondo. Lo desumiamo dal fatto che il record di "Merry" è stato messo all'inizio (ricordiamo che i NULL vengono sempre messi in testa ad un ordinamento ascendente), ovvero ordinando la colonna come si presenta *dopo* l'applicazione della funzione.

Esiste la possibilità (benché altamente sconsigliabile), di dare due alias uguali a colonne diverse, oppure assegnare ad una colonna un alias uguale al nome di un'altra colonna, come nel caso seguente

```
SELECT Nome,  
        Cognome AS Nome  
FROM dbo.Hobbits;
```

Se proviamo ad eseguire questa query non otterremo alcun errore. Diverso invece il caso in cui provassimo ad aggiungere alla fine un **ORDER BY Nome**, perché in quel caso SQL Server non sarebbe in grado di decidere su quale dei due eseguire l'operazione di ordinamento. Agendo così otterremmo un errore simile:

Msg 209, Level 16, State 1, Line 46

Ambiguous column name 'Nome'.

5.4 Operatori Bitwise

Gli operatori bitwise eseguono manipolazioni di bit tra due espressioni di uno qualsiasi dei tipi di dati della categoria dei tipi di dati interi.

Gli operatori bitwise convertono due valori interi in bit binari, eseguono l'operazione AND, OR o NOT su ciascun bit, producendo un risultato. Successivamente, convertono il risultato in un numero intero.

Ad esempio, l'intero 170 si converte in binario 1010 1010.

Vediamo di seguito qualche esempio del loro utilizzo

5.4.1 Operatore AND

Una volta convertiti in binario due numeri, le loro cifre vengono confrontate una per una: se entrambe sono 1, il risultato sarà 1, altrimenti 0.

Se applichiamo l'operatore & ai due numeri 170 e 75, avremo perciò

170	1010 1010
75	0100 1011
10	0000 1010

difatti, se scriviamo in questo modo:

```
SELECT 170 & 75
```

otterremo come risultato proprio 10.

5.4.2 Operatore OR

Una volta convertiti in binario due numeri, le loro cifre vengono confrontate una per una: se anche una sola è 1, il risultato sarà 1, altrimenti 0.

Se applichiamo l'operatore `&` ai due numeri 170 e 75, avremo perciò

170		1010 1010
75		0100 1011
235		1110 1011

difatti, se scriviamo in questo modo:

```
SELECT 170 | 75
```

otterremo come risultato 235.

Quick tip: Si può facilmente notare come due numeri uguali, confrontati sia con l'operatore bitwise **AND** (`&`), sia con l'operatore **OR** (`|`), daranno come output il numero medesimo.

```
SELECT 21 | 21
```

restituisce 21, come anche

```
SELECT 21 & 21
```

Capitolo 6

DDL: creare, distruggere e modificare oggetti

In questo capitolo vedremo come creare e manipolare alcuni oggetti del database, a partire da quello principale, ovvero le tabelle. Questo tipo di manipolazione, come anticipato nel primo capitolo, prende il nome di **Data Definition Language**.

6.1 Creare una tabella - Il comando CREATE

In SQL Server abbiamo a disposizione due metodi per creare una tabella:

- Tramite il comando `CREATE TABLE`, che conterrà tutte le indicazioni necessarie per la creazione di una tabella
- Tramite il comando `SELECT INTO`, che, a partire da un comando di `SELECT`, creerà e popolerà direttamente una tabella a partire dai risultati di tale `SELECT`.

Cominciamo subito a vedere lo script con cui abbiamo creato la tabella `Hobbits` che abbiamo interrogato finora:

```
CREATE TABLE Hobbits
(
    Nome VARCHAR(50),
    Cognome VARCHAR(50)
);
```

Quello che abbiamo appena scritto è il modo più stringato per creare una nuova tabella, non il migliore, intendiamoci, ma contiene già tutto quello che ci serve per definire le caratteristiche di una nuova tabella, ovvero

- Il nome della tabella
- Il nome delle colonne
- Il tipo delle colonne

In teoria, con una tabella così definita è già possibile inserire dati, eseguire interrogazioni, modificare record ecc., insomma, eseguire tutte le operazioni che consentono di sfruttare una tabella.

Abbiamo tralasciato molti aspetti, primo tra tutti lo schema di tale tabella, argomento di cui avevamo già discusso nel paragrafo 13.8.10, e in secondo luogo l'indicazione se la colonna può avere record NULL oppure no. In entrambi i casi verrà applicato il default, ovvero lo schema `dbo` come schema della tabella (che sarà quindi creata come `dbo.Hobbits`), e le colonne che accetteranno NULL.

Vediamo allora come diventerebbe il codice per creare una tabella, indicando esplicitamente queste due caratteristiche.

```
CREATE TABLE dbo.Hobbits
(
    Nome VARCHAR(50) NOT NULL,
    Cognome VARCHAR(50) NULL
);

GO
```

Quick tip: Una tabella può appartenere ad uno ed un solo schema: esiste la possibilità di avere due tabelle con lo stesso nome e due schema diversi nello stesso database, per questo motivo è buona pratica indicare sempre lo schema di una tabella quando si fa una select.

Quick tip: Non è possibile creare degli schema innestati uno dentro l'altro, o con rapporti gerarchici rispetto ad altri schemi: una tabella avrà sempre e solo questa forma

`[Database] . [Schema] . [Tabella]`

Ma vediamo nel dettaglio quali sono le proprietà di una tabella che possiamo specificare al momento della sua creazione

- Il nome del database su cui creiamo la tabella
- Il nome dello schema a cui appartiene la tabella
- Il nome della tabella
- La definizione delle colonne, che comprende

il nome della colonna

il tipo della colonna (INT, VARCHAR etc.)

l'estensione della colonna (nel nostro caso 50 per entrambe le colonne)

la precisione della colonna per tipi di dato numerico (es. NUMERIC(10,5))

tipi di colonna particolari (es. IDENTITY)

la collation della colonna (che indica come confrontare i dati testuali: se case sensitive etc.)

- La definizione di eventuali colonne calcolate
- La definizione dei constraint

se una colonna accetta NULL oppure no

eventuali valori di default

chiavi primarie

chiavi esterne

vincoli di unicità

- Le opzioni della tabella

filegroup

partizionamento

compressione

È pratica abbastanza comune, e non particolarmente deprecata, definire queste proprietà in un secondo momento: dopo aver creato la tabella con la sua struttura principale tramite la `CREATE TABLE`, possiamo infatti modificarla tutte le volte che vogliamo (ma facendo attenzione!) tramite il comando `ALTER TABLE`, che vedremo nel paragrafo 6.2.

Per creare uno schema, invece basterà eseguire il comando

```
CREATE SCHEMA schema_name
    [AUTHORIZATION owner_name];
GO
```

In cui la parte tra parentesi quadre, in cui si specifica il nome dell'owner, è opzionale.

In dettaglio: Abbiamo la possibilità di trasferire una tabella da uno schema ad un altro, eseguendo questo comando

```
ALTER SCHEMA LOTR TRANSFER dbo.Hobbits;
```

6.1.1 Dare nomi alle colonne

Non c'è limite alla fantasia nel momento in cui vogliamo dare il nome ad una colonna, tuttavia, oltre al buonsenso, ci sono delle altre regole da seguire. Parte di queste regole le avevamo già incontrate nel paragrafo , quando abbiamo parlato di come dare gli alias alle colonne di una select.

Esistono due tipi di identificatori: regolari e delimitati.

Gli identificatori regolari sono nomi che seguono regole più restrittive, e non hanno bisogno di essere racchiusi da delimitatori come parentesi quadre ([]) o apici doppi (" "). Per gli identificatori regolari, i caratteri possono essere:

Ecco le regole dettagliate che devono essere rispettate:

- **Lunghezza:**

Gli identificatori devono essere lunghi almeno un carattere e non più di 128 caratteri.

- **Caratteri consentiti:**

Il primo carattere deve essere una lettera definita nello Standard Unicode 3.2 o un trattino basso (_), e non può essere una cifra.

I caratteri successivi possono includere:

- Lettere come definite nello Standard Unicode 3.2.
- Numeri decimali dal latino di base (0 a 9) o altri script nazionali.
- Il simbolo della chiocciola (@), il simbolo del dollaro (\$), il cancelletto (#) e il trattino basso (_).

- **Eccezioni:**

Le variabili devono iniziare con una chiocciola (@).

Le tabelle temporanee devono iniziare con un cancelletto (#).

- **Restrizioni:**

Un identificatore regolare non può essere una parola riservata di T-SQL.

Non può includere spazi incorporati o caratteri speciali diversi da quelli precedentemente menzionati.

Esempi di identificatori regolari validi:

- CustomerName

- `_EmployeeID`
- `Order123`
- `@VariableName` (per variabili)
- `#TempTable` (per tabelle temporanee)

Esempi di identificatori non validi:

- `123Table` (inizia con una cifra)
- `Order Name` (contiene uno spazio)
- `SELECT` (parola riservata di T-SQL)
- `Order!ID` (contiene un carattere speciale non consentito)

Seguendo queste regole, è possibile garantire che gli identificatori utilizzati in SQL Server siano validi e non causino errori durante la scrittura o l'esecuzione di query SQL.

Il fatto che, grazie alle parentesi quadre, sia possibile dare nomi estrosi come `[Columna'$]`, non vuol dire che sia *bello*. Con il set standard dei caratteri latini, volendo numeri e underscore, nel 99.9 % dei casi abbiamo tutto quello che ci occorre per dare alle nostre colonne nomi chiari e parlanti.

6.1.2 Assegnare il tipo ad una colonna

Abbiamo già parlato dettagliatamente nella sezione 4.2 dei tipi di dati in SQL Server, ci limiteremo quindi un semplice ripasso delle regole d'oro da seguire

- Cercate di utilizzare il tipo di dato più efficiente: uno che richieda la minor quantità di spazio su disco, rappresenti adeguatamente i dati e che non debba essere cambiato in un secondo momento quando la tabella viene popolata.
- Quando è necessario memorizzare stringhe di caratteri, se è probabile che varieranno in lunghezza, utilizzate i tipi di dati `NVARCHAR` o `VARCHAR` piuttosto che i fissi `NCHAR` o `CHAR`. Se il valore della colonna potrebbe essere aggiornato

spesso, e specialmente se è corto, utilizzare la lunghezza fissa può prevenire un eccessivo movimento delle righe.

- I tipi di dati `DATE`, `TIME` e `DATETIME2` possono memorizzare dati in modo più efficiente e con una precisione migliore rispetto a `DATETIME` e `SMALLDATETIME`.
- Utilizzate `VARCHAR(MAX)`, `NVARCHAR(MAX)` e `VARBINARY(MAX)` invece dei tipi di dati deprecati `TEXT`, `NTEXT` e `IMAGE`.
- Utilizzate `ROWVERSION` invece del deprecato `TIMESTAMP`.
- `DECIMAL` e `NUMERIC` sono lo stesso tipo di dato, ma generalmente si preferisce `DECIMAL` perché il nome è un po' più descrittivo. Utilizzate `DECIMAL` e `NUMERIC` invece dei tipi di dati `FLOAT` o `REAL` a meno che non abbiate veramente bisogno di precisione a virgola mobile e siate coscienti dei possibili problemi di arrotondamento.

6.1.3 Valori di default

Abbiamo già visto un'infinità di volte quanti problemi possa dare una colonna nullable; nel caso si voglia evitare di trattare i record `NULL` e si abbia un'alternativa accettabile per rimpiazzarli (vedi, ad es. una stringa vuota), è possibile definire un valore di default per una colonna, quando non viene specificato nessun record da inserire.

In fase di creazione della nostra tabella `Hobbits`, avremmo quindi potuto scrivere in questo modo

```
CREATE TABLE dbo.Hobbits
(
    Nome VARCHAR(50) NULL,
    Cognome VARCHAR(50) NOT NULL DEFAULT ('')
);

GO
```

Se invece avessimo voluto aggiungere il valore di default dopo aver creato la tabella, avremmo dovuto usare questa sintassi

```
ALTER TABLE [dbo].[Hobbits]
ADD CONSTRAINT df_ConstraintCognome
    DEFAULT ( ' ' ) FOR Cognome;

GO
```

Quick tip: il fatto di aggiungere il default ad una colonna dopo che questa ha già dei dati, non ha effetto retroattivo sui dati stessi. In questo caso, provando ad eseguire questa select dopo aver modificato il default della colonna,

```
SELECT Nome, Cognome
FROM dbo.Hobbits
WHERE Cognome IS NULL
```

troviamo ugualmente il record relativo a `Gollum`

6.1.4 Chiavi primarie e surrogate

Introduciamo in questa sezione un concetto che non abbiamo ancora affrontato, ovvero quello di **chiave primaria** di una tabella.

La chiave primaria (*Primary Key*) è data da una colonna o un insieme di colonne che identificano univocamente ogni record (riga) in una tabella. Le caratteristiche principali di una chiave primaria sono:

1. **Unicità:** Ogni valore della chiave primaria deve essere unico per ogni riga della tabella. Non possono esistere due righe con la stessa chiave primaria.
2. **Non Null:** Gli attributi che compongono la chiave primaria non possono avere valori nulli. Ogni riga deve avere un valore di chiave primaria.

3. **Immutabilità:** I valori della chiave primaria non dovrebbero cambiare una volta assegnati. Il motivo diventerà chiaro quando tratteremo l'integrità referenziale nei collegamenti tra tabelle (capitolo 9).
4. **Semplicità:** Benché la definizione di "semplice" sia piuttosto vaga, l'indicazione è sempre quella di preferire una chiave numerica anziché una testuale.

Se abbiamo una tabella di anagrafica come quella **Hobbits** utilizzata finora, definire una chiave primaria potrebbe essere un compito non facile. Potremmo scegliere come chiave primaria la colonna **Nome**, ma la cosa comporterebbe un certo rischio, dato che in futuro potremmo voler inserire un nuovo hobbit nella tabella che si chiama Sam, o Frodo, o Bilbo, violando così il vincolo di unicità della chiave primaria. Una soluzione potrebbe essere quella di estendere la chiave primaria a entrambe le colonne della tabella, ovvero **Nome** e **Cognome**, ma, oltre a non poter di nuovo escludere che un giorno abbiamo ci sia la necessità di inserire un nuovo "Frodo Baggins" diverso da quello già presente in tabella, c'è l'ulteriore problema che abbiamo impostato la colonna cognome come nullable.

In occasioni come queste ci viene in aiuto il concetto di chiave surrogata (un tipo particolare di chiave primaria), ovvero una colonna inventata (solitamente costituita da un numero progressivo), che soddisfi tutti i vincoli necessari per una chiave primaria. In questo modo avremo:

HobbitId	Nome	Cognome
1	Bilbo	Baggins
2	Frodo	Baggins
3	Merry	Brandybuck
4	Sam	Gamgee
5	...	...

Quick tip: Potrebbe venirci il sospetto che, una volta stabilita una chiave primaria, quando interroghiamo una tabella senza specificare l'ordinamento, i risultati

ci vengano proposti ordinati secondo quella chiave primaria. Non è così: nonostante capiti la maggior parte delle volte, non possiamo mai avere la garanzia che sia così *tutte* le volte. La regola d'oro resta sempre: se vuoi che i tuoi risultati siano ordinati, specifica sempre la clausola di `ORDER BY`.

6.1.5 PRIMARY KEY e IDENTITY

Vediamo ora nella pratica come creare una chiave primaria, e come impostarne una surrogata.

Se volessimo dichiarare come chiave primaria la (sconsigliabile) colonna `Nome`, tutto quello che dovremmo fare sarebbe scrivere in questo modo la creazione della tabella

```
CREATE TABLE dbo.Hobbits
(
    Nome VARCHAR(50) NOT NULL PRIMARY KEY,
    Cognome VARCHAR(50) NULL
);
```

GO

quindi aggiungendo la dicitura `PRIMARY KEY` in fase di creazione della tabella accanto alla colonna che vogliamo eleggere come chiave primaria.

Se volessimo invece (consigliabile) creare la tabella con una colonna aggiuntiva `HobbitId` autoincrementante che costituirà la nostra chiave surrogata, dovremo scrivere in questo modo:

```
CREATE TABLE dbo.Hobbits
(
    HobbitId INT NOT NULL PRIMARY KEY IDENTITY(1,1),
    Nome VARCHAR(50) NOT NULL,
    Cognome VARCHAR(50) NULL
);
```

GO

quindi aggiungendo il comando `IDENTITY(m,n)`, dove `m` è il numero intero da cui partirà la chiave primaria, e `n` indica invece l'incremento che verrà aggiunto ad ogni nuovo inserimento.

Nel caso scrivessimo, ad esempio `IDENTITY(0,2)`, il primo valore della chiave surrogata sarà 0, e tutti i valori successivi saranno numeri pari. C'è anche la possibilità di prevedere un decremento per i valori della chiave surrogata, vale a dire, specificando un comando come `IDENTITY(-1,-1)`, partiremo dal valore -1, per poi passare a -2 col secondo record e così via a continuare la sequenza. Sicuramente lo svantaggio che comporta l'utilizzo di una chiave surrogata rispetto ad una chiave naturale, è che per un umano ha un significato molto meno leggibile e immediato.

Quick tip: Nelle varie occasioni in cui ci troveremo a scegliere il tipo di dato da usare, sicuramente quello di una chiave surrogata è cruciale. Se la tabella che dobbiamo popolare è quella degli hobbit, probabilmente un `INT` andrà benissimo, ma se c'è la possibilità che la tabella possa superare i due miliardi di record, prendiamo in seria considerazione l'uso del `BIGINT`. Adottare un tipo di dato troppo piccolo potrebbe rendere impossibile scrivere nuovi record in una tabella.

Quando creiamo una chiave surrogata con il comando `IDENTITY(m,n)`, occorre essere coscienti del fatto che, se cancelliamo un dato e ne inseriamo successivamente un altro, il numero corrispondente continuerà a salire, lasciando potenzialmente un buco. Questo accade anche se cancelliamo tutti i record di una tabella. Se creiamo una tabella con `IDENTITY(1,1)`, inseriamo 1000 record e li cancelliamo tutti, il prossimo record che inseriremo avrà come chiave surrogata il valore 1001. Se vogliamo resettare la sequenza di una tabella vuota, occorrerà utilizzare questo comando.

```
DBCC CHECKIDENT('dbo.Hobbits', RESEED, 1);
```

GO

Dove con il valore 1 intendiamo quello da cui vogliamo ripartire.

Al momento della creazione di una chiave primaria, SQL Server rinforza il vincolo di unicità creando un indice univoco su quella colonna, e utilizzando la colonna stessa come chiave dell'indice.

In dettaglio: Per ottenere la lista delle chiavi primarie presenti nel database, si può interrogare la tabella `sys.key_constraints`, filtrando per i valori di tipo PK.

```
SELECT *  
FROM sys.key_constraints  
WHERE type = 'PK';
```

6.1.6 Colonne calcolate

Volendo, è possibile definire delle colonne di una tabella partendo dalle altre colonne, in questo caso parliamo di colonne calcolate: il loro valore viene calcolato di volta in volta da SQL Server.

Se, per esempio, volessimo una colonna in cui viene visualizzato nome e cognome degli hobbit, con il cognome scritto in maiuscolo, anziché scrivere una query apposta, potremmo definire la tabella in questo modo:

```
CREATE TABLE dbo.Hobbits  
(  
    Nome VARCHAR(50) NOT NULL,  
    Cognome VARCHAR(50) NULL,  
    NomeCognome AS CONCAT(Nome, ' ', UPPER(Cognome)) --colonna calcolata  
);  
  
GO
```

Ovviamente la cosa si può applicare anche alle colonne numeriche, applicando le appropriate operazioni matematiche.

Esiste la possibilità di rendere la colonna calcolata persistente, il che significa che SQL Server memorizzerà i valori calcolati e non li dovrà calcolare al volo, operazione che favorisce la velocità, anche se è più pesante in termini di spazio occupato. Tuttavia, se una colonna calcolata deve essere persistente, la colonna non può utilizzare funzioni non deterministiche, il che significa che l'espressione non può fare riferimento a funzioni che danno valori diversi in momenti diversi, come `GETUTCDATE()`.

6.2 Modificare una tabella - Il comando `ALTER`

Dopo aver creato una tabella, si può utilizzare il già citato comando `ALTER TABLE` per cambiare la struttura della tabella e aggiungere o rimuovere determinate proprietà. Possiamo servirci del comando `ALTER TABLE` per:

- Aggiungere o rimuovere una colonna, inclusa una colonna calcolata. (Le nuove colonne sono posizionate alla fine dell'ordine delle colonne della tabella.)
- Cambiare il tipo di dato di una colonna.
- Cambiare la nullabilità di una colonna (cioè, da `NULL` a `NOT NULL`, o viceversa).
- Aggiungere o rimuovere un vincolo, inclusi i seguenti:

Vincolo di chiave primaria

Vincolo unico

Vincolo di chiave esterna

Vincolo di controllo

Vincolo di default

Non possiamo invece servirci del comando `ALTER TABLE` per:

- Cambiare il nome di una colonna.

- Aggiungere una proprietà di identità.
- Rimuovere una proprietà di identità.
- Scegliere gli indici della tabella.

Il fatto di poter modificare la gestione degli indici di una tabella dopo averla creata è una pratica molto diffusa, perché a seconda di come la tabella viene interrogata (vengono tirati fuori tanti dati o pochi? Quali colonne vengono tirate fuori? Quali sono le colonne su cui vengono impostate più spesso le condizioni della **WHERE?**), potremmo avere la necessità di modificare, attraverso gli indici, il criterio con cui SQL Server cerca i dati in una tabella ai fini di ottimizzare le interrogazioni.

In dettaglio: In certi casi, una tabella può essere modificata tramite il comando **ALTER TABLE** anche se contiene già dei dati, ad es. se volessimo rendere impossibile inserire dei **NULL** nella colonna **Nome** della nostra tabella **Hobbits**, potremmo scrivere in questo modo

```
ALTER TABLE dbo.Hobbits
ALTER COLUMN Nome VARCHAR(50) NOT NULL;
GO
```

Diverso è il discorso se applichiamo lo stesso comando alla colonna **Cognome**: dato che il record relativo al cognome di **Gollum** contiene già un **NULL**, riceveremo il seguente messaggio di errore:

```
Msg 515, Level 16, State 2, Line 1
Cannot insert the value NULL into column 'Cognome', table
'Academy.dbo.Hobbits'; column does not allow nulls. UPDATE fails.
```

Quindi sì, possiamo cambiare le caratteristiche di una colonna su una tabella che già contiene dei dati, a patto che queste nuove caratteristiche non entrino in conflitto con i dati preesistenti.

Per risolvere il problema in questo caso, ci basterà scrivere in questo modo

```
ALTER TABLE dbo.Hobbits
```

```
ALTER COLUMN Nome VARCHAR(50) NOT NULL DEFAULT '';
```

```
GO
```

, ovvero specificando il default contestualmente alla modifica della colonna che vogliamo rendere non più NULLABLE.

6.3 Eliminare una tabella - Il comando DROP

Droppare (eliminare) una tabella in T-SQL è un'operazione piuttosto semplice, ma va eseguita con cautela poiché rimuove definitivamente la tabella e tutti i dati in essa contenuti.

Per eliminare una tabella, si utilizza il comando `DROP TABLE` seguito dal nome della tabella. Ecco la sintassi di base:

```
DROP TABLE dbo.Hobbits;
```

```
GO
```

Se una tabella appartiene allo schema `dbo` possiamo anche non specificarlo, ma è sempre consigliabile metterlo per esteso in ogni caso.

Se proviamo ad eliminare una tabella che non esiste, perché magari abbiamo scritto male il nome, riceveremo questo errore

```
Msg 3701, Level 11, State 5, Line 1
```

```
Cannot drop the table 'Hobbitsss', because it does not exist or you do  
not have permission.
```

Una buona pratica consiste nell'utilizzare questa sintassi al momento di droppare una tabella

```
DROP TABLE IF EXISTS dbo.Hobbitsss;
```

```
GO
```

che ci salvaguarda da errori di questo genere, dato che controlla l'esistenza di una tabella prima di provare a eliminarla.

Ricordiamoci anche che, se la tabella ha vincoli di dipendenza da altre tabelle, potrebbe essere necessario rimuoverli prima di poter droppare la tabella.

Un'ultima, importante puntualizzazione è che, essendo l'eliminazione di una tabella un'operazione parecchio distruttiva, è necessario avere i permessi appropriati per poterlo fare, cosa che si evince anche dal messaggio di errore che abbiamo incontrato poco sopra.

6.4 Svuotare una tabella - Il comando TRUNCATE

Il comando **TRUNCATE** in T-SQL è utilizzato per rimuovere rapidamente tutti i dati da una tabella, senza registrare individualmente le eliminazioni delle righe. Si tratta di un modo molto veloce ed efficiente per svuotare completamente una tabella, tuttavia, ci sono alcune considerazioni importanti e limitazioni da tenere a mente.

La sintassi base per il comando **TRUNCATE** è:

```
TRUNCATE TABLE dbo.Hobbits;  
GO
```

Teniamo presente che, rispetto al comando **DROP TABLE**, la **TRUNCATE** mantiene la tabella, limitandosi a svuotarne il contenuto.

Se la tabella contiene una colonna con un identificatore univoco a incremento automatico (**IDENTITY**), il contatore verrà resettato al valore iniziale predefinito.

Capitolo 7

DML: creare, distruggere e modificare record

Se nella scorsa sezione avevamo imparato come manipolare gli oggetti del database (tabelle, viste, etc), in questo qui impareremo come manipolare i dati all'interno di una tabella,

DML	Azione	DDL
CREATE	Creare	INSERT
DROP	Distruggere	DELETE
ALTER	Modificare	UPDATE

7.1 Il comando INSERT

7.1.1 INSERT VALUES

Il primo comando che incontriamo in questo capitolo serve proprio per inserire dei record all'interno di una tabella: avremo una sintassi simile

```
INSERT INTO dbo.Hobbits  
(  
    Nome,
```

```
        Cognome
    )
VALUES
('Grassotto', 'Bolgeri');
```

Se il nostro comando popola tutte le colonne di una tabella, come in questo caso, possiamo anche omettere la parte di codice in cui ne specifichiamo esplicitamente il nome, in questo modo

```
INSERT INTO dbo.Hobbits
VALUES
('Grassotto', 'Bolgeri');
```

Per ragioni di chiarezza, però, è sempre consigliabile indicare tutto per esteso.

Chiaramente, nel caso avessimo una chiave surrogata autoalimentante (chiamata ad es. HobbitId), l'aspetto della insert non cambierebbe dato che, non solo non è necessario indicare un valore per una colonna che incrementa da sola il suo valore, ma, cercare di farlo restituirebbe un errore.

Se però ci trovassimo nella situazione di dover inserire forzatamente un valore in una colonna identity, dovremmo utilizzare la seguente opzione

```
SET IDENTITY_INSERT dbo.Hobbits ON;

INSERT INTO dbo.Hobbits
(
    HobbitId,
    Nome,
    Cognome
)
VALUES
(9, 'Grassotto', 'Bolgeri');
```

```
SET IDENTITY_INSERT dbo.Hobbits OFF;
```

È di cruciale importanza ricordarsi di eseguire il comando opposto `SET IDENTITY_INSERT dbo.Hobbits OFF;`, per ripristinare la colonna in modo che torni ad autoalimentarsi. In tutto il DB questa opzione può essere espressa solamente per una tabella alla volta: per inserire esplicitamente un valore nella colonna `IDENTITY` di una nuova tabella, sono costretto prima a disabilitare l'`IDENTITY_INSERT` sulla prima.

Quick tip: per poter eseguire questa operazione sono necessari permessi abbastanza estesi sulla tabella, ovvero devi esserne owner, e avere il permesso di `ALTER` sulla tabella stessa.

Esistono altri casi in cui non è necessario inserire esplicitamente il valore di una colonna anche se questa non ammette l'inserimento di `NULL`, ovvero per le colonne su cui è stato definito un valore di default. A differenza di una colonna su cui è specificato il vincolo di `IDENTITY`, possiamo indicare esplicitamente il valore che vogliamo dare ad una colonna che ha un default senza dover dare nessuna opzione particolare.

Se avessi un gran numero di righe da inserire, posso farlo senza scrivere per esteso tutte le volte il comando `INSERT INTO`, basta che separi tramite virgola tutte le righe che voglio inserire, in questo modo:

```
INSERT INTO dbo.Hobbits
(
    Nome,
    Cognome
)
VALUES
('Begonia'      , 'Gamgee'),
('Asfodelia'    , 'Brandibuck'),
```



```
('Belladonna' , 'Took');
```

Ci sono due aspetti a cui bisogna prestare attenzione nell'utilizzare questo comando

1. Se il numero di righe che si vogliono inserire in un colpo solo supera le 1000, il comando va in errore
2. Se una qualsiasi delle insert accorpate in questo modo va in errore (es. si sta cercando di inserire un NULL in una colonna non nullable), non ne viene eseguita neppure una

7.1.2 INSERT SELECT

In T-SQL abbiamo la possibilità di inserire record in una tabella non solo fornendo un elenco di valori, ma anche con il risultato di una **SELECT**. Nel nostro caso, se avessi una tabella `dbo.Baggins` in cui inserire tutti gli hobbit con cognome "Baggins" della tabella `dbo.Hobbits`, potrei scrivere in questo modo

```
INSERT INTO dbo.Baggins
(
    Nome,
    Cognome
)
SELECT
    Nome,
    Cognome
FROM dbo.Hobbits
WHERE Cognome = 'Baggins'
```

Va da sé che la struttura della tabella `dbo.Baggins` deve essere compatibile con la tabella di origine, il che significa che i nomi delle colonne di origine e destinazione potrebbero essere diversi, ma i tipi devono essere confrontabili. Possiamo ad esempio trasferire i dati da una tabella con colonna `varchar(10)` ad una `varchar(5)`, ma

solo a patto che i dati trasferiti non superino mai i venti caratteri: in caso contrario riceveremo questo messaggio d'errore.

Msg 2628, Level 16, State 1, Line 17

**String or binary data would be truncated in table 'dbo.tabella',
column 'colonna'. Truncated value: '1234567'.**

Lo stesso discorso vale per tipi diversi di colonne, ovvero, è consentito il passaggio da una colonna testuale a una colonna numerica, purché la colonna testuale contenga solo numeri, e questi numeri non eccedano la capacità del tipo numerico di destinazione.

Non è necessario nemmeno popolare tutte le colonne della tabella di destinazione, sempre che queste colonne non popolate siano nullabili, o che per esse siano stati definiti dei valori di default .

7.2 Il comando UPDATE

Proprio come esiste la possibilità di inserire ex-novo dei dati da una tabella, abbiamo anche la possibilità di modificarli.

La sintassi è la seguente:

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

UPDATE table_name: Specifica la tabella che contiene i dati da modificare.

SET column1 = value1, column2 = value2, ...: Elenca le colonne da aggiornare e i nuovi valori da assegnare a ciascuna colonna. È possibile aggiornare una o più colonne contemporaneamente.

WHERE condition: (Opzionale, ma raccomandato) Specifica le condizioni che determinano quali righe devono essere aggiornate. Senza la clausola **WHERE**, infatti, tutti i record della tabella verranno aggiornati.

Se proviamo ad applicare un'operazione di questo genere al nostro database, magari per aggiornare il NULL a '-' sulla colonna Cognome per il record Gollum, dovremo scrivere in questo modo:

```
UPDATE dbo.Hobbits
SET Cognome = '-'
WHERE Nome = 'Gollum';
```

In modo del tutto simile ad una SELECT, possiamo inserire condizioni multiple nella clausola WHERE con operatori logici come AND e OR.

Possiamo anche operare un aggiornamento su più colonne contemporaneamente, separando ciascuna coppia colonna-valore con una virgola.

```
UPDATE dbo.Hobbits
SET Cognome = '',
    Nome = 'Smeagol'
WHERE Nome = 'Gollum';
```

7.3 Il comando DELETE

L'ultimo tassello ancora mancante è il comando che ci permette di eliminare i dati; avevamo già visto nel paragrafo 6.4 un modo per farlo, ma benché molto veloce ed efficace, aveva due grandi impedimenti:

- Svuotava l'intero contenuto di una tabella, senza la possibilità di scegliere quali righe eliminare
- Se la tabella era relazionata ad altre, l'esecuzione del comando veniva bloccata da un errore

Il comando DELETE, al contrario, non presenta questi inconvenienti: proprio come facciamo quando eseguiamo un'UPDATE o una SELECT, possiamo infatti definire una clausola WHERE.

```
DELETE FROM dbo.Hobbits
WHERE Nome = 'Gollum';
```

Essendo l'oggetto della DELETE l'intera riga di una tabella, è facile notare come non sia necessario specificare in alcun modo le colonne.

Proprio come nell'UPDATE, se si omette la clausola WHERE, il risultato sarà quello di cancellare *tutte* le righe della tabella; se questa è un'operazione non voluta le conseguenze possono essere davvero poco piacevoli.

Se siamo costretti ad eliminare un gran numero di dati da una tabella, tale operazione può essere molto onerosa e generare problemi di inaccessibilità del dato, ad esempio creando un lock sulla tabella. In questi casi è buona norma spezzare in sottoinsiemi più piccoli l'operazione, in modo simile

```
WHILE 1 = 1
BEGIN
    DELETE TOP (1000) FROM dbo.TabellaMoltoGrossa
    IF @@rowcount < 1000 BREAK;
END
```

Si può notare come il loop infinito venga spezzato dal BREAK, dopo che non ci sono più record da cancellare.

7.4 Differenze tra DELETE e TRUNCATE

- **Gestione del transaction log:** Il comando DELETE scrive molte più informazioni nel transaction log rispetto a TRUNCATE, dato che registra nel log i dati effettivamente eliminati; la TRUNCATE registra molte meno informazioni.
- **Reset delle proprietà IDENTITY:** La dichiarazione DELETE non modifica il valore della proprietà IDENTITY associata a una colonna della tabella di destinazione. In poche parole, se cancelliamo il record con id = 10 di una tabella con dieci righe, la insert successiva avrà un id uguale a 11 o superiore, lascian-

do di fatto un buco nella numerazione. La **TRUNCATE** invece resetta il valore proprio come se la tabella fosse stata appena creata.

- **Gestione delle chiavi esterne:** **DELETE** è supportato quando una chiave esterna punta alla tabella padre, purché non vi siano righe correlate nella tabella figlia (vedremo questo elemento in dettaglio nel prossimo capitolo). **TRUNCATE**, invece, non è consentito se una chiave esterna punta alla tabella, anche se non ci sono righe correlate nella tabella di riferimento o se la chiave esterna è disabilitata.
- **Tabelle in viste indicizzate:** **DELETE** è consentito su tabelle coinvolte in viste indicizzate. **TRUNCATE**, invece, non è permesso in questi casi.
- **Permessi richiesti:** **DELETE** richiede i permessi **DELETE** sulla tabella di destinazione. **TRUNCATE**, invece, richiede i permessi **ALTER** sulla stessa tabella.

Capitolo 8

Interrogare più tabelle contemporaneamente

Finora abbiamo visto interrogazioni da una sola tabella per volta, ma questo, alla lunga, può risultare limitante. In questo capitolo affronteremo diversi modi con cui possiamo mettere in relazione più tabelle tra di loro.

8.1 Query di più tabelle senza join

8.1.1 Il comando INTERSECT

In questa sezione introdurremo due nuove tabelle, la tabella `med.Film` e la tabella `med.Libri` (dove `med` sta per *media*). Lo script con la struttura e i dati è disponibile a questo link [Ciascuna di queste due tabelle ha due colonne: Titolo e AnnoDiUscita](#), e al loro interno ritroviamo questi dati

Nome	Cognome
Il Signore degli Anelli	2003
I pirati dei Caraibi	2003
Harry Potter	2001
Memento	2000
Interstellar	2014
Dune	2022
It	1990
2001: Odissea nello spazio	1968

Tabella 8.1: Tabella "med.Film"

Nome	Cognome
Il Signore degli Anelli	1954
Harry Potter	1997
La montagna incantata	1924
It	1986
Dune	1965
Il giovane Holden	1951
Il pendolo di Foucault	1988
2001: Odissea nello spazio	1968

Tabella 8.2: Tabella "med.Libri"

Se vogliamo ottenere l'intersezione dei due insiemi, ovvero l'elenco di quei libri da cui hanno tratto un film, possiamo utilizzare il comando `INTERSECT` in questo modo

```
SELECT [Titolo]
FROM med.Libri
INTERSECT
SELECT [Titolo]
FROM med.Film;
```

Il cui risultato sarà

Quick tip: Il comando `INTERSECT`, come il suo corrispondente `EXCEPT`, che vedremo nel prossimo capitolo, operano implicitamente una `DISTINCT` sui risultati.

Titolo
Dune
Harry Potter
Il Signore degli Anelli
It
2001: Odissea nello spazio

Durante il confronto dei valori di colonna per determinare le righe **DISTINCT**, due valori **NULL** sono considerati uguali.

8.1.2 Il comando **EXCEPT**

Il comando **EXCEPT** restituisce tutte le righe distinte del primo set che non trovano riscontro nel secondo set. In questo modo, la query

```
SELECT [Titolo]
FROM med.Libri
EXCEPT
SELECT [Titolo]
FROM med.Film;
```

restituirà

Titolo
Cent'anni di solitudine
Il giovane Holden
Il pendolo di Foucault
La montagna incantata

Ottenendo tutti i film da cui non sono stati tratti libri.

Ovviamente conta l'ordine con cui scriviamo le due tabelle, perché se le avessimo invertite, avremmo ottenuto tutti i film che non sono stati tratti da un libro.

Le regole di base per la combinazione dei set di risultati di due query che usano **EXCEPT** o **INTERSECT** sono le seguenti:

- Tutte le query devono includere lo stesso numero di colonne nello stesso ordine.
- I tipi di dati devono essere compatibili. Se i tipi di dati Char hanno regole di confronto diverse, viene utilizzata la precedenza delle regole di confronto per eseguire il confronto. Se la conversione non è possibile, si verifica un errore.

Bisogna tenere presente anche questi ulteriori punti

Tipi di Dati: Le query a sinistra e a destra degli operatori **EXCEPT** o **INTERSECT** devono restituire tipi di dati confrontabili. Se i tipi di dati Char hanno regole di confronto diverse, viene utilizzata la precedenza delle regole di confronto. Se la conversione non è possibile, si verifica un errore.

Valori NULL: Durante il confronto dei valori di colonna per determinare le righe **DISTINCT**, due **NULL** sono considerati uguali.

Nomi delle Colonne: **EXCEPT** e **INTERSECT** restituiscono i nomi delle colonne come definiti nella query a sinistra dell'operatore. I nomi o alias delle colonne in clausole **ORDER BY** devono riferirsi ai nomi di colonna della query a sinistra dell'operatore.

8.2 Le join

Per quanto utili, i comandi **EXCEPT** e **INTERSECT** hanno dei limiti, primo fra tutti che non c'è la possibilità di visualizzare colonne che non siano comuni tra le due tabelle. In questo frangente vengono in nostro aiuto le **JOIN**, che diventeranno il modo principale con il quale metteremo in relazione due (o più) tabelle tra di loro.

A seconda del risultato che vogliamo ottenere, potremo utilizzare

- `INNER JOIN`
- `LEFT JOIN`
- `RIGHT JOIN`
- `FULL JOIN`
- `CROSS JOIN`

, che vedremo nel dettaglio una per volta, sempre a partire dal nostro esempio di libri e film.

Per amore di chiarezza, tutti gli esempi che seguiranno faranno riferimento a tabelle senza una vera e propria chiave primaria, o meglio, consideriamo chiave primaria la colonna `Titolo` delle tabelle coinvolte, e difatti opereremo le `JOIN` (poi vedremo cosa vuol dire) su quel campo. In uno scenario reale non sarebbe una scelta ottimale per diversi motivi: nessuno ci garantisce che non esistano due libri (o due film) con lo stesso titolo, in quel caso verrebbe meno il vincolo di univocità; l'altro problema è che confrontare due colonne testuali è sicuramente molto più dispendioso in termini di risorse rispetto a due colonne di interi. Purtroppo l'esempio viene meglio così, ma ovviamente va preso per quel che vale (cioè un esempio) e non una regola di buona programmazione.

8.2.1 La `INNER JOIN`

La `INNER JOIN` permette di confrontare due tabelle, utilizzando una o più colonne come criterio di paragone; i risultati che espone sono dati dall'intersezione degli insiemi delle due tabelle che vengono messe in join. Il suo comportamento è quindi simile, per certi versi, alla `INTERSECT`, dato che una `INNER JOIN` estrae tutti i record che sono comuni alle due tabelle rispetto ad una (o più) colonne di confronto.

Volendo dare una rappresentazione grafica, la inner join risulterebbe così

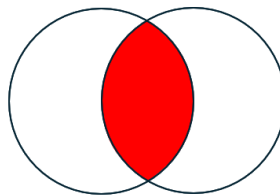


Figura 8.1: Rappresentazione grafica della INNER JOIN

Scrivendo una query del genere, ad esempio

```
SELECT fi.Titolo,  
       li.AnnoDiUscita AS AnnoLibro,  
       fi.AnnoDiUscita AS AnnoFilm  
FROM med.Libri li  
     INNER JOIN med.Film fi  
     ON li.Titolo = fi.Titolo;
```

, troveremo tutti i libri che sono anche dei film.

Quick tip: nella INNER JOIN c'è un rapporto paritario tra le tabelle, quindi invertendone l'ordine nella query, il risultato sarebbe stato identico.

Quick tip: se avessimo scritto la query precedente omettendo la parola INNER, in questo modo

```
[...]  
FROM med.Libri li  
     JOIN med.Film fi  
     ON li.Titolo = fi.Titolo;
```

avremmo ottenuto lo stesso risultato. La dicitura JOIN, presa da sola, intende implicitamente l'uso di una INNER JOIN. Il mio personale suggerimento è di scrivere sempre INNER JOIN per esteso, per eliminare ogni ambiguità, dato che i tipi di join possibili sono molteplici.

Soffermiamoci per un attimo ad esaminare la sintassi: dopo la ben conosciuta **FROM** e il nome della prima tabella **med.Libri** con il suo alias, abbiamo la **INNER JOIN**, seguita dal nome della seconda tabella e il relativo alias. La **ON** successiva serve a esplicitare quali sono le colonne delle due tabelle su cui va eseguito il confronto. Si tratta di una buona pratica (non sempre possibile) operare il confronto tra la chiave primaria della prima tabella, che punta alla chiave esterna della seconda.

Possiamo ottenere non solo la colonna comune (quella del titolo), ma anche le due colonne che indicano gli anni di uscita dei diversi libri e dei film.

Titolo	AnnoLibro	AnnoFilm
Il Signore degli Anelli	1954	2003
Harry Potter	1997	2001
It	1986	1990
Dune	1965	2022
2001: Odissea nello spazio	1968	1968

Si può facilmente notare come le due tabelle abbiano entrambe una colonna chiamata **AnnoDiUscita**, il che ci costringe a specificare, tramite l'alias **li.AnnoDiUscita** e **fi.AnnoDiUscita**, la provenienza delle due. Senza la specifica dell'alias della tabella, avremmo ricevuto questo errore

Msg 209, Level 16, State 1, Line 2

Ambiguous column name 'AnnoDiUscita'.

Se non avessimo specificato un alias per le colonne **AnnoDiUscita**, non avremmo avuto nessun errore, ma avendo le due colonne un significato diverso, ci sarebbe stata ambiguità nel risultato.

Come già anticipato, è possibile estendere il confronto tra le due tabelle a più colonne per ciascuna tabella: volessimo per esempio vedere tutti i libri da cui sono stati tratti dei film e che sono usciti nello stesso anno, possiamo modificare la query di prima in questo modo:

```

SELECT fi.Titolo,
       li.AnnoDiUscita AS AnnoLibro,
       fi.AnnoDiUscita AS AnnoFilm
FROM med.Libri li
      INNER JOIN med.Film fi
      ON li.Titolo = fi.Titolo
      AND li.AnnoDiUscita = fi.AnnoDiUscita;

```

Se abbiamo fatto tutto per bene, dovremmo ottenere una sola riga

Titolo	AnnoLibro	AnnoFilm
2001: Odissea nello spazio	1968	1968

Quick tip: Avremmo potuto ottenere lo stesso risultato anche imponendo la seconda condizione nella clausola `WHERE`, ovvero:

```

SELECT fi.Titolo,
       li.AnnoDiUscita AS AnnoLibro,
       fi.AnnoDiUscita AS AnnoFilm
FROM med.Libri li
      INNER JOIN med.Film fi
      ON li.Titolo = fi.Titolo
 WHERE li.AnnoDiUscita = fi.AnnoDiUscita;

```

8.2.2 La LEFT JOIN

A differenza della `INNER JOIN`, il principio della `LEFT JOIN` è quello di mostrare *tutti* i record della prima tabella, indipendentemente dal fatto che abbiano o no una corrispondenza con la seconda tabella.

La rappresentazione grafica è quindi

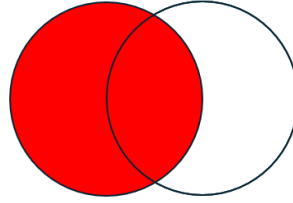


Figura 8.2: Rappresentazione grafica della LEFT JOIN

La sintassi è identica a quanto visto finora, eccetto, ovviamente, la sostituzione di INNER con LEFT. Insieme alla RIGHT JOIN e alla FULL JOIN, questa join fa parte della famiglia delle OUTER JOIN.

```
SELECT li.Titolo,  
       li.AnnoDiUscita AS AnnoLibro,  
       fi.AnnoDiUscita AS AnnoFilm  
FROM med.Libri li  
     LEFT JOIN med.Film fi  
     ON li.Titolo = fi.Titolo;
```

In questo modo otterremo quindi i titoli di tutti i libri e il rispettivo anno di uscita. Se c'è corrispondenza con un record nella tabella med.Film, avremo anche l'anno di uscita del film, altrimenti verrà visualizzato NULL.

Otteniamo perciò

In questo caso, rispetto alla INNER JOIN, è particolarmente importante fare attenzione a quali colonne mostriamo tramite gli alias: avessimo lasciato nella query fi.Titolo (ovvero il titolo del film) anziché li.Titolo (titolo del libro), nella colonna di output avremmo avuto dei valori nulli.

Un altro modo per scrivere la query è specificare la parola OUTER, dopo la JOIN, in questo modo

```
[...]  
FROM med.Libri li
```

Titolo	AnnoLibro	AnnoFilm
Il Signore degli Anelli	1954	2003
Harry Potter	1997	2001
La montagna incantata	1924	NULL
Cent'anni di solitudine	1967	NULL
It	1986	1990
Dune	1965	2022
Il giovane Holden	1951	NULL
2001: Odissea nello spazio	1968	1968
Il pendolo di Foucault	1988	NULL

```
LEFT OUTER JOIN med.Film fi
ON li.Titolo = fi.Titolo;
```

Come non era opportuno omettere la parola **INNER** nella **INNER JOIN**, secondo me non è necessario specificare **OUTER** in una **LEFT JOIN**, dato che **LEFT** caratterizza già il fatto che si tratti di una **OUTER**.

Mi è capitato di constatare che alle volte il segno di eguaglianza sulla clausola **ON** ha generato confusione: non ha lo stesso significato dell'uguale di una clausola **WHERE**, che filtra tutti e soli i risultati che rispettano tale condizione; l'uguale di una clausola **ON** non ha una vera e propria funzione di filtro: lo vedrei più come un criterio che, se non viene rispettato, non provoca l'eliminazione di un record, bensì l'indicazione di un elemento inesistente, tramite il **NULL**.

Quick tip: rispetto a quanto accadeva con la **INNER**, la **LEFT** non è più simmetrica, ragion per cui l'ordine in cui scriviamo le tabelle è importante. Avessimo invertito la tabella dei libri con quella dei film, avremmo ottenuto l'elenco di tutti i film , con l'indicazione dell'anno di uscita del libro valorizzata o no a seconda della corrispondenza sulla tabella dei libri.

8.2.3 La RIGHT JOIN

La RIGHT JOIN ha esattamente la stessa funzione della sua omologa LEFT JOIN, con la differenza che vengono presi tutti i risultati della **seconda** tabella, e confrontati con la prima. A livello pratico non c'è nessuna differenza rispetto a invertire l'ordine di due tabelle in una LEFT JOIN.

Perciò scrivere in questo modo

[...]

```
FROM med.Libri li
      LEFT JOIN med.Film fi
      ON li.Titolo = fi.Titolo;
```

è equivalente a scrivere in questo modo

[...]

```
FROM med.Film fi
      RIGHT JOIN med.Libri li
      ON li.Titolo = fi.Titolo;
```

Tale operazione potrebbe perciò essere rappresentata graficamente come:

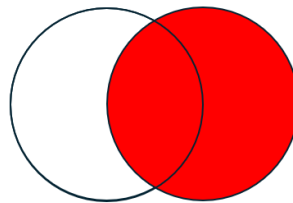


Figura 8.3: Rappresentazione grafica della INNER JOIN

Se infine eseguiamo la query

```
SELECT fi.Titolo,
       li.AnnoDiUscita AS AnnoLibro,
       fi.AnnoDiUscita AS AnnoFilm
FROM med.Libri li
```

```
RIGHT JOIN med.Film fi
ON li.Titolo = fi.Titolo;
```

otterremo come risultato

Titolo	AnnoLibro	AnnoFilm
It	1986	1990
2001: Odissea nello spazio	1968	1968
Dune	1965	2022
Interstellar	NULL	2014
Harry Potter	1997	2001
I pirati dei Caraibi	NULL	2003
Il Signore degli Anelli	1954	2003

ovvero tutti i film della tabella `med.Film`, indipendentemente dal fatto che siano tratti o meno da un libro.

8.2.4 La FULL JOIN

Per ottenere tutti i record di entrambe le tabelle possiamo far ricorso alla `FULL JOIN`, rappresentata in questo modo:

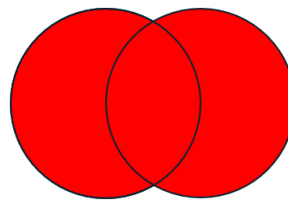


Figura 8.4: Rappresentazione grafica della `FULL JOIN`

Scrivendo in questo modo

```
SELECT COALESCE(li.Titolo, fi.Titolo) AS Titolo,
       li.AnnoDiUscita AS AnnoLibro,
       fi.AnnoDiUscita AS AnnoFilm
```

```

FROM med.Libri li
FULL JOIN med.Film fi
ON li.Titolo = fi.Titolo;

```

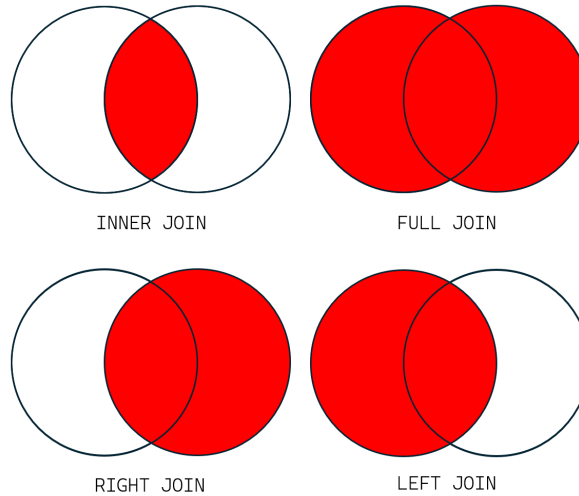
otterremo

Titolo	AnnoLibro	AnnoFilm
Il Signore degli Anelli	1954	2003
Harry Potter	1997	2001
La montagna incantata	1924	NULL
Cent'anni di solitudine	1967	NULL
It	1986	1990
Dune	1965	2022
Il giovane Holden	1951	NULL
2001: Odissea nello spazio	1968	1968
Il pendolo di Foucault	1988	NULL
I pirati dei Caraibi	NULL	2003
Interstellar	NULL	2014

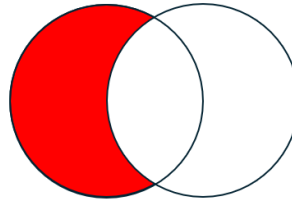
Notiamo l'utilizzo della funzione `COALESCE`, che ci permette di visualizzare sulla stessa colonna i titoli dei libri, oppure dei film se il libro non dovesse esistere (Es. I pirati dei Caraibi e Interstellar).

8.2.5 Altri tipi di Join

Finora abbiamo visto questi tipi di join



, ma ci sono ancora diversi modi in cui possiamo definire le aree di questi due insiemi che non sono stati ancora trattati. Cominciamo da questo:



Volessimo esprimerlo a parole, potremmo dire che si tratta dell'insieme dei libri da cui non è stato mai tratto nessun film. Non abbiamo un comando specifico in T-SQL per esprimere una condizione del genere, per cui dovremo aiutarci con la clausola **WHERE**, in questo modo:

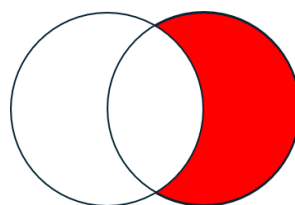
```
SELECT li.Titolo,
       li.AnnoDiUscita AS AnnoLibro,
       fi.AnnoDiUscita AS AnnoFilm
FROM med.Libri li
     LEFT JOIN med.Film fi
       ON li.Titolo = fi.Titolo
WHERE fi.Titolo IS NULL;
```

ottenendo

Titolo	AnnoLibro	AnnoFilm
La montagna incantata	1924	NULL
Cent'anni di solitudine	1967	NULL
Il giovane Holden	1951	NULL
Il pendolo di Foucault	1988	NULL

Anche se in questo particolare caso avessimo posto la condizione non su `fi.Titolo` `IS NULL` ma su un'altra colonna della seconda tabella, ad es `fi.AnnoDiUscita` `IS NULL`, avremmo ottenuto lo stesso risultato, ma il consiglio è sempre quello di porre la condizione `IS NULL` sul campo su cui si è imposta la join, dato che un campo diverso potrebbe essere nullable e portare a un risultato errato. Per esempio se l'anno di uscita del film del Signore degli Anelli non fosse noto (e quindi il campo corrispondente fosse `NULL`), avremmo avuto un falso positivo.

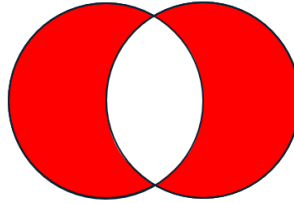
In modo del tutto simile, possiamo rovesciare la query per ottenere



, cioè, molto banalmente

```
SELECT fi.Titolo,
       li.AnnoDiUscita AS AnnoLibro,
       fi.AnnoDiUscita AS AnnoFilm
FROM med.Libri li
      RIGHT JOIN med.Film fi
      ON li.Titolo = fi.Titolo
WHERE li.Titolo IS NULL;
```

L'ultimo caso che ancora dobbiamo considerare è quello con questa rappresentazione insiemistica



, in pratica l'elenco di tutti i libri da cui non sono stati tratti film e tutti i film da cui non sono stati tratti libri.

Potremo esprimere questa richiesta scrivendo la seguente query

```
SELECT fi.Titolo,  
       li.AnnoDiUscita AS AnnoLibro,  
       fi.AnnoDiUscita AS AnnoFilm  
FROM med.Libri li  
      FULL JOIN med.Film fi  
      ON li.Titolo = fi.Titolo  
WHERE li.Titolo IS NULL OR fi.Titolo IS NULL;
```

da cui otteniamo

Titolo	AnnoLibro	AnnoFilm
La montagna incantata	1924	NULL
Cent'anni di solitudine	1967	NULL
Il giovane Holden	1951	NULL
Il pendolo di Foucault	1988	NULL
I pirati dei Caraibi	NULL	2003
Interstellar	NULL	2014

8.2.6 La CROSS JOIN

Ci resta ancora da vedere la CROSS JOIN, che esprime semplicemente un prodotto cartesiano tra gli elementi della prima e della seconda tabella.

Prendiamo ad esempio due semplicissime tabelle da una colonna e due righe,

Tabella 8.3: Tabella `dbo.Tab1`

ID1
1
2

e

Tabella 8.4: Tabella `dbo.Tab2`

ID2
A
B

Se scriviamo in questo modo:

```
SELECT t1.ID1,  
       t2.ID2  
FROM   dbo.Tab1 t1  
       CROSS JOIN dbo.Tab2 t2;
```

otterremo questo risultato

ID1	ID2
1	A
2	A
1	B
2	B

cioè tutte le combinazioni possibili tra i record di una tabella e i record dell'altra.

Quick tip: Notare come per la `CROSS JOIN` non sia necessaria la condizione di `ON` sui campi

Lo stesso risultato avremmo potuto ottenerlo in questo modo

```

SELECT t1.ID1,
       t2.ID2
FROM @TAB1 t1
      INNER JOIN @TAB2 t2
      ON      t1.ID1 = t1.ID1

```

ovvero mettendo nella condizione di ON lo stesso campo ai due capi dell'uguaglianza ma il mio consiglio, quando si vuole fare un prodotto cartesiano, è sempre di utilizzare il comando CROSS JOIN.

8.3 Le subqueries - altri modi di combinare tabelle senza JOIN

T-SQL consente l'utilizzo di query innestate, ovvero siamo in grado, all'interno di una query, di fare riferimento al set di dati esposto da un'altra query.

8.3.1 Self-contained Subqueries

In questo caso abbiamo una query "interna" che non dipende dalla query "esterna", e pertanto può essere eseguita in modo indipendente. Come ormai abbiamo imparato bene, il risultato di una query può dare origine a un singolo valore (ovvero una tabella di una sola riga e una sola colonna), un elenco di dati (ovvero una tabella più righe ma una sola colonna), e addirittura una tabella vera e propria (ovvero una tabella di più righe e più colonne). Ciascuno di questi risultati può diventare il riferimento per un'altra query.

Supponiamo di voler conoscere il titolo del film (o i film) uscito più recentemente; possiamo cominciare con questa semplicissima query:

```

SELECT MAX(AnnoDiUscita)
FROM med.Film;

```

A questo punto basterà imporre ad un'altra query (sempre sulla `med.Film`), che l'anno di uscita sia uguale al risultato della query precedente, ossia:


```

SELECT Titolo
FROM med.Film
WHERE AnnoDiUscita =
    (
        SELECT MAX(AnnoDiUscita)
        FROM med.Film
    );

```

ottenendo "Dune" come output. In questo caso, avendo utilizzato nella sottoquery una funzione di raggruppamento senza alcuna **GROUP BY**, avevamo la garanzia che il risultato fosse un solo valore, ragion per cui abbiamo potuto utilizzare l'uguale per mettere a confronto l'anno di uscita della subquery con quello della tabella.

Allo stesso modo, se consideriamo una **SELECT** normale, la condizione **IN** all'interno di una clausola **WHERE**, è seguita da una lista di valori: questi valori possono essere espressi sia come un elenco separato da virgola, sia come risultati di una **SELECT** di una sola colonna.

Supponiamo allora di avere una tabella separata, chiamata **med.VecchiFilm**, con una sola colonna, **med.TitoloFilm**, in cui abbiamo inserito le due pellicole uscite prima del 2000, 'It' e '2001: Odissea nello spazio'.

VecchiFilm
It
2001: Odissea nello spazio

una query scritta in questo modo

```

SELECT Titolo,
        AnnoDiUscita
FROM med.Film
WHERE Titolo IN ( 'It', '2001: Odissea nello spazio' );

```

restituirà gli stessi risultati di

```

SELECT Titolo,
        AnnoDiUscita
FROM med.Film
WHERE Titolo IN
(
    SELECT TitoloFilm
    FROM med.VecchiFilm
);

```

Quick tip: Notare che i nomi della colonna nella WHERE e di quella tirata fuori dalla SELECT nella IN non devono essere necessariamente uguali.

8.3.2 Subquery Correlate

Questo particolare tipo di subquery è leggermente più complesso rispetto a quello visto in precedenza, dato che la query interna non può essere eseguita indipendentemente, contenendo infatti dei riferimenti a quella esterna.

La richiesta che vogliamo soddisfare è la seguente: vogliamo una query in cui vedere titolo e anno di pubblicazione dei libri da cui sono stati tratti film negli anni dal 2000 al 2009. La possiamo scrivere in questo modo (anche se naturalmente non è l'unico)

```

SELECT Titolo
        , AnnoDiUscita
FROM med.Libri AS li
WHERE EXISTS (
    SELECT *
    FROM med.Film AS fi
    WHERE li.Titolo          = fi.Titolo
        AND AnnoDiUscita BETWEEN 2000 AND 2009
);

```

come risultato, troveremo

Titolo	AnnoDiUscita
Il Signore degli Anelli	1954
Harry Potter	1997

Il predicato **EXISTS** accetta una sottoquery come input e restituisce **true** quando la sottoquery restituisce almeno una riga e **false** in caso contrario. In questo caso, la sottoquery restituisce i film il cui titolo è uguale al titolo nella riga esterna (la correlazione) e dove l'anno di uscita è negli anni 2000. Quindi la query esterna restituisce un titolo solo se c'è almeno un film uscito nel periodo in questione. Come predicato, **EXISTS** non ha bisogno di restituire il set di risultati della sottoquery; piuttosto, restituisce solo **true** o **false**, a seconda che la sottoquery restituisca delle righe.

Ovviamente è anche consentito utilizzare il **NOT EXISTS**, che darà tutti i risultati non forniti dalla query precedente (compresi i libri che non hanno corrispondenza nella tabella dei film)

```

SELECT Titolo
      , AnnoDiUscita
FROM med.Libri AS li
WHERE NOT EXISTS (
    SELECT *
    FROM med.Film AS fi
    WHERE li.Titolo      = fi.Titolo
      AND AnnoDiUscita BETWEEN 2000 AND 2009
);

```

8.3.3 Le Derived Tables

Le Derived Tables (tabelle derivate) sono sottoquery temporanee definite all'interno di una query principale e utilizzate come se fossero tabelle.

Titolo	AnnoDiUscita
La montagna incantata	1924
Cent'anni di solitudine	1967
It	1986
Dune	1965
Il giovane Holden	1951
2001: Odissea nello spazio	1968
Il pendolo di Foucault	1988

Riprendiamo a questo proposito l'esempio del paragrafo 8.2.5 in cui abbiamo cercato di ricavare tutti i libri da cui non sono mai stati ricavati dei film e tutti i film da cui non sono mai stati ricavati dei libri. La query trovata in quell'occasione andava benissimo, ma possiamo provare a rifarla in questo modo.

```

SELECT ISNULL(T.TitoloLibro, T.TitoloFilm) AS Titolo
      , T.AnnoLibro
      , T.AnnoFilm
FROM (
        SELECT  li.Titolo                AS TitoloLibro
              , fi.Titolo                AS TitoloFilm
              , li.AnnoDiUscita AS AnnoLibro
              , fi.AnnoDiUscita AS AnnoFilm
        FROM med.Libri li
        FULL OUTER JOIN med.Film fi
              ON li.Titolo = fi.Titolo
      ) AS T
WHERE T.AnnoLibro IS NULL
      OR T.AnnoFilm IS NULL;

```

In realtà con questo specifico caso ci siamo complicati un po' la vita, dato che la query

originale faceva lo stesso lavoro con meno righe di codice, ma, appunto, prendiamolo come un semplice esempio.

Una sottoquery può essere utilizzata nella **FROM** se e solo se soddisfa i requisiti per essere una buona tabella all'interno dello schema relazionale, il che significa che

- Tutte le colonne devono avere un nome
- Tutte le colonne devono avere un nome distinto dalle altre colonne
- Non può essere presente una clausola **ORDER BY** (perché i set non sono ordinati)

Quick tip: Se vogliamo bypassare per qualche motivo la regola che ci impedisce di ordinare un set di dati in determinate situazioni (questa, o le CTE che vedremo nel prossimo capitolo, possiamo utilizzare l'escamotage di scrivere la **SELECT** in questo modo:

```
SELECT TOP 100 PERCENT
```

brutto ed efficace al tempo stesso.

8.4 Le Common Table Expressions

Benché utili in certi contesti, le tabelle derivate presentano alcune problematiche:

- Se la derived table contengono sottoquery elaborate o sono nidificate, il codice può diventare difficile da leggere e mantenere
- Se la derived table elabora grandi quantità di dati, il consumo di memoria e CPU può aumentare significativamente
- Una derived table esiste solo all'interno della query in cui è definita, il che significa che non può essere riutilizzata in altre query o contesti senza ridefinirla

Le CTE (Common Table Expressions), insieme alle viste o alle tabelle temporanee, che vedremo più avanti, possono in parte risolvere queste problematiche, costituendo perciò un'alternativa valida in molte occasioni.

Le CTE sono una funzionalità potente e versatile di SQL Server che consente di definire una query temporanea, nominata, utilizzabile all'interno di una query principale. Introdotte per semplificare la leggibilità e la manutenzione del codice, le CTE sono particolarmente utili per risolvere problemi complessi, come la gestione di dati gerarchici o l'organizzazione di query con calcoli ricorsivi.

In SSMS, una CTE può essere definita con la parola chiave **WITH**, seguita dal nome della CTE e dalla relativa query. La sintassi chiara e modulare delle CTE aiuta a separare la logica in blocchi comprensibili, migliorando la comprensione del codice anche in scenari complessi.

Grazie alla loro flessibilità, le CTE trovano impiego in diverse situazioni, come:

- Suddividere query complesse in più passaggi. Sostituire le subquery o le derived tables per migliorare la leggibilità.
- Creare calcoli ricorsivi, ad esempio per gestire alberi gerarchici o sequenze numeriche.

Supponiamo di voler ricavare dalle nostre tabelle `med.Libri` e `med.film` il titolo più recente dei libri senza film, includendo però tra questi titoli anche "Il Signore degli Anelli". Sfruttando la CTE possiamo cominciare a definire una prima query, in cui tiriamo fuori semplicemente l'elenco dei titoli e dell'anno di uscita dei libri che soddisfano la richiesta, senza spingerci a indagare quale di questi sia il più recente.

La query seguente, quindi

```
SELECT li.Titolo
       , li.AnnoDiUscita
FROM med.Libri      li
LEFT JOIN med.Film fi
  ON fi.Titolo = li.Titolo
WHERE fi.Titolo IS NULL
      OR li.Titolo = 'Il Signore degli Anelli'
```

restituisce questi record

Titolo	AnnoDiUscita
La montagna incantata	1924
Cent'anni di solitudine	1967
Il Signore degli Anelli	1937
Il giovane Holden	1951
Il pendolo di Foucault	1988

Questa query costituirà la nostra CTE, e la query finale assumerà questo aspetto:

```
WITH CTE_Libri
AS (SELECT li.Titolo
      , li.AnnoDiUscita
      FROM med.Libri      li
      LEFT JOIN med.Film fi
            ON fi.Titolo = li.Titolo
      WHERE fi.Titolo IS NULL
            OR li.Titolo = 'Il Signore degli Anelli')
SELECT TOP (1) WITH TIES
      cte.Titolo
      , cte.AnnoDiUscita
FROM CTE_Libri cte
ORDER BY cte.AnnoDiUscita DESC;
```

Il prefisso CTE_ nell'alias non è assolutamente obbligatorio, ma può essere utile per identificare subito la nostra CTE.

Una delle funzioni più utili della CTE è che la si può richiamare più volte all'interno della stessa query (ma solo in quella). Per esempio, la stessa query di prima si potrebbe riscrivere anche in questo modo:

```
-- Definizione della CTE
```

```

WITH CTE_Libri AS (
    SELECT
        li.Titolo,
        li.AnnoDiUscita
    FROM med.Libri li
    LEFT JOIN med.Film fi
        ON fi.Titolo = li.Titolo
    WHERE fi.Titolo IS NULL
        OR li.Titolo = 'Il Signore degli Anelli'
)

```

-- Query principale

```

SELECT
    cte1.Titolo,
    cte1.AnnoDiUscita
FROM CTE_Libri cte1
WHERE cte1.AnnoDiUscita = (
    SELECT MAX(cte2.AnnoDiUscita)
    FROM CTE_Libri cte2
)

```

Come è facile vedere, abbiamo interrogato due volte la CTE CTE_Libri, cosa che non avremmo potuto fare con una tabella derivata.

Un altro aspetto interessante delle CTE è che ne possiamo definire diverse all'interno della stessa query. Nel caso avessimo voluto estrarre non solo il titolo più recente, ma anche quello più vecchio, ci sarebbe bastato scrivere in questo modo:

```

WITH CTE_Libri
AS (SELECT li.Titolo
      , li.AnnoDiUscita
    FROM med.Libri li

```



```

LEFT JOIN med.Film fi
    ON fi.Titolo = li.Titolo
WHERE fi.Titolo IS NULL
    OR li.Titolo = 'Il Signore degli Anelli')
, CTE_Min
AS (SELECT TOP (1) WITH TIES
    'Libro più recente' AS Categoria
    , cte.Titolo
    , cte.AnnoDiUscita
FROM CTE_Libri cte
ORDER BY cte.AnnoDiUscita DESC)
, CTE_Max
AS (SELECT TOP (1) WITH TIES
    'Libro più vecchio' AS Categoria
    , cte.Titolo
    , cte.AnnoDiUscita
FROM CTE_Libri cte
ORDER BY cte.AnnoDiUscita ASC)
SELECT Categoria
    , Titolo
    , AnnoDiUscita
FROM CTE_Min
UNION ALL
SELECT Categoria
    , Titolo
    , AnnoDiUscita
FROM CTE_Max;

```

che restituisce come risultato

ovvero, proprio ciò che volevamo ottenere.

Categoria	Titolo	AnnoDiUscita
Libro più recente	La montagna incantata	1924
Libro più vecchio	Il pendolo di Foucault	1988

Quick tip: Abbiamo visto come la parola che caratterizza la definizione di una CTE sia `WITH`, ma proprio nella prima query del nostro esempio abbiamo anche usato la sintassi `WITH TIES`. Non c'è rischio che l'utilizzo della stessa parola per due scopi diversi generi confusione? Per il momento lasciamo la questione in sospeso e proviamo ad eseguire la seguente, semplicissima interrogazione:

```
SELECT 1

WITH CTE_Libri
AS (SELECT li.Titolo
     FROM med.Libri      li)

SELECT Titolo
FROM CTE_Libri;
```

Nonostante la sua semplicità, va in errore:

Msg 319, Level 15, State 1, Line 3

Incorrect syntax near the keyword 'with'.

If this statement is a common table expression, an xmlnamespaces clause or a change tracking context clause, the previous statement must be terminated with a semicolon.

Come mai questo comportamento? Proprio per la ragione che dicevamo prima. La definizione di una CTE deve necessariamente essere la prima di una query, ma come facciamo se prima dobbiamo eseguire un altro comando? La risposta sta nelle norme per il buon codice elencate all'inizio di queste dispense (paragrafo 2.1), una delle quali raccomandava di aggiungere sempre (benché non richiesto

esplicitamente dalla sintassi) un punto e virgola alla fine di ogni comando. In questo modo SQL Server capisce che si tratta di comandi separati e non dà più nessun problema. Per questo motivo, per non incorrere in rischi, spesso si vedono delle CTE venire definite cominciando col punto e virgola, es.:

```
;WITH CTE_Libri
AS (SELECT li.Titolo
      FROM med.Libri      li)
SELECT Titolo
FROM CTE_Libri;
```

8.4.1 Le CTE ricorsive

Tra le molte proprietà delle CTE c'è anche quella che permette ad una CTE di riferirsi a sé stessa, possiamo infatti scrivere

```
;WITH CTE_Numbers AS
(
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1
    FROM CTE_Numbers
    WHERE n + 1 <= 5
)
SELECT n
FROM CTE_Numbers
```

ottenendo

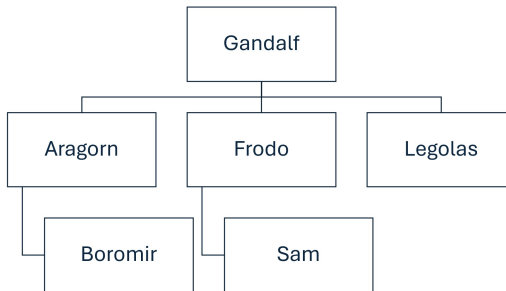
(La sintassi `SELECT n = 1` equivale a un'inizializzazione)

Le CTE ricorsive sono molto utili anche nel caso in cui dobbiamo scrivere una query che coinvolge un rapporto gerarchico.

n

1
2
3
4
5

Supponiamo di avere una struttura simile, e volerla rappresentare su una tabella, `LOTRCharacters`



Avremo una cosa del genere:

ID	Name	BossID
1	Gandalf	NULL
2	Aragorn	1
3	Frodo	1
4	Sam	3
5	Boromir	2
6	Legolas	1

Dove la colonna `BossID` si riferisce all'ID della persona gerarchicamente superiore (ad esempio Sam dipende da Frodo, e Gandalf non dipende da nessuno).

Ora il mio scopo è quello di mostrare a quale livello appartiene ciascuno, dove Gandalf sarà l'unico ad avere livello 1, al livello 2 compariranno tutte le persone che dipendono direttamente da lui e così via.

Potrò scrivere la query in questo modo

```
; WITH CTE_Hierarchy AS
(
    -- Anchor
    SELECT    ID
              ,Name
              ,BossID
              , 1 as nLevel
    FROM LOTRCharacters
    WHERE BossID IS NULL

    UNION ALL

    -- Recursive query
    SELECT    E.ID
              ,E.Name
              ,E.BossID
              ,H.nLevel+1
    FROM LOTRCharacters  E
         INNER JOIN CTE_Hierarchy  H ON E.BossID = H.ID
)

SELECT *
FROM Hierarchy
ORDER BY nLevel
```

La select dell'Anchor tira fuori tutti i nominativi di quelle persone che non hanno capo, quindi sono al vertice della piramide, e a loro viene associato il livello 1 (Gandalf).

La parte ricorsiva della CTE lega ciascun personaggio al suo superiore, incrementando di uno ad ogni livello; difatti, Aragorn e Legolas, che stanno sotto Gandalf, hanno livello 2, mentre Sam, che sta sotto Frodo, ha livello 3.

ID	Name	BossID	nLevel
1	Gandalf	NULL	1
2	Aragorn	1	2
3	Frodo	1	2
4	Sam	3	2
5	Boromir	2	3
6	Legolas	1	3

8.5 Le APPLY

SQL Server fornisce due operatori potenti per lavorare con funzioni di riga e subquery correlate: **CROSS APPLY** e **OUTER APPLY**. Questi operatori sono simili alle **JOIN**, ma con una maggiore flessibilità nella gestione dei dati e nel filtraggio dei risultati.

8.5.1 La CROSS APPLY

CROSS APPLY è un operatore che funziona come una **INNER JOIN**, restituendo solo i record per cui la sottoquery o funzione correlata restituisce almeno una riga.

Le principali caratteristiche della **CROSS APPLY** sono le seguenti:

- Restituisce solo le righe per cui la sottoquery restituisce almeno un risultato.
- Può essere usato con funzioni di tabella o subquery che restituiscono più righe.
- Simile a una **INNER JOIN**, ma con la possibilità di passare dati riga per riga alla funzione o sottoquery.

Per fare un esempio pratico ci aiuteremo con una tabella aggiuntiva, la **med.Recensioni**, che contiene una serie di giudizi dati da vari utenti riguardo ai vari film (ci possono essere più recensioni per ciascun titolo).

La tabella delle recensioni avrà questo aspetto:

La query che dovremo scrivere avrà questo aspetto

RecensioneId	Titolo	Punteggio	TestoRecensione
1	Il Signore degli Anelli	10	Capolavoro assoluto
2	Il Signore degli Anelli	9	Ottima trasposizione
3	Dune	9	Grande adattamento
4	Dune	7	Molto fedele al libro
5	Fight Club	9	Film che lascia il segno

```

SELECT
    l.Titolo,
    l.AnnoDiUscita,
    r.Punteggio AS PunteggioMigliore,
    r.TestoRecensione AS Recensione
FROM med.Film l
CROSS APPLY (
    SELECT TOP (1) r.Punteggio, r.TestoRecensione
    FROM med.Recensioni r
    WHERE r.Titolo = l.Titolo
    ORDER BY r.Punteggio DESC
) r;

```

e il risultato sarà il seguente

Titolo	AnnoDiUscita	PunteggioMigliore	Recensione
Il Signore degli Anelli	2001	10	Capolavoro assoluto
Il Signore degli Anelli	2001	9	Ottima trasposizione
Dune	2021	9	Grande adattamento
Dune	2021	7	Molto fedele al libro

La sottoquery dentro **CROSS APPLY** cerca, per ogni libro, la recensione con il punteggio più alto. **CROSS APPLY** assicura che solo i libri con almeno una recensione vengano restituiti. Se un libro non ha recensioni, non apparirà nei risultati (proprio come una **INNER JOIN**).

8.5.2 La OUTER APPLY

La OUTER APPLY è simile alla CROSS APPLY, ma funziona come una LEFT JOIN, quindi restituisce tutti i record dalla tabella principale, anche quelli che non trovano corrispondenza nella sottoquery.

Queste sono le caratteristiche principali della OUTER APPLY:

- Restituisce tutti i record della tabella principale, anche se la sottoquery non restituisce nulla.
- Se la sottoquery non ha risultati, restituisce NULL per le colonne derivanti dalla sottoquery.
- Simile a una LEFT JOIN, ma più flessibile per gestire subquery che restituiscono più righe.

Vogliamo recuperare tutti i libri, mostrando la recensione con il punteggio più alto, se disponibile.

SELECT

```
l.Titolo,  
l.AnnoDiUscita,  
r.Punteggio AS PunteggioMigliore,  
r.TestoRecensione AS Recensione
```

FROM med.Film l

OUTER APPLY (

```
    SELECT TOP (1) r.Punteggio, r.TestoRecensione  
    FROM med.Recensioni r  
    WHERE r.Titolo = l.Titolo  
    ORDER BY r.Punteggio DESC
```

) r;

E il risultato sarà questo:

Titolo	AnnoDiUscita	PunteggioMigliore	Recensione
Il Signore degli Anelli	1937	10	Capolavoro assoluto
Harry Potter	1997	NULL	NULL
Dune	2021	9	Grande adattamento
La montagna incantata	1924	NULL	NULL
Cent'anni di solitudine	1967	NULL	NULL
...	...	...	...

Se un libro ha recensioni, selezioniamo la migliore (come nella `CROSS APPLY`). Se un libro non ha recensioni, la colonna `PunteggioMigliore` sarà `NULL`, ma il libro verrà comunque incluso nel risultato.

8.5.3 Differenze tra `CROSS APPLY` e `OUTER APPLY`

Conviene servirsi della `CROSS APPLY` se:

- Vogliamo solo i record che hanno corrispondenza nella sottoquery.
- Dobbiamo applicare una funzione di tabella o una sottoquery su ogni riga.
- Vogliamo filtrare i risultati in modo simile a una `INNER JOIN`.

Conviene servirsi della `OUTER APPLY` se:

- Vogliamo tutti i record dalla tabella principale, anche se non trovano match.
- Dobbiamo gestire valori `NULL` per i record senza corrispondenza.
- Abbiamo bisogno di un comportamento simile a una `LEFT JOIN`, ma con maggiore flessibilità.

In breve possiamo dire che `CROSS APPLY` e `OUTER APPLY` sono strumenti potenti per lavorare con funzioni di tabella e subquery correlate; `CROSS APPLY` è simile a una `INNER JOIN`: restituisce solo le righe con match. `OUTER APPLY` è invece più simile a una `LEFT JOIN`: restituisce tutte le righe della tabella principale, con `NULL` per i record senza match. Sono particolarmente utili per selezionare dati dinamici, come il primo o l'ultimo valore correlato a un record, senza dover usare complesse subquery o aggregazioni.

Capitolo 9

Rafforzare l'integrità dei dati

Nei capitoli precedenti abbiamo visto come sia possibile mettere in JOIN tra di loro due tabelle, come la `med.Film` e la `med.Libri` attraverso la colonna `Titolo`, ma non abbiamo posto nessun vincolo sul fatto che i dati provenienti dalle due colonne fossero in qualche modo coerenti o confrontabili. Alcuni dati della colonna `Titolo`, infatti, erano presenti su una tabella e non sull'altra e viceversa.

Se in questo caso i problemi erano limitati, potremmo incontrare più difficoltà nel caso di due tabelle in relazione gerarchica tra di loro. Se ho una tabella con dei numeri di telefono e una tabella di persone, è molto opportuno che ciascun numero di telefono sia effettivamente legato ad una persona, perché da solo è un'informazione un po' inutile. Allo stesso modo è importante che un dato sulla tabella dei numeri di telefono che contiene un riferimento alla tabella delle persone, punti ad una persona effettivamente presente in tabella, e non un dato inesistente.

Un altro caso da cui vorremmo salvaguardarci potrebbe essere l'inserimento in una tabella di dati duplicati, per esempio nel caso della colonna del codice fiscale in una tabella di anagrafica.

Come garantire, quindi, l'integrità dei dati sulle tabelle in SQL Server? Il modo migliore consiste nell'introduzione di vincoli (constraint), come chiavi esterne, vincoli di univocità etc., che possono essere definiti sia contestualmente alla creazione di una tabella, tramite il comando `CREATE TABLE`, oppure, se la tabella è già stata creata, oppure a posteriori, tramite il comando `ALTER TABLE`.

9.1 Le chiavi primarie

Avevamo già affrontato nella sezione 6.1.4 il concetto di **chiave primaria**, ovvero una colonna (o combinazione di colonne, anche se solitamente si privilegia l'utilizzo di una sola colonna) di cui è assicurato il valore univoco per ciascun record della tabella. Spesso è anche più conveniente, in un'ottica a lungo termine, privilegiare l'utilizzo di chiavi surrogate rispetto alle chiavi naturali, ovvero una colonna aggiuntiva con un valore numerico (solitamente intero) che non ha alcun significato ai fini delle informazioni contenute nella tabella, ma serve solamente a garantire la presenza di una colonna univoca.

La chiave primaria fa parte dei vincoli che servono per salvaguardare l'integrità dei dati: possiamo sia definirla nel modo sintetico visto nel paragrafo 6.1, ossia

```
CREATE TABLE lotr.Hobbits
(
  HobbitId INT NOT NULL PRIMARY KEY IDENTITY(1,1)
  Nome VARCHAR(50) NOT NULL,
  Cognome VARCHAR(50) NULL
);

GO
```

sia in un modo leggermente più sofisticato

```
CREATE TABLE lotr.Hobbits
(
  HobbitId INT NOT NULL IDENTITY(1,1),
  Nome VARCHAR(50) NOT NULL,
  Cognome VARCHAR(50) NULL,
  CONSTRAINT PK_lotr_Hobbits_HobbitId PRIMARY KEY(HobbitId)
);
```

, nel quale abbiamo definito non solo la chiave primaria su una determinata colonna,

ma anche il nome della chiave primaria stessa, ovvero `PK_Hobbits`. Se non specifichiamo il nome della foreign key, come nel primo caso, SQL Server gliene assegnerà uno d'ufficio, di solito composto in questo modo:

```
PK__[Nometabella]__[GUID]
```

, in questo caso, difatti, il nome generato automaticamente è

```
PK__Hobbits__E84F2555D7CCAF3
```

Verrebbe da chiedersi come mai un nome così complicato: in SQL server, tutti i constraint sulle tabelle sono oggetti del database (come tabelle, viste etc.), pertanto i loro nomi *devono essere univoci in tutto il database*. Se il nome automatico si limitasse a `PK__[Nometabella]`, potrebbero venire generate delle altre chiavi primarie chiamate allo stesso modo, dato che sappiamo essere possibile la coesistenza di più tabelle con lo stesso nome (purché su schema diversi). L'aggiunta della stringa alfanumerica al fondo ci preserva da questa eventualità, anche se, nel momento di creare una tabella sarebbe sempre meglio utilizzare il secondo metodo, che ci permette di assegnare il nome ai vincoli secondo in base ad una nostra scelta (ed eventuali naming convention aziendali), senza affidarsi alla creatività di SQL Server. La regola generale resta sempre quella di dare nomi chiari e parlanti, ad esempio possiamo prendere spunto dal nome dato in automatico da SQL Server, e chiamare sempre i vincoli di chiave primaria con il prefisso `PK_` seguito dal nome dello schema, da quello della tabella (e volendo dalla colonna). Avremo nomi forse un po' pleonastici, ma che assolvono il duplice ruolo di essere immuni da ambiguità e sicuramente privi di possibili duplicati nel nostro database. Questo spiega come mai, seguendo questa logica, abbiamo deciso che la chiave primaria della tabella `lotr.Hobbits`, imposta sulla colonna `HobbitId`, si chiami

```
PK_lotr_Hobbits_HobbitId.
```

Volendo mantenere la nomenclatura coerente, avremo che il prefisso per il nome delle chiavi esterne sarà `FK_`, degli indici `IX_`, degli unique constraint `UQ_` e così via.

In dettaglio: in SQL Server esistono molte viste di sistema che permettono di accedere ai metadati relativi al database. Questi metadati descrivono la struttura del database, incluse informazioni su tabelle, colonne, tipi di dati, chiavi, vincoli, e molto altro.

Volendo scoprire quale nome SQL server avesse assegnato automaticamente alla chiave primaria della tabella `lotr.Hobbits` prima che lo definissimo noi, una possibilità era quella di interrogare la `sys.key_constraints` che avevamo già incontrato nel paragrafo 6.1.5. In alternativa possiamo interrogare una vista dell'`INFORMATION_SCHEMA`

```
SELECT *  
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS  
WHERE TABLE_NAME = 'Hobbits';
```

Vale la pena di citare altre viste utili dell'`INFORMATION_SCHEMA`, per esempio

- `INFORMATION_SCHEMA.TABLES`: Fornisce informazioni su tutte le tabelle e viste presenti nel database.
- `INFORMATION_SCHEMA.COLUMNS`: Mostra informazioni sulle colonne di ogni tabella, come il nome della colonna, il tipo di dato, le dimensioni, ecc.
- `INFORMATION_SCHEMA.ROUTINES`: Restituisce una riga per ogni stored procedure e funzione accessibile dall'utente corrente nel database corrente.

Come ho affermato all'inizio del capitolo, possiamo anche creare una chiave primaria dopo aver creato la tabella: in questo caso potremo utilizzare il comando `ALTER TABLE`, così:

```
ALTER TABLE lotr.Hobbits  
ADD CONSTRAINT PK_lotr_Hobbits_HobbitId PRIMARY KEY(HobbitId);  
GO
```

È possibile modificare la tabella non solo dopo che è stata creata, ma anche dopo

che sono stati aggiunti dei record: ovviamente, in questo caso, i dati della colonna che viene promossa a chiave primaria devono soddisfare tutti i requisiti necessari (banalmente, avere valori univoci). Se infatti, nel nostro esempio, avessimo inserito due volte il valore "1" nella colonna `HobbitId`, e avessimo cercato di eseguire il comando in cui la promuoviamo a chiave primaria avremmo ricevuto un errore simile:

```
Msg 1505, Level 16, State 1, Line 1
```

```
The CREATE UNIQUE INDEX statement terminated because a duplicate key  
was found for the object name 'lotr.Hobbits' and the index name  
'PK_Hobbits_HobbitId'. The duplicate key value is (1).
```

```
Msg 1750, Level 16, State 1, Line 1
```

```
Could not create constraint or index. See previous errors.
```

Riceverei un errore anche qualora cercassi di aggiungere una nuova chiave primaria su una tabella con chiave primaria già definita. Nel nostro caso, eseguire questa istruzione

```
ALTER TABLE lotr.Hobbits  
ADD CONSTRAINT PK_lotr_Hobbits_Nome PRIMARY KEY(Nome);  
GO
```

che cerca di eleggere la colonna `Nome` a chiave primaria della tabella, darebbe luogo a quest'altro errore

```
Msg 1779, Level 16, State 0, Line 1
```

```
Table 'Hobbits' already has a primary key defined on it.
```

Vorrei porre l'attenzione su una distinzione sottile, che sulle prime potrebbe sfuggire: imporre due chiavi primarie su due colonne diverse *non vuol dire* imporre una chiave primaria composta da due colonne diverse, che invece è un'operazione del tutto lecita. Volessimo definire quest'ultima sulla tabella `Hobbits` (sempre dopo averla ripulita da eventuali precedenti chiavi primarie), dovremmo fare così:

```
ALTER TABLE lotr.Hobbits
ADD CONSTRAINT PK_Hobbits_HobbitId PRIMARY KEY(Nome, Cognome);
GO
```

Notare che, se volessimo definire una chiave primaria composta da due colonne combinate in fase di creazione di una tabella, non potremmo usare il "primo" metodo, quello dove scriviamo `PRIMARY KEY` di fianco ad una colonna, ma dovremmo per forza specificarlo in fondo all'elenco delle colonne

```
... Cognome VARCHAR(50) NULL,
CONSTRAINT PK_lotr_Hobbits_Nome_Cognome PRIMARY KEY(Nome, Cognome)
);
```

Quick tip: Semplifichiamoci la vita. Nei DB più chiari e puliti che abbia visto in vita mia, la colonna che ospitava la chiave primaria si chiamava sempre `[NomeTabella]Id` (o `[NomeTabella]_Id`), su tutte le tabelle.

9.2 I vincoli unique

Capita spesso che in una tabella (specie se abbiamo definito su di essa una chiave primaria surrogata), ci siano altre colonne che soddisfano un criterio di univocità, supponiamo ad esempio la colonna `CodiceFiscale` su una tabella di anagrafica di persone. In questo caso è consigliabile rafforzare la caratteristica di questa colonna, irrobustendone il concetto a livello di definizione della tabella, ovvero aggiungendo uno `unique constraint`.

```
ALTER TABLE anag.Persone
ADD CONSTRAINT UC_anag_Persone_CodiceFiscale UNIQUE (CodiceFiscale);
GO
```

In dettaglio: Una colonna su cui è stato imposto uno `unique constraint` può essere definita come `NULL`. Ne consegue che è possibile inserire un `NULL` al suo

interno, cosa impossibile con la chiave primaria. L'unica controindicazione è che l'eventuale record NULL potrà essere al massimo uno; dovessimo avere più NULL c'è comunque una via d'uscita, ovvero imporre la condizione unicamente sui valori NOT NULL, ma affronteremo la cosa nel capitolo dedicato agli indici.

Proprio come avevamo fatto per ricercare la chiave primaria dentro la tabella di sistema dei constraints, possiamo farlo anche per gli unique constraints, cambiando la condizione di WHERE.

```
SELECT *  
FROM sys.key_constraints  
WHERE type = 'UQ';
```

9.3 Le chiavi esterne

In SQL Server (e in generale nei database relazionali), una **chiave esterna** (o **foreign key**, spesso abbreviata come **FK**) è un vincolo che stabilisce una relazione tra due tabelle. Il suo scopo principale è quello di mantenere l'integrità referenziale, garantendo che i dati inseriti rimangano coerenti tra le tabelle correlate.

Per spiegare meglio questo concetto possiamo fare immaginare due tabelle, una contenente un elenco di stati e un'altra un elenco di città .

CountryId	CountryName
1	Canada
2	Italy
3	UK
4	Australia

CityId	City
1	Roma
2	Brisbane
3	Sidney
4	London
5	Edinburgh

Senza addentrarci troppo profondamente nel discorso della normalizzazione e del disegno del database, sembra un'ipotesi legittima voler collegare una città allo stato in cui si trova. Come potrei sapere, ad esempio, se la Roma nella mia anagrafica è la Roma che si trova in Italia o quella del Queensland, in Australia? O che la London che vedo è la capitale del Regno Unito e non l'omonima London che si trova nella regione dell'Ontario, in Canada?

A questo proposito possiamo creare una colonna aggiuntiva nella tabella delle città: potremmo creare una colonna testuale (a dire il vero un po' ridondante), simile a questa

CityId	City	Country
1	Roma	Italy
2	Brisbane	Australia
3	Sidney	Australia
4	London	UK
5	Edinburgh	UK

ma è molto più conveniente inserire un puntatore alla tabella originale, indicando anziché il nome per esteso del paese, un riferimento al suo Id; sarà anche meno comprensibile a degli occhi umani, ma offre indiscutibili vantaggi per quanto riguarda il risparmio di spazio; non solo: volessimo mettere in join queste tabelle, la condizione di ON che metta a confronto due campi numerici anziché testuali sarà indiscutibilmente più efficiente.

CityId	CountryId	City
1	2	Roma
2	4	Brisbane
3	4	Sidney
4	3	London
5	3	Edinburgh

Se lasciassimo le cose in questo modo senza compiere altre operazioni, però, nulla ci vieterebbe di inserire un record nella tabella delle città che contenga il riferimento ad un CountryId inesistente, nel nostro caso, ad esempio, 6. In questo caso definire una chiave esterna ci salvaguarda da questa eventualità: con essa esplicito il legame che mette in relazione la colonna CountryId della tabella geo.Countries, e la colonna CountryId della tabella geo.Cities.

Possiamo definire la foreign key tramite il comando

```
ALTER TABLE geo.Cities WITH CHECK
ADD CONSTRAINT FK_Cities_Countries_CountryId FOREIGN KEY(CountryId)
REFERENCES geo.Countries (CountryId)
```

Vediamo nel dettaglio come si compone questo comando.

Notiamo come l'istruzione **ALTER** vada eseguita sulla tabella figlia (nel nostro caso, **geo.Cities**), cioè quella che contiene il puntamento (ovvero, *referenzia*) la tabella madre (la **geo.Countries**).

WITH CHECK invece serve per specificare se controlliamo o no eventuali violazioni del vincolo sui dati già esistenti della tabella figlia. Questo vuol dire che, se prima di definire la chiave esterna, nella tabella **geo.Cities** esistevano valori di **CountryId** senza riscontro nella tabella **geo.Countries**, questi verranno lasciati come sono. Gli inserimenti successivi alla creazione della foreign key verranno al contrario sempre controllati.

Perché la creazione della chiave esterna abbia successo è importante che la colonna della tabella padre (nel nostro caso) abbia su quella tabella lo status di chiave primaria, o al massimo che abbia un vincolo di unicità (o un indice univoco), altrimenti ci comparirà questo errore

Msg 1776, Level 16, State 0, Line 8

There are no primary or candidate keys in the referenced table 'geo.Countries' that match the referencing column list in the foreign key 'FK_Cities_Countries'

Oltre a questa norma, teniamo presente che in fase di creazione di una chiave esterna, la colonna o il set di colonne di ogni tabella coinvolta deve avere esattamente gli stessi tipi di dati.

Quick tip: Non possiamo eseguire il comando **TRUNCATE** su una tabella che sia relazionata con altre tabelle da vincoli di chiave esterna.

Se proviamo a farlo, infatti, riceveremo questo messaggio:

Msg 4712, Level 16, State 1, Line 1

Cannot truncate table 'dbo.Table' because it is being referenced by a FOREIGN KEY constraint.

9.4 Altri tipi di constraint

9.4.1 Check constraint

Con un vincolo di tipo **CHECK**, si dichiara che i valori di una colonna devono rispettare una certa condizione. Anche se i valori sono già limitati dal tipo di dato della colonna, un vincolo **CHECK** aggiunge ulteriori restrizioni, come intervalli o insiemi di valori ammessi. Quando si crea un vincolo **CHECK**, si definisce un'espressione che consente a SQL Server di verificare la validità dei valori.

Se volessimo impedire l'inserimento della Svizzera nella nostra tabella delle nazioni, dovremmo definire un **CHECK** in questo modo

```
ALTER TABLE geo.Countries WITH CHECK
ADD CONSTRAINT CHK_Countries_NoSwitzerland
CHECK (Country != 'Switzerland');
GO
```

In dettaglio: l'utilizzo del **CHECK** è preferibile rispetto al **TRIGGER**, talmente evitabile che non l'ho nemmeno mai citato finora.

9.4.2 Default constraint

L'ultimo vincolo che citerò è il **DEFAULT**, che avevamo già incontrato nel paragrafo 6.1.3. Si tratta del meno restrittivo tra tutti i vincoli, dato che non impone nessuna restrizione, ma fornisce semplicemente un valore predefinito nel caso questo non venga specificato durante un'operazione di **INSERT**.

I vincoli **DEFAULT** sono particolarmente utili quando si ha una colonna **NOT NULL**, ma non si vuole impedire il successo di un'operazione di **INSERT** se non viene specificato un valore per quella colonna. Tuttavia, è possibile applicare un vincolo **DEFAULT** anche a una colonna che permette l'inserimento di **NULL**, ma si desidera che venga

inserito un valore predefinito anziché un `NULL` quando un `INSERT` non specifica un valore.

Riportiamo quindi l'esempio citato nel paragrafo 6.1.3, che inseriva stringa vuota nel caso il cognome non venisse specificato.

```
CREATE TABLE dbo.Hobbits
(
    Nome VARCHAR(50) NULL,
    Cognome VARCHAR(50) NOT NULL DEFAULT ('')
);

GO
```

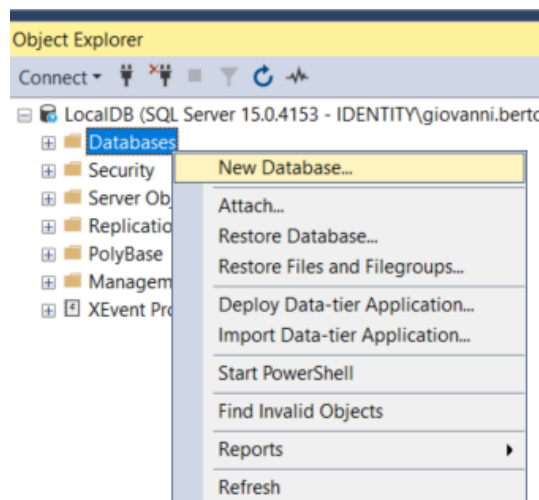
Capitolo 10

Lo stesso, ma su SSMS

Praticamente tutte le operazioni definite nei capitoli precedenti che alteravano la struttura del DB (creazione di tabelle, vincoli, indici etc.) si possono eseguire anche tramite interfaccia grafica su SQL Server Management Studio, vediamo ora come.

10.1 Creare un nuovo database

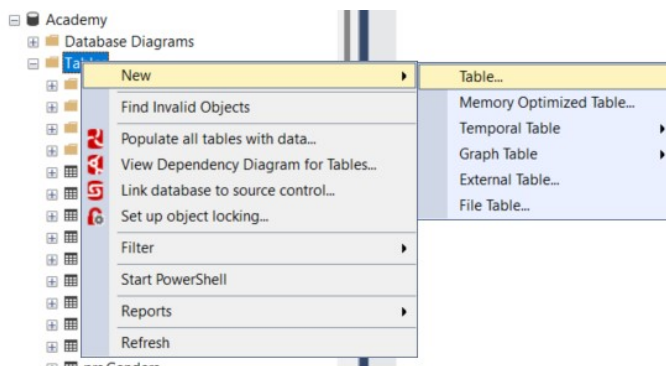
Aprendo l'object explorer, si apre il segno "+" accanto al nome dell'istanza. Basterà cliccare col tasto destro sulla cartella "Database" e scegliere "New database" dal menu a tendina.



Da qui si aprirà una finestra di dialogo che ci consentirà di scegliere le opzioni di creazione per il nuovo database.

10.2 Creazione di una nuova tabella

Una volta creato il nostro database, possiamo creare delle nuove tabelle. Anche in questo caso basterà espandere il menu del DB, scorrere fino a "Tables", cliccarci col tasto destro e scegliere "New>Table".



A questo punto si aprirà una finestra che ci consentirà di definire i nomi delle colonne, il loro tipo, e se permettono oppure no l'inserimento di NULL.

...demy - geo.Cities		...my - dbo.Hobbits	
	Column Name	Data Type	Allow Nulls
	CityId	int	☐
	CountryId	int	☐
	City	varchar(50)	☒
			☐

Sempre dalla stessa schermata potremo definire altre caratteristiche come la chiave primaria, se la colonna è identity etc., agendo direttamente sulla tab "Column properties" che si apre nella parte inferiore, dopo aver selezionato la colonna che vogliamo modificare.

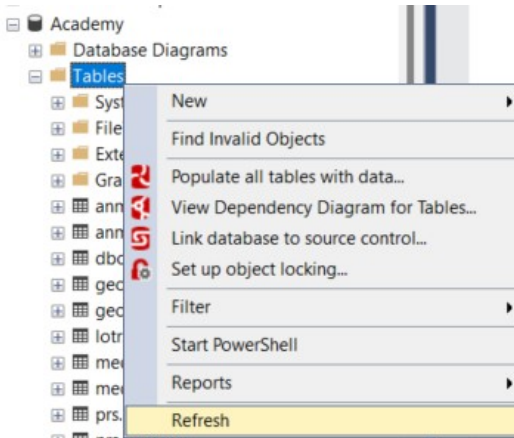
Column Name	Data Type	Allow Nulls
CityId	int	☐
CountryId	int	☐
City	varchar(50)	☒
		☐

Column Properties	
(General)	
(Name)	CityId
Allow Nulls	No
Data Type	int
Default Value or Binding	
Table Designer	
Collation	<database default>
Computed Column Specification	
Condensed Data Type	int
Description	
Deterministic	Yes
DTS-published	No
Full-text Specification	No
Has Non-SQL Server Subscriber	No
Identity Specification	Yes
(Is Identity)	Yes
Identity Increment	1
Identity Seed	1
Indexable	Yes

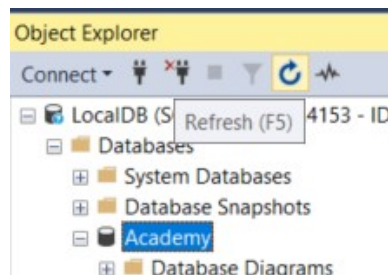
Una volta compiute tutte queste operazioni, si può cliccare sul tasto "Save" (o digitare **Ctrl+S**), e a quel punto ci verrà chiesto di inserire il nome della tabella.

Quick tip: benché molto comodo, questo metodo non è consigliabile perché non consente di scegliere lo schema della tabella o i nomi dei constraint. Si possono sempre modificare in un secondo momento, ma personalmente preferisco ordinare direttamente una pizza margherita che una pizza alla diavola senza salamino piccante.

Per poter vedere nell'elenco delle tabelle la tabella appena creata, sarà opportuno fare clic col tasto destro su "Tables" e scegliere, dal menu a tendina, "Refresh".

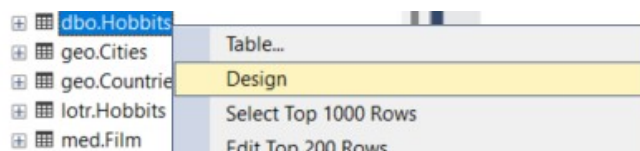


In alternativa, selezionando col tasto sinistro "Tables", occorrerà cliccare sull'icona



10.3 Modifica di una tabella

Alla stessa schermata che consente di definire le colonne di una nuova tabella e il loro tipo, si può accedere anche quando una tabella è già stata creata, ovvero cliccando su "design" dal menu che si apre cliccando col tasto destro sulla tabella che vogliamo modificare.



A questo punto comparirà una finestra dalla quale saremo in grado di cambiare le caratteristiche della nostra tabella, dal nome delle colonne, al loro tipo, al fatto che possano contenere NULL, ai valori di default e così via

Column Name	Data Type	Allow Nulls
Nome	varchar(50)	☒
Cognome	varchar(50)	☒
		☐

Column Properties

(General)

(Name)	Nome
Allow Nulls	Yes
Data Type	varchar
Default Value or Binding	
Length	50

Table Designer

Collation	<database default>
Computed Column Specification	
Condensed Data Type	varchar(50)
Description	
Deterministic	Yes

Capitolo 11

Le viste

Le viste in SQL Server sono oggetti di database che rappresentano risultati di una query memorizzata. Sostanzialmente, una query memorizzata in una vista ha il vantaggio di poter essere richiamata senza dover scrivere per esteso la query tutte le volte. Le viste in sé non contengono dati propri, ma forniscono un modo pratico e sicuro per accedere ai dati esistenti, presentandoli in un formato personalizzato. Le viste possono essere utilizzate per semplificare query complesse, fornire un'interfaccia più accessibile agli utenti e aumentare la sicurezza, limitando l'accesso diretto a tabelle sensibili. Oltre a questo, le viste sono utilizzabili in join con altre viste e altre tabelle, filtrate, raggruppate e manipolate come si farebbe con una normale tabella.

Ogni vista è definita da un'istruzione **SELECT**, che può fare riferimento ad altre tabelle o ad altre viste.

```
CREATE VIEW VW_Baggins
AS
SELECT CONCAT([Nome], ' ', [Cognome]) AS NomeCognome
FROM [lotr].[Hobbits]
WHERE Cognome = 'Baggins';
```

Questo esempio rappresenta la sintassi essenziale per creare una vista, ovvero una semplice **SELECT**, che viene battezzata "VW_Baggins", dato che mostra tutti i record della tabella **Hobbits** che hanno **Cognome = 'Baggins'**. Non è obbligatorio inserire

il prefisso `VW_`, ma può essere utile per distinguere subito una vista da una normale tabella: come dicevamo in precedenza, infatti, le due entità si possono trattare esattamente nello stesso modo; non solo: essendo le viste degli oggetti di database, la combinazione di schema e nome deve essere necessariamente diversa da quello di ciascun altro oggetto del database (quindi non solo tabelle ma anche procedure, funzioni, sinonimi etc.).

Esattamente come faremmo per interrogare una tabella, possiamo interrogare una vista utilizzando una `SELECT`

```
SELECT NomeCognome
FROM dbo.VW_Baggins
```

In modo del tutto simile a quanto accade con le tabelle, anche le viste appartengono ad uno schema; se questo non è specificato in fase di creazione, la vista verrà assegnata automaticamente allo schema `dbo`.

Esattamente come avviene in una tabella normale, inoltre, è obbligatorio che ciascuna colonna abbia un nome: nel nostro esempio, se avessimo ommesso l'alias nella colonna `NomeCognome`, avremmo ottenuto questo messaggio d'errore:

```
Msg 4511, Level 16, State 1, Procedure Baggins, Line 3 [Batch Start
Line 0]
```

```
Create View or Function failed because no column name was specified
for column 1.
```

Come metodo alternativo (ma di lettura più ostica), consiste nello specificare i nomi delle colonne subito dopo il nome della vista, in questo modo:

```
CREATE VIEW VW_Baggins(NomeCognome)
AS
SELECT CONCAT([Nome], ' ', [Cognome]) --no alias
FROM [lotr].[Hobbits]
WHERE Cognome = 'Baggins';
```

Va da sé che nel caso di una vista molto complessa, con tante colonne, può non essere immediato trovare la corrispondenza delle colonne dichiarate in testa con il loro corrispettivo nella query.

11.1 L'opzione WITH SCHEMABINDING

Ma cosa succederebbe se la struttura di una tabella (magari il nome o il tipo di una colonna) utilizzata in una vista dovesse cambiare? Nel peggiore dei casi un cambiamento del genere potrebbe rendere la vista inutilizzabile. Per evitare una situazione del genere ci viene in aiuto l'opzione `WITH SCHEMABINDING`, utilizzata in fase di creazione della vista, che impedisce alle tabelle coinvolte nella vista stessa di subire dei cambiamenti strutturali. La creazione della vista diventerebbe quindi

```
CREATE VIEW VW_Baggins
WITH SCHEMABINDING
AS
SELECT CONCAT([Nome], ' ', [Cognome]) AS NomeCognome
FROM [...]
```

Se a questo punto provassimo a modificare il tipo della colonna `Cognome` sulla tabella `[lotr].[Hobbits]`, in questo modo:

```
ALTER TABLE dbo.Hobbits
ALTER COLUMN Cognome VARCHAR(200)
```

avremmo in risposta questo errore:

```
Msg 5074, Level 16, State 1, Line 23
```

```
The object 'VW_Baggins' is dependent on column 'Cognome'.
```

```
Msg 4922, Level 16, State 9, Line 23
```

```
ALTER TABLE ALTER COLUMN Cognome failed because one or more objects
access this column.
```

È interessante notare che il comando di `ALTER` avrebbe esteso la lunghezza della colonna, portandola dall'originale `VARCHAR(50)` a `VARCHAR(200)`, ma SQL Server non fa nessun tipo di ragionamento: se una modifica ha effetto su una colonna utilizzata in una vista con opzione `WITH SCHEMABINDING`, l'operazione viene bloccata in ogni caso. Un'altra osservazione interessante è che, anche se la nostra vista non coinvolgesse direttamente la colonna di cui vogliamo cambiare tipo, l'operazione verrebbe impedita: dal che possiamo dedurre che l'opzione `WITH SCHEMABINDING` blocca la modifica di *tutte le colonne* su *tutte le tabelle* coinvolte nella vista su cui questa opzione è dichiarata.

11.2 UPDATE su viste

È possibile eseguire un'operazione di `UPDATE` direttamente su una vista? La risposta è: sì, è possibile, ma non è consigliabile. Le viste sono delle specie di scatole nere: senza avere la percezione diretta della tabella che stiamo modificando, rischiamo di aggiornare dei dati in modo incompleto o errato. Poniamo caso che una vista contenga al suo interno una `JOIN`, ed esponga una colonna che si chiama nello stesso modo in entrambe le tabelle. Se eseguiamo un'operazione di `UPDATE` su quella colonna tramite la vista, a meno di non aprirla, non possiamo sapere quale delle due tabelle stiamo cercando di aggiornare.

Potremmo fare errori potenzialmente disastrosi se la vista contenesse clausole come `DISTINCT`, `GROUP BY`, `HAVING`, `UNION`, o funzioni aggregate, ma in questo caso è lo stesso SQL Server che impedisce l'aggiornamento dando errore.

11.3 L'opzione WITH CHECK OPTION

Esiste un modo per impedire che l'`UPDATE` di una vista modifichi il contenuto della vista stessa? Sì, ed è l'opzione `WITH CHECK OPTION`.

Poniamo caso di creare una nuova vista in questo modo

```

CREATE VIEW lotr.VW_HobbitsLikeB
AS
SELECT Nome
        , Cognome
FROM lotr.Hobbits
WHERE Cognome LIKE 'B%'
WITH CHECK OPTION;

```

che seleziona tutti gli hobbit con cognome che comincia con la lettera B. Avendo imposto l'opzione `WITH CHECK OPTION`, se cercherò di eseguire un'operazione di `UPDATE` su questa vista che abbia come conseguenza la modifica dei risultati della vista stessa, tale opzione me lo impedirà.

Provo quindi a lanciare questo comando

```

UPDATE lotr.VW_HobbitsLikeB
SET Cognome = 'Took'
WHERE Cognome = 'Baggins';

```

, che quindi toglierebbe di fatto i due record con cognome "Baggins" dall'output della vista, riceverò questo errore:

```
Msg 550, Level 16, State 1, Line 39
```

```

The attempted insert or update failed because the target view either
specifies WITH CHECK OPTION or spans a view that specifies WITH CHECK
OPTION and one or more rows resulting from the operation did not
qualify under the CHECK OPTION constraint.

```

11.4 Restrizioni per le viste

Le viste sono degli strumenti molto utili, ma dobbiamo essere anche consapevoli delle loro limitazioni:

- In fase di creazione di una vista, non è possibile definire una clausola `ORDER BY`: una vista deve comportarsi come una tabella, e, come abbiamo detto nel paragrafo 2.4 le tabelle nei database relazionali rappresentano insiemi di righe, e come tali non hanno un ordine intrinseco. Certo, possiamo ordinare i risultati a posteriori, ma *non all'interno della definizione della vista*.
- Non si possono passare parametri alle viste
- Una vista non può creare una tabella, ovvero non si può utilizzare il comando `SELECT ... INTO` (paragrafo 6.1) in una vista

11.5 Modificare una vista

Il comando `ALTER TABLE` in SQL Server consente di modificare una tabella esistente, aggiungendo o rimuovendo colonne, cambiando tipi di dati o applicando vincoli. Tuttavia, quando lavoriamo con le viste, utilizziamo il comando `ALTER VIEW` per aggiornarle. Questo comando è utile per aggiornare la definizione di una vista senza doverla eliminare e ricreare. Ad esempio, se si desidera modificare l'alias della colonna generata dalla vista `VW_Baggins`, si può usare il seguente comando:

```
ALTER VIEW VW_Baggins(NomeCognome)
AS

    SELECT CONCAT([Nome], ' ', [Cognome]) AS Nome_e_Cognome
FROM [lotr].[Hobbits]
WHERE Cognome = 'Baggins';
```

In questo modo, la colonna risultante avrà un alias chiaro e leggibile, facilitando l'uso della vista nelle query successive.

11.6 Eliminare una vista

Quando una vista non è più necessaria, possiamo rimuoverla completamente dal database utilizzando il comando `DROP VIEW`. Questo comando elimina la definizione

della vista e il suo riferimento all'interno del database. Ad esempio, per eliminare la vista `VW_Baggins`, si utilizza:

```
IF OBJECT_ID('VW_Baggins', 'V') IS NOT NULL
DROP VIEW VW_Baggins;
```

La prima riga della query è opzionale (`IF OBJECT_ID [...]`), ma serve per salvarci da un errore, nel caso stessimo cercando di cancellare una vista inesistente.

È importante notare che la cancellazione di una vista non elimina i dati sottostanti, poiché questi sono memorizzati nella tabella di origine. La vista è solo un oggetto virtuale che rappresenta una query salvata.

Capitolo 12

Query avanzate

12.1 DML sfruttando le JOIN

12.1.1 UPDATE tramite JOIN

Supponiamo di voler aggiornare il nome di un Paese nella tabella `geo.Countries` in base al nome di una città presente nella tabella `geo.Cities`. Per esempio, vogliamo aggiornare il nome del Paese in "Italy" se contiene una città chiamata "Rome".

```
UPDATE co
SET co.Country = 'Italy'
FROM geo.Countries co
JOIN geo.Cities ci
    ON co.CountryId = ci.CountryId
WHERE ci.City = 'Rome';
```

`UPDATE co`: è il modo in cui indichiamo che stiamo aggiornando la tabella alias `co`, ovvero `geo.Countries`. `SET co.Country = 'Italy'`: è il comando che aggiorna la colonna `Country` nella tabella dei Paesi; anche se può sembrare ridondante, avendolo già citato nella prima riga, è necessario specificare nuovamente l'alias `co`.

12.1.2 DELETE tramite JOIN

Nel caso volessimo eliminare dalla tabella `geo.Cities` tutte le città appartenenti a un Paese specifico, invece? Supponiamo di voler eliminare tutte le città associate al Paese con `CountryCode = 'USA'`; il comando, molto simile a quello visto nel paragrafo precedente, dovrà essere scritto in questo modo:

```
DELETE ci
FROM geo.Cities ci
JOIN geo.Countries co
    ON ci.CountryId = co.CountryId
WHERE co.CountryCode = 'USA';
```

`DELETE ci`: indica che stiamo eliminando righe dalla tabella alias `ci`, ovvero `geo.Cities`. Alla fine dell'esecuzione, tutte le città della tabella `geo.Cities` che appartengono al Paese con codice "USA" saranno eliminate.

12.2 Uso della PIVOT

In SQL Server, le operazioni di `PIVOT` e `UNPIVOT` sono strumenti potenti per la trasformazione dei dati nelle query, che consentono di riorganizzare i dati da una struttura tabellare tradizionale in una forma più adatta a determinate esigenze di analisi e reporting. Questi operatori permettono rispettivamente di trasporre righe in colonne e trasformare colonne in righe, rendendo i dati più comprensibili e facili da manipolare in contesti specifici.

Poniamo di avere una tabella che contiene il numero di libri letti dalle stesse persone in diversi anni:

ID	NomePersona	NumeroLibriLetti	Anno
1	Alice	10	2020
4	Alice	8	2023
11	Alice	13	2025
10	Caterina	3	2024
6	Caterina	4	2021
9	Davide	6	2023
5	Davide	12	2020
2	Lorenzo	5	2021
8	Lorenzo	7	2022
7	Paolo	18	2024
3	Paolo	15	2022
12	Paolo	12	2025

Vogliamo ottenere una tabella di una sola riga in cui le colonne siano i nomi dei lettori, e i valori sottostanti corrispondano alla somma totale dei libri letti. Per fare ciò potremo utilizzare la **PIVOT**, scrivendo in questo modo:

```
SELECT 'NumeroLibri' AS PERSONE
      , [Alice]
      , [Lorenzo]
      , [Paolo]
      , [Davide]
      , [Caterina]
```

```

FROM (
    SELECT NomePersona
           , NumeroLibriLetti
    FROM PersoneLibri
) AS SourceTable
PIVOT (
    SUM(NumeroLibriLetti)
    FOR NomePersona IN
    ( [Alice],
      [Lorenzo],
      [Paolo],
      [Davide],
      [Caterina])
) AS PivotTable;

```

Per prima cosa possiamo concentrarci sulla sottoquery che seleziona le colonne `NomePersona` e `NumeroLibriLetti` dalla tabella `PersoneLibri`. Questi dati vengono passati all'operatore `PIVOT`.

L'operatore `PIVOT` trasforma i dati in modo che i nomi delle persone (`NomePersona`) diventino colonne e i valori siano la somma dei libri letti da ciascuna persona. In questo caso, `SUM(NumeroLibriLetti)` è usato per aggregare i dati.

Con il comando `FOR NomePersona IN`, invece, specifichiamo quali nomi delle persone vogliamo come colonne (ad esempio, Alice, Lorenzo, Paolo, Davide, Caterina).

Il risultato sarà questo:

PERSONE	Alice	Lorenzo	Paolo	Davide	Caterina
NumeroLibri	31	12	45	18	7

Se invece volessimo avere il numero degli anni in cui ci sono record per ciascun lettore? In questo caso ci basterà sostituire il `SUM` con una `COUNT`, che restituirà tre per Alice, due per Caterina e così via.

Per mantenere sempre lo stesso set di dati, la PIVOT ci consente anche di ottenere una tabella in cui incrociamo tutti gli anni con tutti i lettori: in questo modo avremo delle caselle con dei NULL, corrispondenti agli anni in cui non abbiamo valori per un determinato lettore, e le altre invece popolate correttamente.

Anno	Alice	Lorenzo	Paolo	Davide	Caterina
2020	10	NULL	15	12	NULL
2021	8	5	NULL	NULL	4
2022	NULL	7	7	NULL	NULL
2023	NULL	NULL	NULL	6	NULL
2024	NULL	NULL	18	NULL	3
2025	13	NULL	12	NULL	NULL

La query con cui otteniamo questo risultato è la seguente:

```

SELECT Anno, [Alice], [Lorenzo], [Paolo]
        , [Davide], [Caterina]
FROM (
        SELECT NomePersona
                , NumeroLibriLetti
                , Anno
        FROM PersoneLibri
    ) AS SourceTable
PIVOT (
        SUM(NumeroLibriLetti)
    FOR NomePersona IN (
        [Alice], [Lorenzo], [Paolo],

```

```

                                [Davide], [Caterina])
    ) AS PivotTable;

```

12.3 Uso della UNPIVOT

Esattamente come abbiamo trasformato le righe in colonne tramite la PIVOT, possiamo trasformare le colonne in righe tramite la sua omologa UNPIVOT.

se abbiamo ad esempio una tabella del genere

Nome	Cognome	Telefono	DataDiNascita	Luogo	Mail
Elio	Gerva	320123456	1990-05-15	Roma	el.ger@example.com

e volessimo avere i dettagli riga per riga,

Richiesta	Dato
Nome	Elio
Cognome	Gerva
Telefono	320123456
DataDiNascita	1990-05-15
LuogoDiNascita	Roma
Mail	el.ger@example.com

dovremo scrivere in questo modo:

```

SELECT Richiesta, Dato
FROM (
    SELECT      Nome, Cognome, Telefono, DataDiNascita,
                LuogoDiNascita, Mail
    FROM        Utenti
) T
UNPIVOT

```



```
(Dato FOR Richiesta IN (Nome, Cognome, Telefono, DataDiNascita,  
LuogoDiNascita, Mail) ) AS N
```

Bisogna prestare attenzione al fatto che il formato dei record che finiranno nella colonna “Richiesta” (Nome, LuogoDiNascita, etc.) sia lo stesso per tutti; potrebbe essere necessario eseguire una `CONVERT` per uniformarli, basta che le colonne originali siano definite con lo stesso tipo ma lunghezza diversa (es. un `NVARCHAR(10)` e un `NVARCHAR(20)` per dare questo errore:

```
Msg 8167, Level 16, State 1, Line 104
```

```
The type of column "Telefono" conflicts with the type of other  
columns specified in the UNPIVOT list.
```

Un altro elemento necessario per il corretto funzionamento della `UNPIVOT` è ricordarsi di assegnare sempre gli alias alle sottoquery, come nell’esempio.

12.4 Utilizzo non banale delle funzioni MIN e MAX

12.4.1 Come ricavare MIN e MAX da una serie di valori

Abbiamo visto nei capitoli precedenti come utilizzare le funzioni `MIN` e `MAX` per ottenere minimo e massimo da una serie di valori in una tabella, anche raggruppandoli. Ma quale sarebbe la sintassi se volessimo ricavare il massimo da una serie di valori scalari?

```
SELECT MAX(massimo)  
FROM (VALUES (1), (2), (3)) AS AllValues(massimo);
```

dove con `AllValues(massimo)` si sottintende una tabella "volante" chiamata `AllValues`, dotata di una colonna `massimo`, che è stata popolata con i valori 1, 2 e 3; il nome `AllValues` si può scegliere secondo le proprie esigenze.

12.4.2 Come ricavare MIN e MAX da una serie di colonne

Potremmo anche avere l'esigenza di ricavare il massimo non solo da una colonna, ma prendendo in considerazione più di una colonna.

Torniamo all'esempio dei libri letti in un certo periodo di tempo da diverse persone: in una tabella abbiamo un elenco di lettori e, sulle colonne, l'indicazione del numero di libri letti per gli anni dal 2021 al 2024

NomeLettore	Anno2021	Anno2022	Anno2023	Anno2024
Alice	15	10	8	13
Lorenzo	7	6	4	9
Paolo	18	22	25	19
Davide	9	7	10	12
Caterina	4	5	2	6

Per ottenere il numero massimo dei libri letti da ciascuna persona nel corso degli anni, posso scrivere questa query

```
SELECT (
    SELECT MAX(LibriLettiAnnuali)
    FROM (
        VALUES (a.Anno2021)
              , (a.Anno2022)
              , (a.Anno2023)
              , (a.Anno2024)
    ) AS AllValues (LibriLettiAnnuali)
) AS MAX_LibriLettiAnnuali
, NomeLettore
```

```
FROM LibriLettiAnnuali a;
```

il risultato è questo:

NomeLettore	MAX_LibriLettiAnnuali
Alice	15
Lorenzo	7
Paolo	25
Davide	10
Caterina	5

12.4.3 MIN e MAX raggruppati

Per restare sempre sulle funzioni MIN e MAX, potremmo avere l'esigenza di scrivere una query in cui mostriamo contemporaneamente il valore massimo e minimo per ciascun gruppo di record; riprendiamo a questo proposito i dati della tabella ??, in cui avevamo una colonna con il nome di una persona, un'altra con il numero di libri letti in un anno e nella terza l'anno corrispondente.

Il metodo classico che ci viene subito in mente è il seguente:

```
SELECT NomePersona
      , MIN(NumeroLibriLetti) AS [min]
      , MAX(NumeroLibriLetti) AS [max]
FROM PersoneLibri
GROUP BY NomePersona;
```

Ma ci sono alcuni casi in cui potrei preferire una soluzione alternativa: per esempio non voglio utilizzare una `GROUP BY`, magari perché sto interrogando un dataset di grandi dimensioni e ho bisogno di qualcosa che lavori su una finestra di dati senza operare raggruppamenti espliciti. Un altro motivo per cui posso volere utilizzare questa sintassi alternativa è che ho bisogno di includere delle colonne senza operare su di loro una `GROUP BY` esplicitamente. In questo caso posso scrivere in questo modo

```
SELECT DISTINCT
```

```
    NomePersona
```

```
, MIN(NumeroLibriLetti) OVER (PARTITION BY NomePersona) AS [min]
```

```
, MAX(NumeroLibriLetti) OVER (PARTITION BY NomePersona) AS [max]
```

```
FROM PersoneLibri;
```

ottenendo, come prima

il risultato è questo:

NomeLettore	min	max
Alice	8	13
Caterina	3	4
Davide	6	12
Lorenzo	5	7
Paolo	11	18

12.5 La funzione MERGE

La funzione `MERGE` in SQL Server consente di eseguire operazioni di inserimento (`INSERT`), aggiornamento (`UPDATE`) e cancellazione (`DELETE`) in un'unica istruzione, basandosi su una tabella di origine e una tabella di destinazione. Questo comando è particolarmente utile per la sincronizzazione dei dati tra due tabelle.

Immaginiamo di gestire una tabella **LibriAttuali** con i libri attualmente disponibili e una tabella **LibriAggiornati** con nuovi dati provenienti da un sistema esterno.

Questa è la tabella **LibriAttuali**

Id	Titolo	Autore	Prezzo
1	Il Signore degli Anelli	J.R.R. Tolkien	25.50
2	1984	George Orwell	18.00
3	Il Nome della Rosa	Umberto Eco	22.00

E questi record corrispondono alla **LibriAggiornati**

Id	Titolo	Autore	Prezzo
1	Il Signore degli Anelli	J.R.R. Tolkien	26.00
2	1984	George Orwell	18.00
3	Brave New World	Aldous Huxley	20.00

Ora utilizziamo **MERGE** per aggiornare **LibriAttuali** basandoci su **LibriAggiornati**:

```
MERGE INTO LibriAttuali AS target
USING LibriAggiornati AS source
ON target.ID = source.ID

WHEN MATCHED AND target.Prezzo <> source.Prezzo THEN
    UPDATE SET target.Prezzo = source.Prezzo

WHEN NOT MATCHED THEN
    INSERT (ID, Titolo, Autore, AnnoPubblicazione, Prezzo)
```

```
VALUES (source.ID
        , source.Titolo
        , source.Autore
        , source.AnnoPubblicazione
        , source.Prezzo)
```

```
WHEN NOT MATCHED BY SOURCE THEN
```

```
DELETE;
```

La prima parte del comando specifica (in modo simile ad una `JOIN`), quale è la colonna sulla quale stiamo confrontando i due set, che nel nostro caso corrisponde all'ID, qual è la tabella che costituisce la fonte della verità, e alla quale daremo l'alias `source`, e quella invece che modificheremo in base al contenuto della prima, ovvero la `target`.

Conclusa questa prima parte, possiamo specificare il comportamento che desideriamo nei vari casi.

In corrispondenza di `WHEN MATCHED`, abbiamo confrontato i prezzi dei vari libri comuni alle due tabelle, aggiornando la tabella `target` in caso di differenze; in questo caso il Signore degli Anelli costava 25.50 nella tabella attuale e 26.00 in quella aggiornata, ragion per cui mi aspetto che questo valore cambi sulla tabella `target`.

Se invece nella tabella `target` non esistono dei libri che invece esistono nella `source` (`WHEN NOT MATCHED`), li inseriremo. Notiamo come l'inserimento esplicito nella colonna `ID` sia possibile solamente se non abbiamo imposto che la sequenza con cui viene popolata tale colonna non sia autoincrementante.

L'ultimo caso che possiamo trattare è quello in cui la tabella `target` contenga dei record in più rispetto alla tabella di origine (`WHEN NOT MATCHED BY SOURCE`), in questo caso elimineremo i record dalla destinazione che non esistono più nella sorgente.

Alla fine delle operazioni, troveremo infatti che la tabella di destinazione `LibriAttuali`, è diventata identica alla tabella di origine `LibriAggiornati`.

Capitolo 13

Funzioni utili

13.1 Le funzioni di rango

Le funzioni di rango in SQL Server sono funzioni di finestra utilizzate per assegnare un numero sequenziale alle righe restituite da una query. Queste funzioni vengono spesso utilizzate per classificare i dati in base a un criterio specifico. Le principali funzioni di rango sono:

- `ROW_NUMBER()`
- `RANK()`
- `DENSE_RANK()`
- `NTILE(n)`

In questo capitolo, analizzeremo ciascuna di queste funzioni con esempi basati sulla tabella che avevamo già incontrato nel paragrafo 12.2:

ID	NomePersona	NumeroLibriLetti	Anno
1	Alice	10	2020
4	Alice	8	2023
11	Alice	13	2025
10	Caterina	3	2024
6	Caterina	4	2021
9	Davide	6	2023
5	Davide	12	2020
2	Lorenzo	5	2021
8	Lorenzo	7	2022
7	Paolo	18	2024
3	Paolo	15	2022
12	Paolo	12	2025

13.1.1 La funzione ROW_NUMBER

La funzione `ROW_NUMBER` assegna un numero sequenziale unico a ciascuna riga, in base all'ordinamento specificato nella clausola `ORDER BY`.

Vediamo quindi un esempio dell'utilizzo di questa funzione: lo scopo è quello di ordinare i record della tabella per numero di libri letti.

Se eseguo questa query:

```
SELECT NomePersona
        , NumeroLibriLetti
```



```

, Anno
, ROW_NUMBER() OVER (ORDER BY NumeroLibriLetti DESC) AS RowNum
FROM PersoneLibri;

```

L'output sarà il seguente

NomePersona	NumeroLibriLetti	Anno	RowNum
Paolo	18	2024	1
Paolo	15	2022	2
Alice	13	2025	3
Paolo	12	2025	4
Davide	12	2025	5
Alice	10	2020	6
...	...	...	...

La funzione `ROW_NUMBER` ha però anche delle altre caratteristiche molto utili: ci dà non solo la possibilità di ordinare un intero set di dati con un numero sequenziale, ma anche di ordinare dei sottoinsiemi all'interno del set. Poniamo caso che io voglia avere una classifica dei lettori più accaniti, ma suddivisa per anno: in questo caso potrei scrivere così

```

SELECT NomePersona
, NumeroLibriLetti
, Anno
, ROW_NUMBER() OVER (PARTITION BY Anno\
ORDER BY NumeroLibriLetti DESC) AS RowNum
FROM PersoneLibri;

```

Aggiungere la parte del `PARTITION BY Anno` ha avuto l'effetto di suddividere il nostro insieme di dati anno per anno, e in seguito abbiamo ricavato la classifica dei lettori più compulsivi di ciascun anno.

NomePersona	NumeroLibriLetti	Anno	RowNum
Davide	12	2020	1
Alice	10	2020	2
Lorenzo	5	2021	1
Caterina	4	2021	2
Paolo	15	2022	1
Lorenzo	7	2022	2
...	...	...	...

13.1.2 La funzione RANK

Riprendiamo per un attimo il primo esempio del paragrafo precedente, quello in cui abbiamo utilizzato il `ROW_NUMBER` senza `PARTITION BY`. Forse avrete notato che, nonostante Paolo e Davide abbiano letto lo stesso numero di libri, Paolo occupa in classifica un posto più alto di Davide. Questo accade perché la funzione `ROW_NUMBER` non gestisce i pareggi. Il suo scopo è quello di ordinare una lista secondo un determinato criterio, con numeri sempre crescenti. Nella colonna `RowNum` ci saranno sempre e solo numeri naturali consecutivi; se la funzione trova due record che, in base al nostro criterio di ordinamento, hanno pari dignità, sceglierà uno dei due arbitrariamente. Per questo motivo esiste la funzione `RANK`, che ha una sintassi del tutto simile alla `ROW_NUMBER`, ma se trova due (o più record) che si equivalgono, assegnerà ai due lo stesso numero: vediamo con l'esempio di prima.

```

SELECT NomePersona
       , NumeroLibriLetti
       , Anno
       , RANK() OVER (ORDER BY NumeroLibriLetti DESC) AS RowNum
FROM PersoneLibri;

```

L'output sarà il seguente

NomePersona	NumeroLibriLetti	Anno	RowNum
Paolo	18	2024	1
Paolo	15	2022	2
Alice	13	2025	3
Paolo	12	2025	4
Davide	12	2025	4
Alice	10	2020	6
...	...	...	...

Si può notare come, in questo caso, a Paolo e Davide sia stato assegnato lo stesso numero nella colonna `RowNum`.

Esattamente come accadeva per la funzione `ROW_NUMBER`, possiamo implementare la parte di `PARTITION BY` anche per la `RANK`.

13.1.3 La funzione `DENSE_RANK`

La funzione `DENSE_RANK`, infine, è estremamente simile alla `RANK`, tranne che per un particolare. Mentre la numerazione della `RANK`, dopo i due "4" assegnati a Paolo e Davide, saltava il numero 5 per dare 6 ad Alice, la `DENSE_RANK`, pur continuando

a rispettare l'eguaglianza tra Paolo e Davide, continuerà la numerazione in modo consecutivo, assegnando perciò "5" ad Alice.

La `DENSE_RANK` è come il podio del salto in alto alle olimpiadi di Tokio 2020, dove due atleti hanno vinto l'oro a pari merito, uno l'argento e uno il bronzo, per un totale di quattro atleti premiati. La `RANK`, per contro, assomiglia un po' di più a un test di ingresso a numero chiuso, dove vengono ammessi solo i cento migliori candidati.

13.1.4 Il comando `NTILE`

13.2 Come gestire la paginazione

13.2.1 `FETCH NEXT`

13.3 Riferimenti ad altre righe in una query

13.4 Il comando `LAG`

13.5 Conteggi sfasati

`CURRENT ROW UNBOUNDED PRECEDING`

13.6 Per generare una stringa univoca

`NEWID()`